The Craft: How One Engineer and Many Agents Ship Software

For the engineer who inherits this setup, and for the agents that work beside them.

Inside: the mental model, the workshop setup, the daily loop, every stage from ticket to merge, the vault method, the wins, and the falls.

Read it in order once. Then keep it open as a map. Every section points to real files in the pack.

Contents

Read Me First

- What this book is
- How to read the book
- The pack folder map
- Reading paths

The Big Idea

- One sentence
- The nine words
- How the parts connect
- The lifecycle in one line
- Five laws behind the system

The Workshop

- The harness and its config
- System prompt files
- Rules: always-on and scoped
- Guards and extensions
- Approval gates: ask-before and auto
- MCP servers and the template
- The task-folder model
- Naming of folders and branches
- Setup scripts
- Install order

The Daily Loop

- How a day flows
- The morning brief
- My PR status
- The daily work poll
- The wiki task loop
- Session hygiene
- A day, step by step

Triage: Know Before You Dig

- Two paths: Bug and Story
- Bug triage: is it real, is it ours
- The actionable-bug rules
- Ticket triage with `/triage`
- Story refinement, Session A
- The Agent Contract
- Why a story never starts without the contract
- Grilling
- When to split a ticket
- Area path and iteration path

Bootstrap: Make the Cave

- What bootstrap makes
- Bootstrap, step by step
- The branch script
- LSP and debug setup
- Repo onboarding
- What the folder looks like after

The Bug Hunt

- The whole bug path
- Step 0: the returned bug
- Step 1: reproduce at the true failure boundary
- Playwright flows
- Using the debugger
- The root cause doc
- Fix design per repo
- Live verification against the real dependency
- Commit rules
- State moves and the team message

Building a Story

- Session B in one picture
- Contract gate, then bootstrap
- Plan per repo
- One worker per repo
- Test, with a live check
- The verification gate
- Open the PR, release resources
- The state ladder for a Story

Proof: Tests and Verification

- Where does the test go
- Fail-first and the mutation check
- E2E, and why a passing mock is not proof
- Screenshots and evidence

Running Things Locally

- Why there is a reference and a tool
- The services tool and its ports
- Start order
- Port policy and the ownership rule
- Green but wrong: cross-wiring
- Teardown after use
- Debug attach

Shipping: The Pull Request Road

- The road in one picture
- Gate one: the pre-PR security gate

Opening the PR
The auto-complete trap
The team note: one Slack line, automatic
Handling review comments
Merge conflicts
Merging your own PR
After the merge: state moves and vault entry
The deployed-to-QA check
Task cleanup

Judging Others: Code Review

What review is for
Read the ticket first
Three lanes in parallel
Compile and check every claim
Writing comments
Never touch their branch
A vote is not merge authorization

Talking to Tribe

Ticket voice
PR text
Slack: draft or auto
Approval language
The writing standard (ASD-STE100) in plain words
Translating codes for humans
The daily update
Scan findings: Checkmarx and security issues

The Long Memory: Your Vault

Why a personal wiki
The eight page types
The schema is the contract
How agents reach the vault
Ingest: put a source on the wall
Meeting notes
Sync with the tracker
The task loop
Lint, log, query, comms
Who writes back: you, the scribe, two cron jobs
Set it up in six steps

Glory: What Worked

Start clean
Prove the fix
Tickets and plans as contracts
Fences that hold
Review and merge

Honest, short, plain
Memory and automation

The Fall: What Hurt

Agent behaviour (A1 to A7)
Harness and tooling (H1 to H11)
Git and worktrees (G1 to G5)
Local services (S1 to S7)
Tracker (T1 to T6)
PR and review (P1 to P8)
QA and deploy (Q1 to Q5)
Comms (C1 to C5)
Vault (V1 to V6)
Skill drift: when recipes fight
Meta-lessons
Team and process lessons
The numbers

Make It Yours

The plan at a glance
Day 1: light the fire
Week 1: one bug, end to end
Weeks 2 to 4: the PR road, review, vault
What to keep, what to adapt
The placeholder list
Open gaps worth fixing first
Grow your own skills

Dictionary

Read Me First

What this book is

What. A teaching book. It hands you one engineer's way of shipping software with AI agents.

Why. The author is leaving. The way of working lives in files, not in a head. Files can be handed over. Heads cannot.

You are the successor. You may not know the agent harness, the tracker, or the repos. That is fine. This book starts at the fire and walks you to the hunt.

The book teaches one loop. A ticket comes in. An agent does the work. A human holds the gates. A pull request (PR = a proposed code change waiting for review) goes out. The tracker tells the truth. Around the loop sit tools, folders, rules, and a long memory.

> Rock rule: if it matters, it lives in a file.

How to read the book

What. Read it twice. Once fast, once with hands on the keyboard.

Why. The first pass gives the shape. The second pass puts the shape in your fingers.

How.

1. Read "The Big Idea" in one sitting. It is short.
2. Skim "The Workshop". Set up nothing yet.
3. Read the later parts in order. Each teaches a stage of the loop.
4. Return to "The Workshop" with the pack open. Set up one piece at a time.
5. Use the dictionary at the end. Every strange word is in it.

Every section has the same shape: **What**, **Why**, **How**, **Where in pack** (real paths), sometimes an "Ugh. Fall:" (a true story of pain) and a "Rock rule:" (a one-line law).

Names in this book are roles, never people:

- **The author** built the setup. The author is leaving.
- **Tech lead** reviews code and owns repo standards.
- **QA tester** retests fixes and closes bugs.
- **Product owner** accepts finished stories.
- **The client** owns the platform. **The consultancy** employs the author.
- **The assistant** is the product the platform builds. It is not your coding agent. Your coding agent is "the agent".

The pack has no real names. Client facts are placeholders like `<PROJECT>`, `<ADO_ORG>`, `<QA_URL>`. You fill them from your own world.

The pack folder map

What. One folder, `handoff/`. Everything the author used is inside, cleaned of private facts.

Why. You copy parts into your home folder and project folder. Know which part is which.

Folder or file	What	When you need it
`README.md`	index of the pack	first look
`HANDOFF-FOR-HUMANS.pdf`	this book	now
`HANDOFF-FOR-AGENTS.md`	the same craft, written for the agent	when you wire or tune an agent
`build/`	`build.ts`, `style.css` that make the PDF	only to rebuild this book
`skills/`	recipes, one folder each with `SKILL.md`	every day
`skills/third-party/`	upstream skills copied unchanged, `CREDITS.md`	grilling, research, review
`rules/global/`, `rules/project/`	laws	day one
`commands/`	eleven slash commands	every day
`agents/`	four subagent definitions	review, scan, flows
`harness/global/`, `harness/project/`	config, prompt files, constants, advisors	day one
`harness/extensions/`	guard and tool code	day one, then rarely
`harness/mcp.template.json`	template for tool servers	day one
`scripts/`	five shell scripts	day one, then daily
`automation/`	two morning jobs, login helper, n8n folder	week one and after
`vault/`	wiki schema, templates, views, fictional examples	month one

Four facts about the map:

1. `skills/` holds about three dozen skills. The `wiki-*` ones teach the vault. The rest teach the ticket loop.
2. `skills/workflow-router/SKILL.md` is the index. Lost? Read it.
3. Third-party skills are copied as they were. Do not edit them.
4. The pack has no secrets. No `mcp.json`, no `.env`, no tokens. You make your own.

Where in pack. `README.md`, `skills/workflow-router/SKILL.md`, `skills/third-party/CREDITS.md`.

> Rock rule: the pack is a seed, not a cage. Copy it. Then make it yours.

Reading paths

What. Three paths. Pick by time.

Why. Reading everything on day one teaches nothing. Learn in layers.

Day one

Goal: start a session and read the state of the world. Change nothing.

1. Read "The Big Idea" and "The Workshop".
2. Install the harness. Follow "Install order" at the end of "The Workshop".
3. Start a session in the tasks root. Run `/pr-status`. It only reads.
4. Run `/morning`. Read the brief. Approve nothing yet.
5. Read `harness/project/RULES.md` and `harness/project/AGENTS.md` slowly.
6. Read `skills/workflow-router/SKILL.md`. It maps every recipe.

Week one

Goal: ship one small bug fix with the author's recipes.

1. Read the lifecycle parts of this book in order.
2. Pick one small Bug. Run `/bug <id>`. Watch every gate.
3. Read `skills/bug-workflow/SKILL.md` and `skills/ticket-bootstrap/SKILL.md` while the agent works.
4. Read `rules/global/BUG_FIXING.md`, the bug-diagnosis law.
5. Open the PR. Check it against `skills/pr-creation/SKILL.md`.
6. Read the merge gates in `skills/pr-merge/SKILL.md`. Merge your own PR.
7. Log every pain in `docs/PAPERCUTS.md` in the tasks root. Make the file if missing.

Month one

Goal: own the system, not only use it.

1. Run a Story through both sessions: Session A refines (`skills/story-refinement/SKILL.md`), Session B builds (`skills/story-workflow/SKILL.md`).
2. Set up the vault. Read `vault/schema.md` and `vault/examples/`.
3. Install the two morning jobs. Read `automation/README.md` first.
4. Audit for drift. Pick one fact. Search every skill for it. You will find a second copy.
5. Change one rule. Test it. Write down why. Then read the wins and falls part of this book.

> Ugh. Fall: the author changed one rule three times in five weeks. The first two versions were guesses.
> Rock rule: ask why before you edit a rule a fourth time.

The Big Idea

One sentence

What. Human holds the gates. Agent does the work.

Why. The agent is strong and fast. It also forgets, guesses, and says "done" too early. The human is slow and wise. Put each where it is strong.

Agent strong. Agent also forget. You write the recipe. Recipe called skill. Agent reads recipe each time. Recipe is a file, so it does not forget.

The human does not type the code. The human does three things:

1. Gives the ticket number.
2. Answers questions at decision points.
3. Approves what has an audience: a merge, a post in chat, a state that closes a ticket.

Everything else is the agent's job, inside fences.

The nine words

What. Nine ideas. Learn them as nine words.

Gates. Some gates are prose ("ask before you comment on a ticket"). Some are a skill step ("draft, then wait"). Some are config: the harness itself stops the call. Three layers. See "The Workshop".

Ticket folder. A folder like `BUG-12345-prompt-search-mismatch`. Inside are links to every repo, on the ticket's own branch. Plans, evidence, and review notes live there. Two tickets never share a branch. You run many tickets without branch juggling.

Skills. A markdown file with a name and description on top. The prompt carries only name and description. When a job matches, the agent reads the whole file. The prompt stays small. The author built about three dozen, one per stage.

Rules. Laws like "never touch the bare repos" and "never write scratch files to `/tmp`". Some sit in the prompt always. Some fire only when text or a tool call matches.

Guards. Code that checks the final tool call once, before it runs. Prose gets missed. A guard does not. The author moved every hard law into a guard once prose failed.

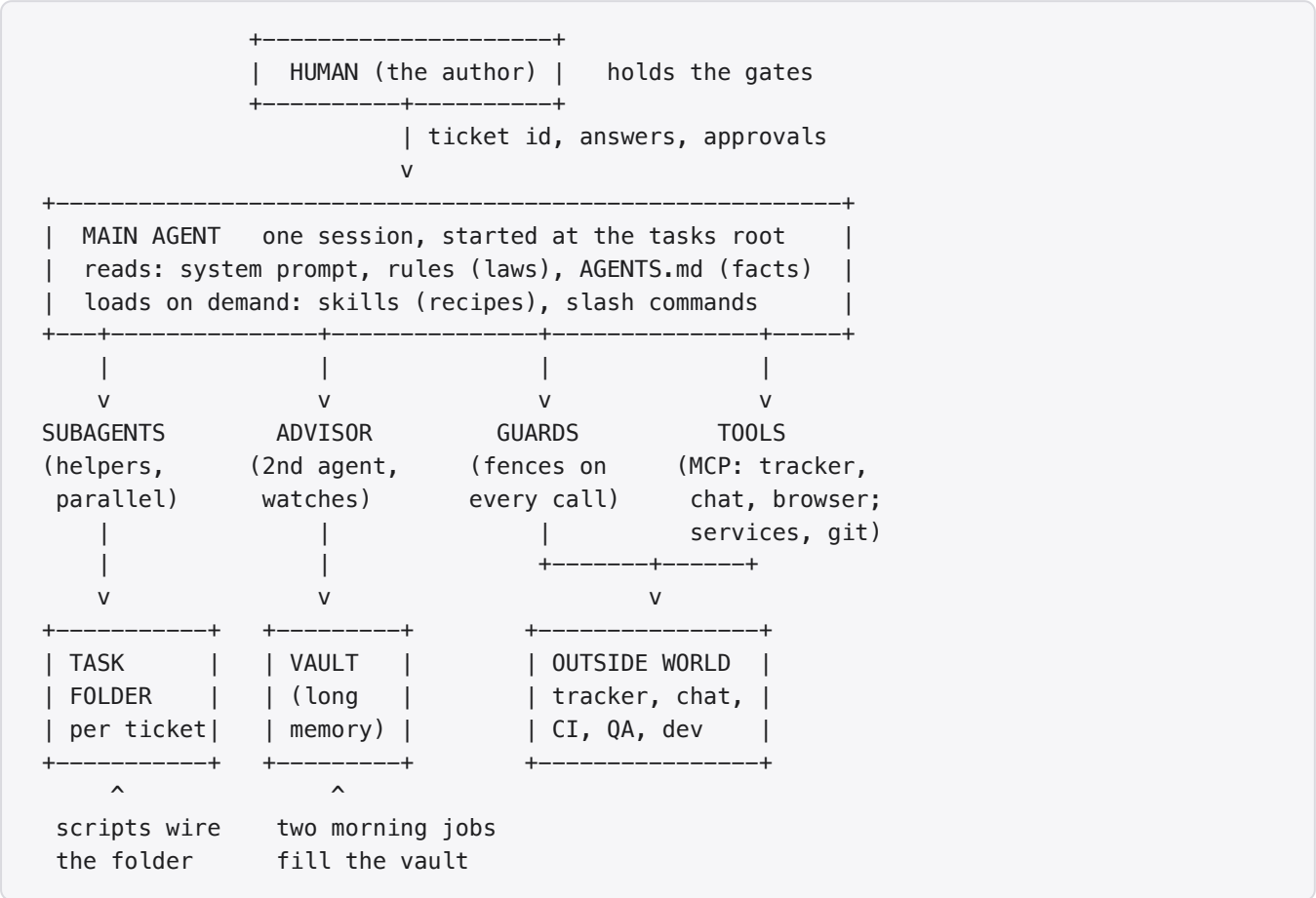
Advisor. A second model reads the work as it goes. It speaks only on a real problem. Silence means fine. Never ask it to say "looks good". That only makes the main agent re-check everything.

Subagents. Short-lived helpers: one writes code in one repo, one reviews security, one runs a browser flow, one correlates logs. They share no memory with the main agent. Write a full brief. Read their real diffs. A helper's "done" is a claim, not proof.

MCP. Model Context Protocol, a way to plug tool servers into an agent. The author used four: the tracker (Azure DevOps), the cloud (Azure), chat (Slack), and a browser (Playwright).

Vault. A personal wiki, opened as an Obsidian folder. Tasks, decisions, meetings, people, and a log. It outlives any chat.

How the parts connect



One human, one main agent. Every tool call passes a guard. Results land in the task folder, the tracker, and the vault.

The lifecycle in one line

```
triage -> bootstrap -> reproduce -> plan -> test first -> fix -> verify live
   -> security gate -> open PR -> review -> merge -> post-merge -> QA check -> cleanup
```

Bugs run in one session. Stories run in two. Session A makes the ticket a good contract (a written spec the agent trusts). Session B reads only the ticket and builds. Each stage has a skill. Each skill has gates.

Five laws behind the system

1. **Evidence over claims.** After every tracker write, re-fetch and check. A success message is not proof. A passing mock is not proof. Read the real diff.
2. **The ticket is the contract.** A thin ticket fails the gate in plain view and goes back to refinement.

3. **A check replaces a prompt.** If a question always gets "yes", make it a fact check. The agent opens the PR when the proof list is complete. It does not ask.
4. **The human owns the end.** The agent writes states up to "fixed, not yet verified" and "ready to retest". The tester closes. The product owner accepts.
5. **Guard in code what prose cannot hold.** If a rule broke once, build a fence.

Where in pack. `skills/workflow-router/SKILL.md`, `harness/project/AGENTS.md`, `harness/project/RULES.md`, `harness/global/RULES.md`, `harness/project/WATCHDOG.yml`, `harness/global/WATCHDOG.yml`.

> Ugh. Fall: a PR merged unreviewed in about fifteen seconds. The reviewer was only optional, and auto-complete was armed at once. No policy required an approval.
>
> Rock rule: the branch policy is the real gate. Prose only points at it.

The Workshop

This part sets up the fire pit. One section at a time.

One warning first. **Extensions, settings, skills, rules, and commands load when the harness starts.** Create one mid-session and nothing happens. Restart. Test in a fresh session. This cost the author hours.

The harness and its config

What. The harness is the program that runs the agent: it loads prompt, tools, skills, rules, and calls the model. The author used OMP, a terminal coding agent. The shapes fit other harnesses. The file names fit OMP.

Why. All the craft hangs on the harness. If it does not load your files, nothing works.

Two levels:

- **Global** at `~/.omp/agent/`: config, prompt additions, global rules, skills, extensions, advisors. The pack copy is `harness/global/`.
- **Project** at `<tasks-root>/.omp/`: config, rules, constants, skills, commands, agents, extensions, MCP servers, advisors. The pack copy is `harness/project/`.

How.

1. Install the harness. Check it runs.
2. Make `~/work/<PROJECT>/tasks`. This is the **tasks root**, the only place you start sessions.
3. Make `<tasks-root>/.omp/`. Copy `harness/project/*` into it. Add `skills/`, `rules/project/`, `commands/`, `agents/`, and project extensions in the matching sub-folders.
4. Copy `harness/global/*` into `~/.omp/agent/`. Add `rules/global/` and global extensions.
5. Fill every placeholder. Search for `<` to find them.
6. Restart. Start a fresh session in the tasks root. Ask the agent which skills it sees.

What the configs hold:

- `harness/global/config.yml`: model roles (planning, small jobs, advisors, auditor), advisor settings, display, default approval conditions. It turns off the "unexpected stop detector", which re-prompted finished answers up to three times.
- `harness/project/config.yml`: the project approval list, `git.enabled: false` for the status line, the `vibeExtraTools` list, and the project block of approval conditions.

Use model names you have. The pack keeps only roles: a big model for design, a mid model for scoped work, a small one for mechanical work, a small one for the auditor.

Where in pack. `harness/global/config.yml`, `harness/project/config.yml`, `harness/project/dap.json`.

> Ugh. Fall: the author edited an extension and tested it in the same session. Nothing changed. Another extension failed with "Cannot find package": the compiled harness resolves deep package imports only through a build-time map.
> Rock rule: restart, then test in a fresh headless run. Import only types from the harness package.

System prompt files

What. Text the harness adds to the system prompt. It sets the agent's manners.

Why. Rules say what to do. The prompt says how to talk, how to know, how to stop.

- `harness/global/APPEND_SYSTEM.md`: added to every prompt. Sections: Knowledge limits, Wiki, Writing standard, Communication, Session close, Context clear.
- `harness/global/TITLE_SYSTEM.md`: prompt for the session title: kebab-case, two to five words.
- `harness/global/WATCHDOG.md`: manners for every advisor. Silence means fine. A narrow scope wins.
- `harness/project/AGENTS.md`: standing **facts**: names, repos, who I am, state ladders, vocabulary, folders, environments.
- `harness/global/RULES.md`, `harness/project/RULES.md`: the always-on laws and gates.

What `APPEND_SYSTEM.md` teaches:

- **Say "I do not know".** Check a tool before you state a fact. Never invent a path, flag, number, or URL. Mark only unverified claims inline as `[INFERENCE]`.
- **Answer first.** Ten lines or fewer. Bullets. No tool narration.
- **Translate codes.** Tracker numbers for votes and states become words.
- **Drafts are exact.** Write what was dictated. Add nothing. Show drafts in a fenced block.
- **Plain English.** Simplified Technical English (ASD-STE100): short sentences, active voice, one word per meaning.
- **Close on request.** When the human says "close" or "clear", call the tool. Do not say "type /exit".

`AGENTS.md` is the home of facts. Skills point to it. They never repeat it. About half of sessions match no skill, so facts that live only in skills never load. Keep no people roster there. Roles live in vault `people/` pages.

How. Copy the three global prompt files and `AGENTS.md`. Fill placeholders. Tell the harness to read the prompt files as additions.

Where in pack. `harness/global/APPEND_SYSTEM.md`, `harness/global/TITLE_SYSTEM.md`, `harness/global/WATCHDOG.md`, `harness/project/AGENTS.md`.

> Ugh. Fall: the agent showed vote numbers and thread ids. The author complained on three separate days before a vocabulary table fixed it.
> Rock rule: one table turns tracker codes into words. Keep it in `AGENTS.md`.

Rules: always-on and scoped

What. A rule is a markdown file with a small header. Two kinds.

Why. An always-on law costs prompt space each turn. A scoped law costs nothing until its trigger fires.

- **Always-on:** header `alwaysApply: true`. Example: `rules/global/read-tool-first.md` sends reads to the `read`, `grep`, `glob` tools, never `cat` or `ls`.
- **Scoped:** header with a `condition` (regex) and a `scope` (which text or tool to watch). Example: `rules/project/ado-area-path.md` fires when the tracker write tool is called with create or update. The harness injects a reminder mid-stream. OMP calls this TTSR.

Project rules: `no-parent-folder-repo-access` (never touch bare repos or paths above the tasks folder), `tool-gotchas` (a table of tool traps), `azure-cli-fallback` (tracker work goes through MCP; the CLI is for login only), `services-via-services-tool`, `no-orphan-processes-after-use`, `teardown-verification-tooling-after-pr`, `task-files-in-ticket-folder` (scratch never in `/tmp`), `pr-thread-resolution` (never mark another person's thread resolved), `ado-ticket-voice`, `ado-area-path`, `py-runtime-assert`, `py-log-exception`.

Global rules: `read-tool-first`, `no-emoji`, `no-redundant-reverify`, `vault-writes-via-vault-uri`, `BUG_FIXING`, `vibe-extra-tools`.

How to write a scoped rule that works.

1. Scope to one tool: `tool:<exact tool name>`. A bare scope matches every tool's arguments.
2. Add regex flags. Without `(?m)`, `^` matches only at offset 0. Without `(?s)`, `.` skips newlines.
3. Test with the harness's rule-test tool against logged calls. A guess is not a test.
4. Remember a rule fires on partial streamed text. For a hard guarantee, use a guard.

Where in pack. `rules/global/`, `rules/project/`.

> Ugh. Fall: a path rule interrupted sessions 37 times in four days. It re-tested half-streamed text, so a chunk ending in "tas" matched. Each hit threw away partial output. A guard on final arguments replaced it.
>
> Rock rule: a rule is a tripwire, not a judge. Hard laws go in a guard.

Guards and extensions

What. An extension is a TypeScript file the harness loads at start. It adds tools (`services`, `move_session`) or hooks every tool call (a guard).

Why. A guard checks the final call once, before it runs.

File under `harness/extensions/`	Does
`global/guardrails.ts`	blocks reads of keys, secrets files, browser profiles; masks secrets as `*****`; asks before `env`
`global/tool-approval-gate.ts`	applies approval conditions to three tools; fails closed

File under `harness/extensions/`	Does
`project/no-parent-folder-guard.ts`	blocks any path that leaves the tasks folder
`project/process-ownership-guard.ts`	blocks stopping a process this session did not start
`project/services/index.ts`	the `services` tool: start, stop, check local services; keeps a ledger
`global/move-session.ts`	the `move_session` tool
`global/end-session.ts`, `clear-context.ts`, `compact-context.ts`	close, clear, compact when you ask in plain words
`global/advisor-gate.ts`, `advisor-drain.ts`	wake dormant advisors; hold the turn open until advisors finish
`global/mode-presets.ts`, `title.ts`, `vibe-extra-tools.ts`	model and mode presets; session names; extra tools for the vibe-mode director

Limits:

- `guardrails` is "a guardrail, not a sandbox". Command substitution, `bash -c`, and variable indirection bypass it.
- `no-parent-folder-guard` is lexical. It does not resolve symlinks, because task-folder links resolve into the bare repos and that route is mandated.
- A block reason that starts with `process-ownership-guard:` is final. Report it. Stop a foreign process yourself with `omp ps stop <name>` in your own terminal.
- The `services` tool refuses bare-repo paths and non-symlink paths.

How. Copy global extensions to `~/.omp/agent/extensions/`, project ones to `<tasks-root>/.omp/extensions/`, with `lib/`. Start fresh. Try a call the guard should block. If a guard blocks a good call, fix the call, not the guard.

Where in pack. `harness/extensions/global/`, `harness/extensions/project/`, `harness/extensions/project/services/plan.test.ts`.

> Ugh. Fall: a service-helper agent killed a process another task owned. Later it tried `kill $(lsof -ti :PORT)`. "Never stop a foreign process" in prose had failed. A ledger guard fixed it.
>
> Rock rule: whoever starts a process owns it. Nobody else stops it.

Approval gates: ask-before and auto

What. Two lists. What the agent must ask. What it does alone.

Why. Too many prompts and you click "yes" unread. Too few and the agent does harm. The author cut every always-yes prompt.

Three layers enforce the lists:

1. **Prose rule** in `RULES.md`: "Ask before you comment on a work item."

2. **Skill step** in `SKILL.md`: "Draft, then wait" or "Open the PR now".
3. **Harness config**: `deny` on a tool, or a prompt with an auditor review.

A skill step that says "open the PR", "post the notice", or "move the state" is itself the approval. Do it. Then re-fetch. A step that says "draft, then wait" keeps its gate.

Ask first:

- Any Slack post except the one-line PR-open notice. Draft in a fenced block. Wait for "send it".
- A tracker comment, when the skill says "show the draft".
- A tracker state, title, or description write outside the auto list.
- A PR vote. A merge. Arming auto-complete outside the merge skill.
- A push after review or conflict fixes.
- Deleting task folders.
- Starting a service by hand, not through the `services` tool.

Auto:

- All reads.
- Area path, iteration path, assign to self.
- Skill-directed moves: `New` to `Ready for Triage`; `Ready for Triage` to `Ready for Fix` after the bug reproduces; `Resolved` or `Ready for Test` once every linked PR has merged; `Ready for Retest` or `In Test` once the QA deploy check proves the fix is live.
- Open a PR when the proof list passes. Post the PR-open line.
- Make folders and worktrees through the scripts.

Never by the agent: `Closed`, `Pending PO Review`, `Accepted`; votes; merges outside `pr-merge`; QA login.

How the config does it:

- `tools.approval` in `harness/project/config.yml` holds `deny` or `prompt`. It denies the browser tools that read network and console data, tracker branch creation (branches come from the script), and the backlog tool.
- `toolApprovalConditions` governs three tools: tracker work-item writes, tracker comments, and Playwright `run_code_unsafe`. Global defaults sit in `harness/global/config.yml`. The project block deep-merges over them: objects merge, scalars override, the `rules` list replaces whole.
- Work-item writes have fallback `allow`. The harness does **not** prompt for a state, title, or description write. The agent must ask in chat. Only the `/rev` test, area path, iteration path, and assign-to-self with your own identity pass by rule.
- Comments and browser snippets use fallback `prompt` plus an **auditor**: a small model reads the text. SAFE runs with no dialog. Anything else shows the text and the auditor's note. Rubrics: `harness/global/tool-call-auditor.md`, `harness/project/tool-call-auditor.md`. A project may add rules but not relax a global one.
- **Fail closed.** If a config file cannot be read, the three governed tools are blocked.

How. Copy both `config.yml` files and both auditor files. Replace `<YOUR_EMAIL_ESCAPED>` and `<YOUR_DISPLAY_NAME>` in the self-assign rule. Test one call per governed tool: see the prompt, the allow, the block. Review prose and config in one pass, or they will disagree.

Where in pack. `harness/project/RULES.md`, `harness/project/config.yml`, `harness/global/config.yml`, `harness/extensions/global/tool-approval-gate.ts`.

> Ugh. Fall: prose said "ask before modifying a work item", but nothing prompted. Config had no entry. Prose and config disagreed.
> Rock rule: name the enforcer of each gate. No named enforcer, no gate.

MCP servers and the template

What. Four tool servers set in `mcp.json`. The pack gives a template, not the file.

Why. They are the agent's hands. Without them it cannot read a ticket, post a PR, or click a page.

- `azure-devops`: tickets, PRs, pipelines, wiki, repos. Logs in with a scope-limited personal access token.
- `azure-mcp`: cloud resource reads. Uses your cloud CLI login.
- `playwright`: a browser. Runs with `--secrets`. It types credentials by key name, so the value never reaches the model.
- `slack`: read and post chat, with a user token. Set the add-message flag only if you want posting.

How.

1. Copy `harness/mcp.template.json` to `<tasks-root>/.omp/mcp.json`.
2. Replace each `<PLACEHOLDER>`. The file holds the **name** of an environment variable, not the secret.
3. Export the variables in your shell **before** you start the harness. A later export stays invisible. Relaunch. Do not just reload MCP.
4. The tracker variable holds base64 of `:` plus the token, not the raw token.
5. Keep the Playwright secrets file outside git. Agents never read or print it.
6. Pin the Playwright MCP version (the template says `@latest`; replace it). Run it isolated, one per session. Four processes on one profile caused "Browser is already in use".
7. Always pass `project` to tracker calls. Without it: "Client does not support form elicitation".

The cloud CLI is for login only, after an MCP failure. Then retry the MCP call once. One exception: no MCP tool exposes the action at all, such as deleting a posted PR comment.

Where in pack. `harness/mcp.template.json`, `rules/project/azure-cli-fallback.md`, `rules/project/tool-gotchas.md`, `automation/playwright/`.

> Ugh. Fall: the tracker said "success" on a comment that came back with empty backticks. It had stripped angle-bracket tags.
> Rock rule: re-fetch after every write. A reply reports the call, not the state.

The task-folder model

What. One folder per ticket. Inside, one link per repo, each pointing at the ticket's own branch copy.

Why. Many tickets in parallel. No branch switching. No cross-talk. Scratch files stay in one place.

```
~/work/<PROJECT>/
  <repo>-bare/              one bare clone per repo (sessions never touch it)
    .env                    one env file per repo (never read, printed, copied)
    main/                   reference copy of main
    fix/<slug>/             one worktree per task branch
  tasks/                    THE SESSION ROOT. Start every session here.
    .omp/                   the project harness
    setup-task-branch.sh    and the other four scripts
    docs/PAPERCUTS.md       global log of anything that cost time
    main/                   repo links, each synced with main
    archive/                notes of removed task folders
    BUG-12345-<slug>/       one folder per ticket
    STORY-12346-<slug>/
    PR-REVIEW-4567-<slug>/
    RESEARCH-00000-<slug>/
```

Words:

- **Bare clone** (`<repo>-bare`): a git copy with history and no checked-out files. The store.
- **Worktree**: a second working copy on its own branch, same history. Cheap.
- **Tasks root**: the one start folder. The harness reads `<cwd>/.omp` only. Start elsewhere and MCP servers, rules, and skills vanish.
- **Repo symlink**: a link in the task folder, named after the repo, pointing at that ticket's worktree. `read` follows it. `git -C <folder>/<repo>` works through it.
- **The `main` folder**: a folder in the tasks root with a repo link per worktree, each synced with `main`. The agent reads current main there, never from the bare repos. If it is missing, the agent tells you and asks you to create it with the setup script.

Inside a task folder:

- `<repo>` links: one per in-scope repo. Leave them all.
- `plan/`: design docs (`repro.md`, `FIX-PLAN.md`, `STORY-PLAN.md`, `VERIFICATION.md`) and commit-message files.
- `docs/`: evidence: `attachments/`, `flow.yaml`, `flow-runs/`, `HANDOFF.md`.
- `review/`: `SECURITY-GATE.md`, `REVIEW.md`, lane reports, scan reports.

Scratch stays in the ticket folder. Never `/tmp`.

The `.env` file. It lives once at the bare-repo root. The script **symlinks** it into each worktree. One edit changes every worktree of that repo. Some old notes say "copies". They are wrong.

- Never read, print, copy, or commit `.env` values. The guard masks them as `*****`. Never write the stars to a file. Ask the human for a secret.

- Send sibling URLs through process env at start (the `services` tool `env` argument). Never edit `.env` for that.
- `.env.local` is the agent's. Its first line is exactly `# managed-by: omp-agent (non-secret overrides only)`. No secrets. Check it is gitignored.
- Review mode (`--pr`) makes no `.env` link.

Start at the root, then move. A session starts in the tasks root. For a ticket, the agent calls `move_session` into the ticket folder. It can move only to the launch folder or a direct child. The ticket folder links the shared `.omp`, so MCP stays attached. A subagent cannot move. It uses absolute paths.

1. `glob` for `*-<NUM>-*` in the tasks root.
2. Folder exists: `move_session` into it before any work, even a status check.
3. No folder: stay at the root. Bootstrap makes it and moves.
4. A Story gets **no folder** before Session B passes its contract gate.

Never remove what the script made. After `setup-task-branch.sh`, never remove a repo symlink, branch, or worktree it made. Keep every repo linked, even out-of-scope ones, because later testing may need them. Only the retiring legacy repo is skipped, by the script. Only `/cleanup` removes a finished task, after you approve.

Where in pack. `scripts/setup-task-branch.sh`, `scripts/setup-task-lsp.sh`, `scripts/cleanup-task-folders.sh`, `skills/ticket-bootstrap/SKILL.md`, `skills/task-branch-script-internals/SKILL.md`, `skills/repo-onboarding/SKILL.md`, `skills/task-cleanup/SKILL.md`, `rules/project/no-parent-folder-repo-access.md`, `rules/project/task-files-in-ticket-folder.md`, `harness/extensions/global/move-session.ts`.

> Ugh. Fall: an agent tidied up. It deleted branches, worktrees, and symlinks of out-of-scope repos. The author needed them to test and rebuilt them by hand.
> Rock rule: never remove what the script made.

> Ugh. Fall: a subagent wrote `../plan/...` from inside a repo symlink. The shell resolved the link first. The folder landed outside the tasks folder.
> Rock rule: subagents get absolute paths. Never `../` from a repo link.

> Ugh. Fall: `grep` said "No matches found" for text that existed. The path was the symlink itself. `grep` fails on a link and works on a real directory below it.
> Rock rule: search a whole repo with `git -C <folder>/<repo> grep`. Full table: `rules/project/tool-gotchas.md`.

Naming of folders and branches

What. A strict pattern.

Why. Tools find folders with a glob on the ticket number. Free-form names break the lookup.

Folder: `{TYPE}-{NUM}-{very-brief-dash-summary}`. Type uppercase. Summary is a short dash-cased paraphrase of the title. The script makes the folder. Never `mkdir` it first.

Work	Folder	Branch
Bug	`BUG-12345-prompt-search-mismatch`	`fix/<slug>`
Returned bug, round n	same folder	`fix/<slug>-<N>-r<n>`
Story	`STORY-12346-memory-prompt-guard`	`feat/<slug>`
Research, no ticket	`RESEARCH-00000-<slug>`	`research/<slug>`
Test sandbox	`SANDBOX-00000-<slug>`	default branch
PR review	`PR-REVIEW-4567-<slug>`	the PR's source branch
PR, no work item	`PR-4567-<slug>`	the PR's source branch

Roads:

- One folder per ticket. The script refuses a second folder with the same `{TYPE}-{NUM}-` and a new slug. Reuse the name it reports.
- Stories use `STORY-`, not `US-`. Old notes say `US-`. They are stale.
- A vault-driven non-ticket task uses branch `task/<slug>`.
- One commit per repo per ticket. Amend, do not stack.
- Bare clones have no fetch refspec. A plain `git fetch origin` updates only `FETCH_HEAD`. Use `+refs/heads/<b>:refs/remotes/origin/<b>`.
- Fix a PR conflict by merging, never rebasing. `--force-with-lease` only on your own amended branch. Never `--force`.

Where in pack. `skills/ticket-bootstrap/SKILL.md`, `skills/pr-creation/SKILL.md`, `skills/merge-conflict/SKILL.md`, `harness/project/AGENTS.md`.

Setup scripts

What. Five shell scripts that do the boring work the same way each time.

Why. Hand-made worktrees and links drift. A script is idempotent: run twice, same result.

- `scripts/setup-task-branch.sh <branch> <target_dir>`: make or reuse the branch worktree in every bare repo, link `.env`, link each worktree into the new task folder. Exit 0, or 1 naming failed repos.
- `scripts/setup-task-branch.sh --pr <repo> [--target-branch <b>] <branch> <folder>`: review mode. One repo, branch fetched from origin, no `.env` link.
- `scripts/setup-task-lsp.sh <task_dir>`: link `.omp`, install each repo's dependencies, install pinned language-server tooling, write `lsp.json`, `dap.json`, `pyrightconfig.json`.
- `scripts/self-review.sh <repo-path> [base]`: write the diff to `review/self-<repo>.diff`. Flag new code comments, env files, dependency changes, secrets. Exit 0 clean, 1 findings, 2 usage.
- `scripts/secret-scan.sh <repo-path> [base]`: gitleaks over commits, staged, unstaged, untracked. Values redacted. Same exit codes.

- `scripts/cleanup-task-folders.sh`: `report`, `clean`, `clean-worktree`, `strays`, `prune`. Dry run unless `--apply`.

Facts:

- Run `setup-task-branch.sh` from the tasks root. It finds every `*-bare` folder in the **parent** of the folder you run it from. A relative `target_dir` resolves against your cwd.
- Script broken? Read `skills/task-branch-script-internals/SKILL.md`. Repo has no bare clone? Read `skills/repo-onboarding/SKILL.md`.
- `secret-scan.sh` needs the `gitleaks` tool. It also expects `gitleaks.toml` and `gitleaks-report.tmpl` in the tasks root. Copy both from `scripts/`. Smoke-test the config: one case that must fire, one that must pass. Use a rule-scoped allowlist. A global one is silently ignored when default rules are extended.
- The shell tool caps a command at 300 seconds. Clean in batches of three or four folders. Pass `timeout` explicitly.

Where in pack. `scripts/`, `skills/pr-security-gate/SKILL.md`, `skills/task-cleanup/SKILL.md`.

> Ugh. Fall: scripts passed on the author's machine and crashed only under cron or in tests.
> Rock rule: test scripts in a sandbox, in the real shell, with a rerun.

Install order

1. Install the harness and tools: `git`, `pnpm`, `uv`, `gitleaks`, a cloud CLI, `lsof`.
2. Make `~/work/<PROJECT>/tasks`.
3. Bare-clone each repo as `<repo>-bare` with a `main` worktree (`skills/repo-onboarding/SKILL.md`).
4. Copy harness files, rules, skills, commands, agents, extensions, scripts.
5. Make `mcp.json` from the template. Export variables. Relaunch.
6. Make the `main` folder. Make `docs/PAPERCUTS.md`.
7. Fill every placeholder. Restart. Start fresh in the tasks root.
8. Test: run `/pr-status`. Try a blocked call. Run the branch script in a sandbox.

> Rock rule: finish one piece, test it, then the next.

The Daily Loop

How a day flows

What. A day has a shape: morning read, pick, work, review, close.

Why. Shape saves thinking. The agent does the reading before you wake.

```
before      start        pick          work            review        end
jobs run -> brief  --> a ticket --> bug or story --> comments,  --> poll, log,
            + PR card               (own session)     merge         cleanup
```

The author worked in an 11-hour band. Two unattended jobs ran before it began.

Entry points in the order you meet them: `/morning`, `/pr-status`, then `/bug`, `/story`, or `/triage` for a ticket, `/comments` and `/conflict` for review work, `/pr-merge` for your own PR, `post-merge` after it, `/qa-deployed` to learn if a fix is live, `daily-work-poll` to recall a day, `/cleanup` each week.

About 15 percent of the author's human turns were saved-command calls. Each command is a prompt under ten lines. It names a skill and passes one argument. It never restates steps, so it cannot drift from the skill.

Where in pack. `commands/`, `skills/workflow-router/SKILL.md`.

The morning brief

What. A list of tickets whose PRs merged and wait for a state move, plus items that need you.

Why. Manual status checks were the most frequent recurring prompt. A job reads. You approve once.

How the brief is made. A cron job (`board-ingest`) runs on weekday mornings. It makes one headless agent call that can read the tracker but **cannot write to it**: a config overlay denies every tracker write tool and the chat post tool. The job:

1. Stops if the log already holds `board scan <date>`. Idempotent.
2. Queries the team board and your own open items.
3. Counts open threads on your active PRs.
4. Proposes one move per item by a fixed table: Bug `Ready for Fix` with all PRs merged to `Resolved`; Story before test with all PRs merged to `Ready for Test`. Never `Closed` or `Accepted`.
5. Overwrites two vault notes (board state, morning brief). Appends one log entry.

How you work it. Type `/morning`.

1. The agent reads the brief. If its date is not today, it says so and re-derives "ready to close" live. Never act on a stale list.
2. It shows each row: ticket, title, state to proposed state, merged PR.
3. You give **one** approval for the list. A subset is fine.

4. For each approved row it writes the state with a `/rev` check, then re-fetches.
5. It marks matching vault tasks done and logs.
6. For "Needs QA deploy check" rows it runs `tasks-deployed-to-qa`. On proof it writes `Ready for Retest` (Bug) or `In Test` (Story) with no prompt.

Where in pack. `skills/morning-brief/SKILL.md`, `commands/morning.md`, `automation/board-ingest/prompt.md`, `automation/board-ingest/run.sh`, `automation/README.md`.

> Ugh. Fall: "who touched this?" answers were wrong. A "changed by me" query returns only the last editor.
>
> Rock rule: attribute work through revision fields (changed by, resolved by, closed by), never by date.

My PR status

What. One fixed card of your open PRs.

Why. One fixed card scans in seconds. A live fetch avoids stale votes.

Type `/pr-status`. Output is one fenced block. The agent:

1. Lists your active PRs, one call per repo, in parallel.
2. Reads merge status, reviewers, linked ticket, threads.
3. Classes each build: `no run`, `running`, `failed`, `stale` (green, but `main` moved or a newer push has no run), `passed`.
4. Prints per PR: number, short title, ticket, votes (names, no numbers), comments, checks, one state line.

State order: merge conflict, pipeline failed, rejected, waiting for author, open comments, pipeline needs re-run, needs an approval, ready to merge.

Read-only. It never votes, comments, or posts. A Slack list of PRs needing approval is made only on request: draft in chat, post after "send it".

Where in pack. `skills/my-pr-status/SKILL.md`, `commands/pr-status.md`.

> Ugh. Fall: a PR with a green build could not auto-complete. `main` had moved, so the policy build expired. A fresh plain run did not attach to the policy.
>
> Rock rule: re-queue through the policy action, not a new run.

The daily work poll

What. A report of what you did on a day. Ask "What did I do on ?".

Why. The tracker alone misses PR reviews saved as local files and commits on feature branches.

Three sources, merged:

1. **Tracker.** Items changed in the range with you as changer. The tracker stores UTC. Pin the range twice: UTC for the tracker, local for file times.
2. **Local review folders.** `PR-*` and `PR-REVIEW-*`. File times in `review/` prove the day. `REVIEW.md` holds the verdict.
3. **Git history.** Every branch, matched to ticket numbers so one piece counts once. Read through task-folder links, never the bare repos.

Report per item: number, title, type (Bug, Story, PR review, commit-only), short description. Say plainly when a source returned nothing. A recap happens only when asked: one line each, no codes.

Where in pack. `skills/daily-work-poll/SKILL.md`.

The wiki task loop

What. A generic dev loop driven by vault task pages. It works a task on a `task/<slug>` branch with one approval.

Why. Some work has no ticket. A vault task page is the ticket. The loop gives it the same shape.

1. Pick a task: one you name, or the top-priority page with `status: todo`.
2. Set `status: doing`. Make sure a tracker item exists, or make one.
3. Resolve the repo from the project page. Branch `task/<slug>` before any edit.
4. Implement. Test end to end with the repo's own commands.
5. Commit and push. Write a decision page per design choice.
6. **Gate.** Show the diff. One approval covers the PR, the tracker review move, and `status: review`.
7. Open the PR. Move the item. Draft a short chat update. Wait for approval.
8. Append a log entry.

On a blocker: add a `## Blocker` section to the task page, keep `status: doing`, report, log.

Inside the tasks root, the project rules win over this generic gate. The PR opens with no prompt after verification. Tracker moves follow the exception list in `harness/project/RULES.md`.

Sibling vault skills (run one only when you ask): `wiki-ingest`, `wiki-meeting`, `wiki-sync`, `wiki-query`, `wiki-lint`, `wiki-comms`, `wiki-spec`.

Where in pack. `skills/wiki-task-loop/SKILL.md`, `skills/wiki-sync/SKILL.md`, `vault/schema.md`.

Session hygiene

What. Habits that keep a session clean and cheap.

Why. In five weeks, 70 of 300 of the author's sessions ended aborted (23 percent), and 87 of 280 were harness or tooling work (31 percent). A messy session wastes context and loses state.

One session per ticket.

1. One ticket, one session. Do not mix.

2. Start at the tasks root. Always.
3. For a ticket, the agent moves into its folder with `move_session`. Even for a status check.
4. Run setup scripts with the shell `cwd` at the tasks root.
5. A subagent cannot move. Give it absolute paths.
6. Start a Story's Session B fresh. It trusts only the ticket.

Where state lives. A session can die. Put state in files: plans and evidence in the ticket folder, decisions and task status in the vault, ticket state in the tracker. The author logged 110 `/clear` in five weeks, and one recap showed ticket state lost after a clear. A per-ticket resume file (`plan/STATE.md`) was proposed. It is a proposal, not shipped.

Close, clear, compact. Say it in plain words.

- "Close the session": the agent calls `end_session` with a short reason, writes one line, stops.
- "Clear the context": it calls `clear_context`, confirms in one line.
- "Compact": it calls `compact_context`.

No confirmation. No "type /exit". Never while your requested work is unfinished. Never to escape a hard task. After a clear or compact the session may return to the launch folder, so move into the ticket folder again. [INFERENCE: the return to the launch root was designed; a live run is not recorded in the sources.] State a durable fact one turn **before** you close. The exit gate runs before the scribe advisor reviews.

Other habits.

1. After you touch a tool or extension, restart. Test fresh.
2. Stop every process you start before you yield. Run `lsof` after. Report the result.
3. Check the port before you start a service. If another process owns it, never stop it. Pick another port, or ask the owner.
4. Close the browser and stop verification services before the PR link and notice.
5. A clean advisor or reviewer verdict is final. Do not re-run checks for comfort.
6. When a tool costs you time, append one line to `docs/PAPERCUTS.md`: date, symptom, fix, project. Read it first when a tool fails for no known reason.
7. Never start a session inside a repo worktree. It loses MCP, rules, skills, and commands.

Where in pack. `harness/global/APPEND_SYSTEM.md`, `harness/extensions/global/end-session.ts`, `harness/extensions/global/clear-context.ts`, `harness/extensions/global/compact-context.ts`, `harness/extensions/global/move-session.ts`, `rules/project/no-orphan-processes-after-use.md`, `rules/global/no-redundant-reverify.md`.

> Ugh. Fall: a browser and dev servers kept running after the PR. Orphan helpers piled up.
> Rock rule: leave nothing running that was not running before.

> Ugh. Fall: a helper's "done, tests pass" was forwarded as truth. The tests had not run.
> Rock rule: a helper's report is a claim. Read the real diff and output.

A day, step by step

What. One sample day. Use it as a checklist.

How.

1. Open a session in the tasks root. Run `/morning`, then `/pr-status`.
2. Fix what waits on you: `/comments <pr>`, `/conflict <pr>`.
3. Pick a ticket: `/bug <id>`, `/story <id>`, or `/triage <id>` for a thin one.
4. Watch each gate. Approve drafts. Read evidence.
5. PR approved: `/pr-merge <pr>`. Then `post-merge`. Later, ask "is it on QA?".
6. End of day: ask `wiki-comms` to draft the manager update if you owe one. Close the session.

> Rock rule: the agent reads and drafts. You decide what has an audience.

Triage: Know Before You Dig

Cave elder says: dig in wrong spot, lose a week. Look first. Think first. Then dig.

This part covers the work before any code. A ticket arrives. You decide what it is. You make sure everyone understands it. Only then you build.

Words, once:

- **Ticket** = one job in the tracker (here: Azure DevOps). Also called work item.
- **Bug** = something works wrong. **Story** = something new to build.
- **State** = the stage of a ticket, like `New` or `Ready for Fix`.
- **AC** = acceptance criterion. One sentence a test can pass or fail.
- **Skill** = a recipe file the agent reads. **Command** = a saved prompt like `/triage 12345` that loads a skill.

Two paths: Bug and Story

What. Bugs and Stories take different roads.

Why. A bug already has proof: you can make it break. So the bug road is one session. A story has no such proof, so it gets two sessions, with the ticket as the handoff note between them.

How.

1. Bug: bootstrap the folder, reproduce, fix, prove, open the PR. One session.
2. Story: Session A talks to you and writes the contract into the ticket. Session B starts blank, reads only the ticket, builds.
3. The router skill picks the road from ticket type and state.

```
Bug   : ticket -> bootstrap -> bug-workflow ------------> PR
Story : ticket -> Session A (grill, contract) -> ticket
                  Session B (gate, build)     -> PR
```

Where in pack. `skills/workflow-router/SKILL.md`, `commands/bug.md`, `commands/story.md`, `harness/project/AGENTS.md`.

> Rock rule: Bug reproduction is the contract. Story contract is written in the ticket.

Bug triage: is it real, is it ours

What. Before hours of work, decide: does this bug need you now?

Why. Boards fill with bugs already fixed, already taken, or waiting on a tester. Guessing wastes days.

How.

1. Fetch the ticket fresh. Read the state: `New` = nobody looked; `Ready for Triage` = looked at, not yet shown to break; `Ready for Fix` = shown to break.
2. Check if someone picked it up (next section). Read the linked pull requests (PR = request to merge a branch).
3. Yours? Bootstrap it. Bootstrap moves `New` to `Ready for Triage`, nothing more. Never write `Ready for Fix` until the bug breaks on your own machine.

Where in pack. `skills/ticket-bootstrap/SKILL.md` (Step 5), `skills/bug-workflow/SKILL.md` (Step 2).

The actionable-bug rules

What. Four rules say when a board bug has work left for you.

Why. "Which bugs need me?" got a different answer each time. Rules end the argument.

How. A bug has work for you when at least one is true:

1. **Open PR, unresolved comments.** An Active linked PR has a thread a person wrote and nobody resolved. System threads ("policy updated", "added reviewers") do not count.
2. **Triage, not picked up.** State `Ready for Triage`, nobody on it.
3. **New, not picked up.** State `New`, nobody on it.
4. **Merged PR, tester moved it back.** A linked PR is merged. A tester moved the state back and commented why.

"Picked up" means one of: a task folder exists; a branch or worktree exists; a linked PR exists or a comment claims the work; a live agent session works on it. A name in the assignee box is not "picked up". Old session logs are not evidence: they list every board id.

The board is an **area path**, not an iteration path. For rules 2 to 4 skip `Closed`, `Cancelled`, `Duplicate`. For rule 1 do not skip them: a closed bug can hold an open PR.

Where in pack. The mechanical table is `automation/board-ingest/prompt.md` (Step 7). The brief is `skills/morning-brief/SKILL.md`.

> Source gap: the four rules come from the author's wiki note, not from the pack. Only the board table is in the pack.

Ticket triage with `/triage`

What. `/triage 12345` explains a ticket, grills you, and writes a spec into the ticket. Any ticket type.

Why. Agents misread thin tickets. So do you. The agent explains until you can teach it back.

How.

1. Run it on the big model. The command says so; it does not check.
2. Load the profile. `ticket-triage` is tool-neutral. `ticket-triage-profile` binds it to your tracker, fields, repos. A profile rule beats a generic rule.

3. Pre-flight: id is all digits; look for an existing task folder and move into it; never run the branch script to triage.
4. Fetch everything: type, state, every long text field, area and iteration path, tags, comments, links, every image. Make the download folder first; the tool will not.
5. Ground in code: one read-only scout per candidate repo, in one call. Each reports `file:line`, current behaviour, and whether the wanted behaviour already exists. Scouts read `origin/main`, not a stale worktree.
6. Search the vault by ticket id and keywords (never write to it here). Check the main claim against the code: `confirmed`, `contradicted`, or `insufficient detail`. Run nothing.
7. Brief (8 bullets at most). Walkthrough: 3 to 6 topics, one per turn. Then the teach-back gate: you explain the ticket in your words, the agent compares it with a private checklist and re-explains gaps. Pass only when all points are covered. Never skip.
8. Grill (below). Draft the spec in a fenced block, one version per turn. Write only after "approved".
9. Write the spec as a **comment**. Never touch the description. Re-fetch and check.
10. Offer a state move. Bug in `New`: offer `Ready for Triage`. Never offer `Ready for Fix`.

Bug heading: `## Triage Specification`. Story heading: the contract heading. If a named outsider must answer first, write `## Triage Notes` instead.

Where in pack. `commands/triage.md`, `skills/ticket-triage/SKILL.md`, `skills/ticket-triage-profile/SKILL.md`.

Story refinement, Session A

What. A talk with you. Its only product is the ticket: a full Agent Contract in the description and state `Ready for Development`.

Why. The old way did talk and build in one session, so the ticket was never tested as a handoff. Now Session A ends and Session B lives on the ticket alone.

How.

1. Fetch the story. Note the description and its format (HTML or Markdown).
2. `Backlog`: move to `In Refinement`, re-fetch. Past `Ready for Development`: stop.
3. Read code through the `main` worktrees. Do not run the branch script. The grill may shrink, split, or reject the story, and the script makes branches in every repo.
4. Grill.
5. Write the contract into the **description**. Read-modify-write: take old text, append the contract, write the whole field back. Match the format: HTML field, HTML contract. Use `update_batch`, which takes a format; plain `update` does not.
6. Re-fetch. Contract renders. Human text intact.
7. One short comment: refinement done, tag the product owner. Comments carry reasons and questions, never the contract.
8. Ask you. Move `In Refinement` to `Ready for Development`. Re-fetch.
9. Add a vault log entry. End the session.

Where in pack. `skills/story-refinement/SKILL.md`, `commands/story.md`.

The Agent Contract

What. Six sections in the ticket. All mandatory. No `TBD`.

Why. It is the full instruction for a builder who was not in the room. The Session B gate reads it by machine rule, so a gap cannot hide.

How.

```
## Agent Contract (Ready for Development)
### Acceptance criteria (restated, testable)
AC-1: ... (a test or live check can pass or fail it)
### Resolved questions
Q: ... / A: ...
### Affected repos
<repo list>, explicit non-goals, repos NOT to touch
### Verification plan
per AC: the test command or live check that proves it
### Deploy-order notes
what the user sees for each merge or deploy order
### Stop conditions
named conditions that force the builder to block and comment
```

Where in pack. `skills/story-refinement/SKILL.md` (Step 4), `skills/ticket-triage-profile/SKILL.md` (Spec targets).

> Source gap: "search every repo on `origin/main` before the contract is final" comes from lessons. The bundled refinement skill does not say it. Add it.

Why a story never starts without the contract

What. A gate at Session B Step 0. No contract, no work.

Why. An agent that makes do with a thin ticket builds the wrong thing fast. The refusal is the quality tool. The team retro asked for a Definition of Ready. This gate is one made of code.

How. Pass only if both hold:

1. State is `Ready for Development`.
2. The description, or the newest comment headed `## Agent Contract (Ready for Development)`, has all six sections non-empty with no `TBD`. A `(1/2)` and `(2/2)` split counts as one.

On failure: comment naming each gap and tagging the refiner or product owner; move back to `In Refinement`; re-fetch; stop. No folder, branch, or code.

Where in pack. `skills/story-workflow/SKILL.md` (Step 0), `skills/ticket-bootstrap/SKILL.md` (Step 2).

Grilling

What. The agent asks hard questions, one at a time, each with its recommended answer.

Why. You hide doubts from yourself. Ordered questions pull them out. One per turn keeps you thinking.

How.

1. Load the `grilling` skill.
2. One question per turn, with a recommended answer.
3. Ground it in code: "The ticket says X, but `file:line` does Y. Which wins?"
4. A tool can answer a fact: look it up. Only ask you decisions.
5. Also grill the task breakdown: tier, who, what depends on what.
6. Stop when: every AC testable; every doubt resolved or deferred with an owner; repos touched and untouched listed; one named check per AC; deploy-order effects known; stop conditions written; task list agreed.

Where in pack. `skills/third-party/grilling/SKILL.md`, used by `skills/ticket-triage/SKILL.md` (Step 6) and `skills/story-refinement/SKILL.md` (Step 3).

When to split a ticket

What. The grill can show one ticket is really two, or too big.

Why. Big tickets make big PRs, and review waits for days. The retro said: split large PRs into smaller tickets from the start.

How.

1. Signals: an AC maps to a repo the contract does not name; two pieces share no files; a deploy order hurts users; an outsider must decide something.
2. Finish and verify the first ticket's contract.
3. Draft each new story in a fenced block: title, description, ACs, area path, a related link back. Show it.
4. Create it only after approval. Re-fetch. Report the id.
5. In Session B, never widen scope. Use the escalation rule (comment with `file:line`, tag a person, `Blocked`, stop).
6. A pre-existing hole the security gate finds becomes a new Bug, not part of this PR.

Where in pack. `skills/ticket-triage-profile/SKILL.md` (Next step), `skills/story-workflow/SKILL.md`, `skills/pr-security-gate/SKILL.md`.

Area path and iteration path

What. Area path = which team board owns the ticket. **Iteration path** = which sprint.

Why. Wrong area path and the ticket vanishes from the board. The board query uses it.

How.

1. Set the area path on every ticket you create or edit. The value lives in one rule file (it may be a temporary override). Skills point at the rule and never hold the value.
2. Use the work-item write tool, field `System.AreaPath`, never the CLI. Re-fetch.
3. You may change area path, iteration path, and self-assign without asking. Any other edit asks first.

Where in pack. `rules/project/ado-area-path.md`, `harness/project/RULES.md`.

Bootstrap: Make the Cave

Cave elder says: every hunt needs a cave. Dry floor. Tools on the wall. Same layout every time. Then you never lose your spear.

Words, once:

- **Bare clone** = repo copy with history but no working files. One per repo.
- **Worktree** = second copy of the repo, own branch, same history.
- **Symlink** = a shortcut that points to a real folder.
- **LSP** = language server: "go to definition", "find references". **DAP** = how a debugger talks to a program.

What bootstrap makes

What. One folder per ticket: one shortcut per repo, each pointing at a worktree on your task branch, plus code-intelligence config, plus the session moved inside.

Why. Tickets run in parallel. One folder keeps plans, evidence, and code together. One script builds it, so it never drifts.

How.

```
~/work/<PROJECT>/
  <repo>-bare/             bare clone per repo. Never touch
    main/                  reference worktree
    fix/<slug>/            your task branch worktree
  tasks/                   start the agent HERE
    setup-task-branch.sh   setup-task-lsp.sh
    BUG-12345-<slug>/      the task folder
      orchestrator -> ../../orchestrator-bare/fix/<slug>
      web-app      -> ../../web-app-bare/fix/<slug>
      ...one symlink per repo      docs/  plan/  review/
```

Where in pack. `skills/ticket-bootstrap/SKILL.md`, `scripts/setup-task-branch.sh`, `scripts/setup-task-lsp.sh`.

Bootstrap, step by step

What. `ticket-bootstrap` has numbered steps. Keep the order.

Why. Each step stops a real mistake.

How.

1. **Step 0.** Glob for `*-12345-*`. If a folder exists, `move_session` into it first. Bootstrap refuses a second folder for one ticket.
2. **Step 1.** Fetch the ticket with the tracker tool. On failure, run the CLI login fallback and retry the tool. Never swap the tool call for a CLI read.

3. **Step 1b.** Bug: detect a returned bug (see the bug hunt).
4. **Step 2.** Branch on type. Bug goes to `bug-workflow`. Story only if `Ready for Development` with a contract; else back to refinement.
5. **Step 3.** Run the branch script. **Step 3a.** Look for a colleague's work in three places: ticket links, PRs per repo, remote branches. Found? Rerun the script with their branch name, work on top, never rewrite their commits, push to their open PR. (Written, never run on a real branch.)
6. **Step 4.** Never remove a symlink or delete a branch or worktree after the script ran.
7. **Step 4b.** Run the LSP script. Call `move_session` with the folder name.
8. **Step 5.** Bug in `New`: write `Ready for Triage`. Any other state stays. Story: do not set; check it is `Ready for Development`. Re-fetch.

Branches: `fix/<slug>` for a Bug, `feat/<slug>` for a Story. Folder: `{TYPE}-{NUM}-{very-brief-slug}`, TYPE uppercase (`BUG`, `STORY`, `RESEARCH`, `PR-REVIEW`). A ticketless research item uses `RESEARCH-00000-<slug>`.

Start the agent in `~/work/<PROJECT>/tasks`. Rules, skills, and tool servers load from the start folder only. The task folder links the shared config, so they survive `move_session`.

Where in pack. `skills/ticket-bootstrap/SKILL.md`, `harness/extensions/global/move-session.ts`, `harness/project/RULES.md`.

> Rock rule: Start at the tasks root. Move in on purpose.

The branch script

What. `./setup-task-branch.sh <branch> <target_dir>`, run from the tasks root.

Why. Hand-made worktrees drift. A boring script does it the same way every time.

How. Per repo, the script:

1. Finds every `*-bare` folder in the parent of the tasks root.
2. Skips its standing exclusion (the retiring `legacy-agent` repo).
3. Makes the branch as a worktree off the default branch. Reuses it if present.
4. Symlinks the bare repo's root `.env` into the worktree. One edit changes every task's link. The script links. It never copies.
5. Makes one relative symlink in the task folder.

Rules:

- It makes the folder itself. Never `mkdir` it. Never delegate that.
- It refuses a second folder in the same `{TYPE}-{NUM}-` family under another slug. Reuse the name it reports.
- Read the folder listing yourself afterwards.
- `--pr <repo>` mode wires one repo for reviewing someone else's PR: fetches the branch, resets any stale local copy to the fresh head, makes no `.env` link.

Where in pack. `scripts/setup-task-branch.sh`, `skills/task-branch-script-internals/SKILL.md`.

> Ugh. Fall: A rerun made the branch in every bare repo, out-of-scope ones too. Later an agent tidied up and deleted five out-of-scope worktrees. The author needed them. Exclusions now live in the script.

> Rock rule: Never hand-roll worktrees. Never remove a link after setup.

LSP and debug setup

What. `./setup-task-lsp.sh <task_dir>` makes the folder a place where code intelligence and debugging work.

Why. Without it the agent guesses symbols with text search, which misses shadowing and re-exports.

How. The script links the shared `.omp` config; installs every repo (`uv sync` for Python, `pnpm install` for the web app, `debugpy` where missing); installs pinned language-server tooling; writes `lsp.json`, `dap.json`, `pyrightconfig.json`, and a task-root marker. Then `move_session`. A subagent cannot move.

Debug notes:

- Name the adapter in every `debug` call. Generated adapters carry no file types.
- Python launch: `program` is the repo's `.venv/bin/uvicorn`. Never `--reload`; it forks a child the debugger cannot see.
- The web app has no process debugger. `infra` (Terraform) gets no language server.

Where in pack. `scripts/setup-task-lsp.sh`, `harness/project/dap.json`, `skills/ticket-bootstrap/SKILL.md` (Step 4b).

Repo onboarding

What. What to do when a repo has no bare clone yet.

Why. The branch script only sees `*-bare` folders. A new repo is invisible until onboarded.

How. Follow `repo-onboarding`: find the repo in the tracker, make the bare clone, make the `main` worktree, link `.env`, add the repo to the standing facts file. Agents read `main` code through the `main` folder inside the tasks root (one symlink per repo). They never open the parent folder or bare clones. No `main` folder? Tell the author to create it with the setup script.

Where in pack. `skills/repo-onboarding/SKILL.md`, `rules/project/no-parent-folder-repo-access.md`, `harness/extensions/project/no-parent-folder-guard.ts`, `harness/project/AGENTS.md`.

What the folder looks like after

What. The shape to expect.

Why. Later skills write to named places. Know them, find evidence fast.

How.

Path	Holds
`<repo>` symlinks	worktrees. Use `read`, `git -C <task>/<repo>`
`plan/`	repro, fix plans, verification doc, return rounds
`docs/`	findings, attachments, `flow.yaml`, flow runs
`review/`	self-review diffs, `SECURITY-GATE.md`

- Scratch, notes, commit-message files stay in the ticket folder. Never `/tmp`.
- `read` follows repo symlinks. `glob` works from the symlink down. `grep` fails on the symlink itself: use `git -C <task>/<repo> grep`.

Where in pack. `rules/project/task-files-in-ticket-folder.md`, `rules/project/tool-gotchas.md`, `harness/project/AGENTS.md`.

The Bug Hunt

Cave elder says: bug is a beast in the dark. You do not swing at noise. You make it show itself. You hold it with a light. Then you strike once.

Words, once:

- **Reproduce** = make the bug happen on purpose.
- **Failure boundary** = the exact place where the wrong thing happens.
- **Root cause** = the real reason, not the symptom.
- **Worker** = a helper agent with one narrow job and no memory of your chat.

The whole bug path

What. One session, folder to PR.

Why. The reproduction is the contract. If you cannot make it break, you cannot say you fixed it.

How.

```
bootstrap
  Step 0   returned bug? (skip if fresh)
  Step 1   start services, make it break
  Step 2   Ready for Triage -> Ready for Fix
  Step 2b  debugger, root cause
  Step 3   fix plan per repo, failing test first
  Step 4   live verify, mutation check, flow run, tear down
  Step 5   one commit per repo, self-review, security gate, PR
  Step 6-8 link PR, hold state, team line, vault record
```

A session told "Fix bug 12345" has consent to implement from its first message. Ask-first gates are review or done state moves and any Slack post but the PR-open line.

Where in pack. `skills/bug-workflow/SKILL.md`, `commands/bug.md`, `rules/global/BUG_FIXING.md`.

Step 0: the returned bug

What. A tester moved your fixed bug back. Step 0 finds out why.

Why. The first fix missed a case. Doing it again fixes nothing.

How. Bootstrap Step 1b detects it from the full state history. POST set = `Resolved`, `Ready for Retest`, `Closed`. RETURN set = `New`, `Ready for Triage`, `Ready for Fix`, `Bug Test Failed`, `Reopen`, `Blocked`. Returned bug = history touched POST and now sits in RETURN. PR mode: only active PRs = open-PR mode; only completed or none = merged mode; both = stop and ask. Merged mode branch: `fix/<slug>-<N>-r<n>`.

Then, writing to `plan/return-r<n>.md`:

1. Collect evidence since the move: comments, repro-step edits, new attachments, PR threads. Found repro steps win; else copy the originals and say so.
2. Merged mode: QA build check. `git merge-base --is-ancestor <merge-sha> <deployed-ref>`, exit 0 = QA holds the fix. QA lacks it? Stop, record, draft a comment, write no state.
3. Reproduce on QA first (UI: `docs/flow-r<n>.yaml`, env `qa`, expect `broken`, you log in by hand). Then locally.
4. Decide:

QA	Local	Action
breaks	breaks	Confirmed. Step 2
breaks	clean	Match QA locally, 3 tries, change one variable each
clean or not run	breaks	Confirmed. Step 2
clean	clean	Stop. Draft a comment asking the tester for environment, build, account, steps

5. Vault: one line under `## Returns` in every outcome.

Never overwrite the tester's state. Moving to `Ready for Fix` asks you first. Fix plans are new files, `plan/fix-plan-<repo>-r<n>.md`, each with `## Why the previous fix failed`: merged commit, the `file:line` it changed, the case it missed. Never edit earlier plan files.

Where in pack. `skills/bug-workflow/SKILL.md` (Step 0), `skills/ticket-bootstrap/SKILL.md` (Step 1b), `skills/deployed-version-check/SKILL.md`.

> Source gap: lessons say Step 0 was folded in but not yet run on a live bug. Designed, not proven.

Step 1: reproduce at the true failure boundary

What. Start every service the bug could touch. Make it break where the wrong thing really happens.

Why. A middle layer can hide the signal. A dev proxy once swallowed a client disconnect, so browser tests said "fine". Only a direct backend call with a hard connection kill triggered it.

How.

1. UI bug in the web app: write a flow (next section) and run it first. A `pass` verdict on the old build is your reproduction proof.
2. Per repo in the folder: read the README for the start command, never guess. Start through the `services` tool. Prove health. Report command, port, PID, log, proof.
3. Confirm real observable state. A clean exit is not proof. Poll the real database or downstream effect.
4. Does not break? Stop. Report exact requests, responses, environment. Next step is your call.
5. Breaks? Write `plan/repro.md`: `## Environment`, `## Steps`, `## Observed`, `## Expected`, `## Evidence`. Draft the steps as a comment in a fenced block. Post after approval. Never write the repro-steps field.

Auth blocks a live check? Invent no bypass. Read no secrets file. Try reusing a legitimate session first. Else state the blocker and use the strongest static proof.

Where in pack. `skills/bug-workflow/SKILL.md` (Step 1), `rules/global/BUG_FIXING.md` (1, 2).

> Correction: the bundled Step 1 says "start as a detached background process". The rule wins: start only through `services`.

Playwright flows

What. A **flow** is a file of steps a cheap vision model follows in a browser, returning a verdict. `flow-schema` is the format. `flow-runner` is the agent that runs it.

Why. A visual bug needs a repeatable check. Screenshots become PR evidence. It costs less than a human walkthrough.

How. One flow, three runs:

Run	env	expect	Build	Why
1	local	broken	`main`, pre-fix	Reproduce. Proves the flow can see the bug. Never skip
2	local	fixed	fix branch	Verify
3	qa	fixed	QA deploy	Retest. Human logs in

1. Load `flow-schema`. Gather all inputs: ticket and every image, vault task page, UI map note, web app source for every page touched. Source beats the map. No UI surface? Say so and stop.
2. Write `docs/flow.yaml`: routes only, never a host; targets by `role` plus `name`, or `text`; one action per step; `discriminating: true` only where broken and fixed differ; `check: screenshot` for layout, overlap, icons, visibility; no `validated` block (the runner writes it).
3. Run 1: check out the pre-fix commit. Start the web app through `services` with `repo_paths` on port 5173. **Prove the served build**: `lsof` the listener (its folder must be the pre-fix worktree) and `curl` one changed module (it must lack the fix).
4. Dispatch `flow-runner` with `flow_path`, `env`, `expect`, `base_url`, `sha`, `served_sha_verified: true`. Read `result.yaml` yourself. Check one screenshot against its citation.
5. Run 2: restart on the fix worktree, port 5173 (auto-login works only there). Run 3: `env: qa`; on `AUTH HANDOFF` ask the human to log in, then send `continue`.
6. Write one line per run under `## Flow runs` on the vault task page. Tear down.

The runner refuses `expect: fixed` with no `validated` block, a host in a step, or an unverified `base_url`. A returned bug writes `docs/flow-r<n>.yaml`. Playwright traps: `target` takes only a snapshot ref like `e24` or CSS; calls time out at 30 seconds; policy denies the evaluate, console, and network tools.

Where in pack. `skills/flow-schema/SKILL.md`, `agents/flow-runner.md`, `skills/e2e-verification/SKILL.md`.

> Source gap: lessons say local auto-login with the runner end to end was not run. Expect to debug it once.

Using the debugger

What. Step 2b. After it breaks, pause the program at the suspect spot and read real values.

Why. Prints guess. A debugger shows the exact frame, type, and value. They go straight into the root-cause doc.

How.

1. Restart only the suspect service in debug mode: `services`, `action: restart`, `debug: [<id>]`. Leave the rest normal.
2. Attach, in this order: `debug launch` with the named adapter; for a service the `services` tool started in debug mode, a raw DAP socket client in the eval Python kernel (the built-in attach timed out on a clean listener); `debug attach` only for a service started some other way.
3. Set breakpoints. Re-fire the exact reproduction request. Read `stack_trace`, `variables`, `evaluate`.
4. Never probe the debug port with a connect. `debugpy` allows one client and a probe burns the slot. Check passively: `lsof -nP -iTCP:<debug_port> -sTCP:LISTEN`.
5. What changed lately? `git -C <repo> log --since="30 days ago" --oneline -- <files>` and `git -C <repo> blame -L <n>,<n> <file>`. Record commit, PR id, date. Fix in light of that change, not around it.
6. Detach and restart in normal mode: a service paused on a breakpoint fails health checks and looks like a second bug. Skip debugging only when request and response alone prove the cause; say so.

Where in pack. `skills/bug-workflow/SKILL.md` (Step 2b), `skills/service-startup-reference/SKILL.md`, `harness/project/dap.json`, `agents/error-detective.md`.

> Rock rule: `debug launch` first. Raw DAP client for a `services`-started debug service. `debug attach` only for a service started another way. The bug skill and the service reference both say this.

The root cause doc

What. The root-cause section at the top of each `plan/fix-plan-<repo>.md`.

Why. A fresh session reads it and knows the cause without your chat.

How. Write: what you saw at the breakpoint (values, types, frame); the real `file:line` where it goes wrong; the recent change, with commit, PR id, date; the trigger request or click; what you did not prove, marked plainly.

Where in pack. `skills/bug-workflow/SKILL.md` (Steps 2b, 3).

Fix design per repo

What. Step 3. One short design doc per affected repo, plus a shared doc when two or more repos change.

Why. A design can be right once both sides ship and still break in the middle. Find that before shipping.

How.

1. `plan/fix-plan-<repo>.md` with exact files and lines.
2. Two or more repos: `plan/FIX-PLAN.md`. It links each doc and checks that each repo's design matches the other's chosen design, in plain words.
3. List every deploy order (A first, B first, together) and what the user sees at each step.
4. Spot-check a few citations against current source. Source drifts.
5. Smallest change. List every file. A file outside the list needs a one-line reason first.
6. Failing test first. Name the test file and function, write it, run it, record the failure under `## Failing test`. Then the fix. Place the test by the table in the proof part.
7. Delegating? One worker per repo, in parallel, each with a full brief: exact files, what is out of scope, exact test names and commands, and "report a blocker, do not invent a result". Then read the real diff and output. A worker's "done" is a claim.

Where in pack. `skills/bug-workflow/SKILL.md` (Step 3), `rules/global/BUG_FIXING.md` (3, 4, 6).

Live verification against the real dependency

What. Step 4. Prove the fix with real parts, not only unit tests.

Why. A unit test checks your code against your model of a dependency, not the model. A mock once said "the call rejects on abort". The real library resolves. All tests green, fix did nothing.

How.

1. Dev proxy in the way? Test the backend port directly.
2. A local UI check logs in by itself. Never ask the human to log in locally. QA needs the human.
3. A fix that depends on library, framework, or proxy behaviour needs one live test against the real thing.
4. Run the repo's PR-build checks. Live checks add to them, never replace them.
5. Run the mutation check and place the regression test (next part). UI bug with a flow: run 2, `expect: fixed`. Then tear down: close Playwright, stop what you started.

Where in pack. `skills/bug-workflow/SKILL.md` (Step 4), `skills/e2e-verification/SKILL.md`, `rules/global/BUG_FIXING.md` (5).

Commit rules

What. Step 5. How code leaves your folder.

Why. Small clean commits make review quick and amends safe.

How.

1. **One commit per repo**, code and tests together.
2. A late finding? `git commit --amend`, never a second commit. Force-push your own branch only, as `--force-with-lease=<branch>:<sha>`. Never plain `--force`. Confirm a push with `git ls-remote origin refs/heads/<branch>`, not the exit code.
3. Write the message to a file in the ticket folder, then `git commit -F <file>`. A heredoc inside `-m` gets mangled.
4. Before any push: `./secret-scan.sh <task>/<repo> <base>` from the tasks root. Push only on `RESULT: clean`. Exit 2 means scanner missing or base wrong: fix it, do not push.
5. `./self-review.sh <task>/<repo>` writes `review/self-<repo>.diff` and prints five sections: files changed, new code comments, env and secret files, dependency changes, secrets in content. Every file must be in the fix plan. Fix until `RESULT: clean`.
6. Run `pr-security-gate`. Result in `review/SECURITY-GATE.md`.
7. Open the PR without asking when all hold: it reproduced; new test failed before and passes after; mutation check passed; PR-build checks pass; live check passed; self-review clean; security gate passed (clean, deferred findings only, or the draft path). One failure, no PR.

Where in pack. `skills/bug-workflow/SKILL.md` (Step 5), `scripts/self-review.sh`, `scripts/secret-scan.sh`, `skills/pr-security-gate/SKILL.md`, `agents/pr-security.md`, `skills/pr-creation/SKILL.md`.

State moves and the team message

What. Tracker state follows facts. Slack gets one fixed line.

Why. Each move waits for a checkable fact, so nobody has to ask.

How. The Bug ladder:

Move	Who	Trigger
`New` to `Ready for Triage`	agent	bootstrap, only from `New`
`Ready for Triage` to `Ready for Fix`	agent	the bug reproduced
hold `Ready for Fix`	none	while the PR is open, in review, or revised
`Ready for Fix` to `Resolved`	agent	every linked PR merged (abandoned ones do not count)
`Resolved` to `Ready for Retest`	agent	`tasks-deployed-to-qa` proves the fix is on QA by ancestry check
`Ready for Retest` to `Closed`	tester only	human verified

- One state per write. Re-fetch. Use the `/rev` test operation.
- A merge never moves the ticket, even with the auto-transition flag.
- Link each PR to the work item. Re-fetch relations and find an entry named `Pull Request`. Do not trust the link call.
- Never write `Closed`, `Pending PO Review`, `Accepted`.

Team message: right after the PR opens, post to `#<team-channel>` with no draft and no approval. One message per PR. Exactly one line: `PR <PRID> for review (<repo>): <summary> (Bug <N>)`, plus `(re-fix)` for a returned bug. No @mentions. Draft PRs and pushes to open PRs get no post. Send with a `blocks` rich-text payload so the link renders, then re-read history. (The bundled bug command says it stops before Slack. The later rule wins.) After merge run `post-merge`: dev deploy check, evidence, tester handoff note.

Where in pack. `skills/bug-workflow/SKILL.md` (Steps 6 to 8), `skills/pr-and-team-comms/SKILL.md`, `skills/post-merge/SKILL.md`, `skills/tasks-deployed-to-qa/SKILL.md`.

> Ugh. Fall: A PR auto-merged in about 15 seconds, unreviewed, and moved the bug to `Resolved`. The reviewer was added as optional. The author reverted `main` by hand. Mark the reviewer required, re-fetch, and only then arm auto-complete.

Building a Story

Cave elder says: a house is not a beast hunt. The chief gives a plan. You follow it. If the plan has a hole, you do not guess. You go ask the chief.

Session B in one picture

What. The autonomous half of a story: contract gate to opened PR and released resources.

Why. All the talking happened in Session A.

How.

```
Step 0 contract gate        Step 5 verification gate
Step 1 bootstrap            Step 6 open PR (no prompt)
Step 2 plan per repo        Step 7 release resources
Step 3 one worker per repo  Step 8 test and acceptance states
Step 4 test + live check    Step 9 team line, decision record
```

`/story 12345` reads the state. `Backlog` or `In Refinement` goes to Session A. `Ready for Development` or later goes here. A `STORY-12345-*` folder already there means resume: move in, skip finished steps.

Where in pack. `skills/story-workflow/SKILL.md`, `commands/story.md`.

> Correction: the story command text says the Slack message waits for approval. The later rule says the PR-open line posts with no draft. Other Slack posts still need a draft.

Contract gate, then bootstrap

What. Step 0 gate (see the triage part). Step 1 bootstrap on branch `feat/<slug>`.

Why. A story that fails the gate must leave no mess.

How. Fetch the ticket and comments. Run the gate. Fail = comment, `In Refinement`, stop. Pass = the contract is the spec; do not reinterpret it. Run `ticket-bootstrap`. Do not set state; it should already be `Ready for Development`.

Where in pack. `skills/story-workflow/SKILL.md` (Steps 0, 1).

Plan per repo

What. Step 2. Move to `In Development`. Write plan docs.

Why. Plans are memory. They also check the contract against real code before you write any.

How.

1. `Ready for Development` to `In Development`. Re-fetch.

2. Per repo: `plan/story-plan-<repo>.md` with real `file:line`.
3. Two or more repos: `plan/STORY-PLAN.md`. Link each doc, cross-check designs, restate the contract's deploy-order notes against the real design.
4. Map every AC to the plan and to its verification. An AC that maps to nothing is a gap, and goes to the escalation rule.

Escalation rule, to the end. Stop when: a contract stop condition hits; the code contradicts a contract item; the work needs an unnamed repo or touches a non-goal; an AC maps to nothing or its verification cannot run. Then: comment with `file:line` evidence, tag the right person, move to `Blocked`, stop. Never reinterpret the contract. Never widen scope in silence.

Where in pack. `skills/story-workflow/SKILL.md` (Step 2).

One worker per repo

What. Step 3. One helper agent per repo, in parallel.

Why. Repos are separate. Helpers share no memory, only their brief.

How.

1. Re-check plan citations against live source.
2. Each brief holds: absolute task-folder paths; exact file list; out-of-scope list; exact test names; the exact verification command from the contract; "report a blocker, do not invent a result".
3. Read the real diff, test file, and output. Never forward "done, tests pass".
4. Start services from the main agent only. A subagent's services die with it. Parallel live runs cross-wire ports, so serialise live verification.

Where in pack. `skills/story-workflow/SKILL.md` (Step 3), `rules/global/BUG_FIXING.md` (6).

Test, with a live check

What. Step 4. Run each repo's real checks, then the live checks the contract names.

Why. A passing mock is not proof.

How.

1. Run the PR-build checks from the service reference table. Those are the commands the pipeline runs.
2. `integration`-marked tests in the two MCP repos are not in the PR build. Run them once against real dependencies.
3. Test first for every behaviour AC: write, run, record the failure under `## Failing tests` in the plan, then implement.
4. Place each test by the placement table. A change to what the assistant says or does gets a catalogue case, proven on the local orchestrator.
5. A live web-app check uses the Playwright browser on port 5173 and logs in by itself.
6. Every AC whose contract verification names a live check gets that check run for real. Report the real result.

Where in pack. `skills/story-workflow/SKILL.md` (Step 4), `skills/regression-test-placement/SKILL.md`, `skills/service-startup-reference/SKILL.md`.

The verification gate

What. Step 5. Eight checks, all must pass before any PR. It replaces an approval prompt.

Why. The author said: do not ask me when all is in order. The gate is the proof it is in order. Long, named, checkable.

How. One commit per repo first. Each item needs command output read in this session:

1. **Build.** Web app typecheck and build. Python: `uv sync` works, service imports or starts.
2. **Tests and lint.** Every PR-build command, plus integration tests once.
3. **Runs.** Each changed service starts through `services` and answers a real request on the changed path.
4. **Does what the task says.** Every AC's contract verification run, result recorded.
5. **Conforms to the plan.** Diff `main...HEAD`. Every planned change exists. No unplanned file.
6. **Mutation check** for every new unit test, in `plan/VERIFICATION.md`.
7. **Self-review.** `RESULT: clean`, every file in the plan, dependency changes named in the PR.
8. **Security gate.** `review/SECURITY-GATE.md`.

Write an AC to evidence table in `plan/VERIFICATION.md`. A failed item that the contract cannot fix means escalate. Never open a PR on a red gate. An early branch push for CI feedback is fine after a clean secret scan.

Where in pack. `skills/story-workflow/SKILL.md` (Step 5), `scripts/self-review.sh`, `skills/pr-security-gate/SKILL.md`.

Open the PR, release resources

What. Steps 6 and 7.

Why. The PR is the product. Then you clean the cave.

How.

1. One PR per repo, no prompt. Follow `pr-creation`: conventional-commit title like `feat(scope): description (STORY-NNNNNN)`, the story template, an `## Evidence` section with before and after images from the flow run folder for every visual change.
2. Do not arm auto-complete.
3. Paste each PR URL in the final report.
4. Security gate's third round still has an introduced finding? Open with `isDraft: true`, skip the Slack line, name the finding. State stays `In Development`.
5. Release everything right after the PRs open, before the Slack line: close Playwright if you opened it; `services` `action: stop` and read `port_free` and `debug_port_free`; `docker compose down` (no `-v`) for each compose project you started; confirm with `lsof -nP -iTCP:<port> -sTCP:LISTEN` and `docker ps`. Never stop what you did not start.

Where in pack. `skills/story-workflow/SKILL.md` (Steps 6, 7), `skills/pr-creation/SKILL.md`, `rules/project/teardown-verification-tooling-after-pr.md`.

The state ladder for a Story

What. Step 8 and the rungs around it.

Why. Same idea as bugs. Fact, then move. Humans keep the end.

How.

Move	Who	Trigger
`Backlog` to `In Refinement`	agent	refinement or triage starts
`In Refinement` to `Ready for Development`	agent, after you approve	end of Session A
`Ready for Development` to `In Development`	agent	Session B Step 2
any to `Blocked`	agent	escalation stop
back to `In Refinement`	agent	gate failure
`In Development` to `Ready for Test`	agent	every story PR merged
`Ready for Test` to `In Test`	agent	`tasks-deployed-to-qa` proves QA deploy
`In Test` to `Pending PO Review` to `Accepted`	tester, then product owner	never the agent

- One neighbouring state per write. A bigger jump returns HTTP 400, though the type metadata says all-to-all.
- An AC fails after merge? The contract or plan missed something. Comment with evidence, use the escalation rule.
- Step 9: the PR-open line, and one decision page per repo with code change, with tradeoffs from the resolved questions.

Where in pack. `skills/story-workflow/SKILL.md` (Steps 8, 9), `skills/tasks-deployed-to-qa/SKILL.md`, `harness/project/AGENTS.md`.

> Rock rule: The agent writes up to "ready". Tester and product owner write the end.

Proof: Tests and Verification

Cave elder says: "I killed the beast" is not proof. Show the tooth. Show the track. Show it twice.

Words, once:

- **Regression test** = a test that fails if an old bug returns.
- **Fail-first** = watch the test fail before you write the fix.
- **Mutation check** = remove the fix, keep the test. If the test still passes, it guards nothing.
- **Catalogue** = a JSON list of prompts and expected assistant behaviours, run on a schedule.

Where does the test go

What. A lookup table. The kind of thing that broke decides the place.

Why. "Add a regression test" became a lookup, not a guess. Cheap tests run every PR. Costly live ones run at night.

How. Use `regression-test-placement`:

What broke	Test goes in	Runs
A function, parser, tool	the repo's `tests/`	every PR
A field the API sends	e2e contracts file in `orchestrator`	every PR, mock mode
What the assistant says or does	prompt-regression catalogue (JSON) in `orchestrator`	nightly on dev; PRs check JSON shape only
A deck, Word file, or image	media test in `orchestrator` e2e	nightly on dev, live only
A screen or button in `web-app`	`*.test.tsx` by the component	every PR
A journey in `web-app`	`browser/*.spec.ts` (Playwright)	no pipeline today. Say so in the PR

A catalogue case needs an `id` like `regression.<area>.<slug>`, `source: regression`, a `prompt`, a `guards` line naming PR and ticket, `expect` keys (such as `completes`, `tools_none`, `min_chars`), and a generous `max_seconds`. Bump the count assertions in the catalogue rules test. The test is the truth; the README may lag.

Prove the case on the local fix, not on dev. Dev runs `main`, so the case fails there until the fix deploys; `post-merge` runs it after the dev deploy. Never edit a case to make it pass, lower a count, or delete a case without a ticket.

Where in pack. `skills/regression-test-placement/SKILL.md`, `skills/post-merge/SKILL.md`.

Fail-first and the mutation check

What. Two habits. The test fails before the fix. The test fails again when the fix is taken away.

Why. A test that never failed might never fail. A test that passes without the fix guards nothing.

How.

1. Write the test. Run it alone. It must fail. Record it under `## Failing test` (bug) or `## Failing tests` (story).
2. Write the smallest fix. The test must pass.
3. Mutation check: remove the implementation, keep the test, run the one test (must fail), restore, run again (must pass). Record both outputs under `## Mutation check` (bug) or in `plan/VERIFICATION.md` (story).
4. Pick the form by `git -C <repo> diff --quiet HEAD -- <files>`, which exits 1 for an uncommitted fix.
   - Uncommitted: `git stash push -- <files>`, test fails, `git stash pop`, test passes.
   - Committed (after commit, amend, or a returned bug): `git checkout origin/main -- <files>`, test fails, `git checkout HEAD -- <files>`, test passes, `git status --short` shows nothing.
   - Never stash a committed fix. It saves nothing, and `stash pop` restores an unrelated stash.
5. Single-test commands: Python `uv run pytest <file>::<test> --no-cov -q`; web unit `pnpm exec vitest run <file> -t "<name>"`; web browser `pnpm exec playwright test <spec> -g "<title>"`.

What is in the skills: fail-first, the mutation check with both forms, where to record them, and their use in the gates (paths below).

What is a gap, from lessons:

- The mutation check has two forms. Stash is for an uncommitted fix. Checkout from `origin/main` is for a committed fix, because stash saves nothing on a committed fix. Pick by `git diff --quiet HEAD -- <files>`. Both workflows point at the placement skill.
- Model behaviour is probabilistic. Lessons record a rule: the baseline on `main` must show the bad behaviour at least once in 15 runs, else stop; repeat runs per prompt; reset memory again after background jobs settle; add control prompts that must still call the tool; add a catalogue rule like `tools_never`. The bundled placement skill does **not** hold this text. Add it when you work on assistant behaviour.
- Mutation check, returned-bug Step 0, and self-review were folded in and not yet run on a live bug. Treat as designed, then tune.
- The global `tdd` skill was referenced by no project skill, and the pack does not bundle it. Fail-first lives in the workflow skills.

Where in pack. Fail-first: `skills/bug-workflow/SKILL.md` (Step 3), `skills/story-workflow/SKILL.md` (Step 4). Mutation check and both forms: `skills/regression-test-placement/SKILL.md`. Gate use: story gate item 6, and a bug PR condition.

> Rock rule: A test that passes without the fix is not a test.

E2E, and why a passing mock is not proof

What. Live verification through real services, storage, and model gateway, using `e2e-verification`.

Why. A mock holds your guess. A real call tests it. A mock once said "the call rejects on abort". The real library resolves. All tests were green and the fix did nothing. A verification on a stubbed API was thrown out by the author: test the real app. One live run of a probabilistic behaviour looked like proof, but a background job had undone the memory reset. Report each check as ran live, ran mock, or not run.

How.

1. Start services through `services`. Prove health and no cross-wiring.
2. Auth by service. The orchestrator has a local-dev bypass (needs `ENV=local`); read the fake token through the app's own config loader, never `cat .env`. The file gateway has no bypass and needs a real token. The web app uses real sign-in: local runs log in as a test user, QA needs the human.
3. Local browser login: navigate, click the continue button, type the key name for username and password (the MCP swaps in the real value), run the TOTP helper for the code, click Verify. Failure: report `auto-login failed: <code or step>`. Never ask the human to log in locally, type a literal credential, or read the secrets file.

Where in pack. `skills/e2e-verification/SKILL.md`, `automation/playwright/totp.mjs`, `rules/global/BUG_FIXING.md` (5), `skills/service-startup-reference/SKILL.md` (Dev-only features).

> Rock rule: Mock passing is not proof. Real call at least once. Say what was real and what was not.

Screenshots and evidence

What. Proof a reviewer can open.

Why. The reviewer has no context. A PR that says "tested" is a claim.

How.

1. Flow runs write screenshots and `result.yaml` to `docs/flow-runs/<date>-<env>-<expect>/`. Read `result.yaml` yourself. Check one screenshot against its citation.
2. Visual change: PR `## Evidence` holds a before and an after image from that folder.
3. After merge, `post-merge` writes dev evidence and a click path for the tester. Move to `Ready for Retest` or `In Test` only after the QA deploy check passes.
4. Keep these out of PR text: "Fixes N" lines, "suite passing" counts, "live-verified" claims. Evidence goes in artefacts.
5. Never put a secret in evidence. Key names only.

Where in pack. `skills/flow-schema/SKILL.md`, `agents/flow-runner.md`, `skills/pr-creation/SKILL.md`, `skills/post-merge/SKILL.md`, `skills/tasks-deployed-to-qa/SKILL.md`.

> Rock rule: Evidence a stranger can open. Not "trust me".

Running Things Locally

Cave elder says: fire is useful and dangerous. Light your own fire. Put it out when done. Never stamp on another tribe's fire.

Words, once:

- **Service** = a program in the background that answers requests. **Port** = a numbered door on your machine, one program per door.
- **PID** = the number of a running process.

Why there is a reference and a tool

What. `service-startup-reference` explains the facts. The `services` tool does the work.

Why. Ports are hard-coded across repos and tasks run in parallel. People once started services by hand and killed each other's. One owner now holds the details: the tool. Docs point at it. A copied port table goes stale the day a default changes.

How.

1. Read the reference before a first start. Then do everything through the tool. The tool code is the authority for ports and start commands.
2. A direct `hub` start or a named bash start needs the human's explicit yes. Do not send a subagent to start a service.

Where in pack. `skills/service-startup-reference/SKILL.md`, `harness/extensions/project/services/index.ts`, `harness/extensions/project/services/plan.ts`, `rules/project/services-via-services-tool.md`.

The services tool and its ports

What. Five services.

Why. You need to know each door.

How.

Service id	What it is	Port	Debug port
`file-gateway`	proxy to blob storage	9000	5681
`media-mcp`	makes images, decks, Word files	8001	5680
`data-mcp`	wraps public data APIs	8002	5679
`orchestrator`	backend API that runs the LLM agent	8000	5678
`web-app`	React and Vite frontend	5173	none

`infra` has no local start. `legacy-agent` is excluded.

Parameters: `action` (`start`, `stop`, `restart`, `status`, `logs`); `services` (list or `"all"`); `task_dir`; `repo_paths` (a worktree per service); `debug`; `ports` and `debug_ports` (strict); `env`; `lines`, `grep`.

```
{"action":"start","services":"all","task_dir":"<TASK>"}
{"action":"restart","services":["media-mcp"],"debug":["media-mcp"]}
{"action":"stop"}
```

Facts:

- Local mode needs `ENV=local` for the orchestrator fake-auth bypass.
- Orchestrator settings classes use different env prefixes: check each class. Sibling wiring is `AGENT_MCP_<SERVICE>_URL=http://localhost:<port>/mcp`.

Where in pack. `skills/service-startup-reference/SKILL.md`.

Start order

What. Who starts first.

Why. Some services need others alive.

How.

```
file-gateway --+
media-mcp ------+--> orchestrator ----> web-app
data-mcp -------+    (needs a DB)       (also needs file-gateway)
```

1. The first three have no dependency on each other. Start them together.
2. `orchestrator` next. Every start runs a local database container through compose. A Docker failure returns `docker_unavailable`: start the Docker VM yourself, ask before restarting it.
3. `web-app` last. The orchestrator loads MCP tools once at startup, so restart it if an MCP server came up late.
4. Errors carry codes such as `port_in_use` and `health_failed`. After `health_failed` the process stays up so you can look. Stop it with `action: stop`.

Where in pack. `skills/service-startup-reference/SKILL.md` (Dependency graph and start order).

Port policy and the ownership rule

What. The law about other people's fire.

Why. A lifecycle agent once killed a process another task owned. One worker's `kill -9` took down a healthy sibling. Prose did not hold, so code now enforces it.

How.

1. A process on a port belongs to another session until proven otherwise.

2. Before every start: `lsof -nP -iTCP:<port> -sTCP:LISTEN` (`lsof` lists who holds a port), then `lsof -a -p <pid> -d cwd` and `ps -p <pid> -o pid,etime,command`. A folder that is not your worktree, or an age of minutes right after you "just started it", means it is not yours.
3. Never stop, kill, or restart it. Not even with the human's live authorization. Pick another port and rewire callers, or ask the human to talk to the owner.
4. Tool policy: try the canonical port; if a foreign process holds it, take the next free port and rewire siblings. An explicit port is strict: a busy one fails with `port_in_use` and the owner's PID, folder, command. `web-app` is always strict.
5. `harness/extensions/project/process-ownership-guard.ts` enforces this. It keeps an ownership ledger. It blocks `pkill`, `killall`, `xargs kill`, and any `kill` with no literal PID. A block reason starting `process-ownership-guard:` is final. Report it. Only the human stops a foreign orphan, in their own terminal.

Where in pack. `skills/service-startup-reference/SKILL.md` (Absolute rule), `harness/extensions/project/process-ownership-guard.ts`.

> Ugh. Fall: "Never stop a foreign process" was prose. An agent killed another task's process. Later bypasses used `kill $(lsof -ti :PORT)`. Now a code guard.

> Rock rule: Never kill what you did not start. Not even if told it is fine.

Green but wrong: cross-wiring

What. Every health check green, yet the stack talks to the wrong branch.

Why. Sibling URLs are hard-coded in each repo's `.env`. A service that fell back to another port still passes its own check. Callers keep hitting whatever sits on the canonical port.

How. When more than one task's stack may run:

1. Work out expected sibling URLs from proxy config source or a startup log line. Not from `.env`.
2. `lsof` each port. The PID's folder must be your worktree. A fallback port means the caller needs the new URL; fix it from your side only.
3. After every start, re-check health fresh. Never use a cached report.
4. A chain across more than two repos? Spawn `error-detective`.

Where in pack. `skills/service-startup-reference/SKILL.md` (Cross-wiring detection), `agents/error-detective.md`.

Teardown after use

What. Stop everything you started, same turn.

Why. A Playwright browser and dev servers once stayed running after the PR. Orphan helpers hogged ports and confused the next session.

How.

1. Plan the stop before you start.
2. Stop through the tool that started it: `services action: stop`.
3. After each stop: `lsof -nP -iTCP:<port> -sTCP:LISTEN`. Report the literal result. Check child, reloader, and helper PIDs. The tool reports `port_free` and `debug_port_free`. Close the browser and `docker compose down` (no `-v`) for compose projects you started.
4. Final report: each process stopped, each port free. If not, say which and why.
5. The guard blocks a stop of something you started? Report at once and ask the human. Never use another route.

Where in pack. `rules/project/no-orphan-processes-after-use.md`, `rules/project/teardown-verification-tooling-after-pr.md`, `skills/bug-workflow/SKILL.md` (Step 4), `skills/story-workflow/SKILL.md` (Step 7).

> Rock rule: Light it, use it, put it out. Same turn.

Debug attach

What. Attach a debugger to a running service.

Why. Bug Step 2b needs it.

How.

1. Restart only the suspect service in debug mode via the tool. It runs `debugpy` with `--wait-for-client`. The debug port stays closed until a client attaches. That is normal.
2. Never use `--reload` under a debugger. Never probe the debug port with a connect: check passively with `lsof -nP -iTCP:<debug_port> -sTCP:LISTEN`.
3. Only `media-mcp` declares `debugpy`. The tool runs the other three with `uv run --with debugpy`.
4. Own the launch? `debug launch` with the named adapter. Tool-started service? The raw DAP socket client from the reference. Match every response by `request_seq`. Keep the socket open.
5. After a debug stop a helper may keep the port. The tool's stop reports `debug_port_free`. By hand, kill only a literal PID whose `ppid` is 1 and whose command is `debugpy/adapter --for-server ... --port <debug_port>` for the same repo.
6. The web app has no process debugger. Use a browser debugger. The dev server binds the IPv6 loopback: probe with the `localhost` hostname.

Where in pack. `skills/service-startup-reference/SKILL.md` (debug sections), `harness/project/dap.json`.

> Rock rule: Do not touch the debug door. Look from far away.

Shipping: The Pull Request Road

Code is done. Fix works. Now code must leave your cave and join the village. The road has gates. Each gate exists because something went wrong once. Walk in order. Agent walks with you, but you own every step.

pull request (PR) = a request to merge your branch into `main`. Others read it first.

worktree = second copy of the repo, own branch, same history. squash = all your commits become one commit on `main`.

The road in one picture

What Nine stops between "code works" and "folder tidy".

Why See the whole road, and each gate makes sense. A gate you do not understand, you skip. A skipped gate bites later.

How Read the picture. Then read each stop.

```
code done, one commit per repo
       |
  [1] security gate --- finding? fix (2 rounds max) or draft PR
       |
  [2] open PR ---------- title, short body, link ticket, reviewers
       |
  [3] team note -------- one Slack line, posts by itself
       |
  [4] review comments - reply, never resolve their thread
       |
  [5] conflict? -------- merge target in, never rebase
       |
  [6] merge ------------ gates green, one ask, auto-complete
       |
  [7] after merge ------ Resolved, dev check, tester note, vault
       |
  [8] QA proof --------- ancestry check, then Ready for Retest
       |
  [9] cleanup ---------- report first, delete only what you name
```

Where in pack `skills/workflow-router/SKILL.md` routes a ticket to the chain. Commands that start the stops: `commands/comments.md`, `commands/conflict.md`, `commands/pr-merge.md`, `commands/qa-deployed.md`, `commands/cleanup.md`, `commands/pr-status.md`.

> Rock rule: Learn the reason for each gate. Then the gate is not a chore.

Gate one: the pre-PR security gate

What A secret scan plus a security helper agent. Both read your own diff before any reviewer does.

Why Reviewer is tired and misses a hole. Machine scan finds a committed key. Helper finds a guard that fails open (lets everyone in when it breaks). Catch them yourself.

How

1. Per repo with changes, from the tasks root: `./self-review.sh <task>/<repo> origin/main`. It writes the diff to `<task>/review/self-<repo>.diff` and four secret-scan reports (commits, staged, unstaged, untracked).
2. Start one `pr-security` subagent per repo, all in one batch. Give it repo name, worktree path, diff path, report paths, plan paths (and the contract text for a story). Read line 1 of each answer yourself: `VERDICT: clean` or `VERDICT: findings`.
3. Apply each ruling:

Ruling	You do
clean	Gate passed.
false-positive	Add a rule-scoped allowlist entry to `gitleaks.toml` in the tasks root. Rerun. Not a fix round. Nothing goes into the repos.
fix-now	Amend the same commit (`git commit --amend -F <file>`). Rerun checks and `self-review.sh`. Rerun the live check if a non-test file changed. Ask `pr-security` again.
too-big	Draft PR path.
defer-bug	Draft one Bug and one ticket comment. PR goes on.

4. At most 2 fix rounds. A third verdict with an unresolved finding you introduced takes the draft path. Think an "introduced" tag is wrong? Show `file:line` proof and ask once more. The second answer is final.
5. Write `<task>/review/SECURITY-GATE.md`: one section per repo per round, verdict lines copied word for word, then "Drafts awaiting approval", each in its own fenced block.

Origin decides: a finding on a `+` line of your diff is "introduced", so fix it now. Any other is "pre-existing", so file a Bug and let the PR go on.

Light check. Before any other push of your own code (early CI push, amend force-push, push onto a colleague's branch), run `./secret-scan.sh <task>/<repo> <base>`. Push only on `RESULT: clean`. Exit code 2 means scanner missing or base wrong. Do not push.

Draft PR path. Open with `isDraft: true`. Link the ticket, add reviewers. No auto-complete, no Slack line, no ticket move. Draft a Bug. Publish with `isDraft: false` once the gate passes. Re-fetch and check.

Secret already on the remote. An amend changes the commit id, so "which remote branches contain this commit" lies. Fetch the branch with an explicit refspec and scan the pushed range. A failed fetch means unknown: treat the secret as exposed. Remove it in this PR, draft a rotation Bug, and say "rotate credential" first in your report. Cite rule id and `file:line`. Never print the value. A merge-conflict resolution is exempt from this gate.

Where in pack `skills/pr-security-gate/SKILL.md`, `agents/pr-security.md`, `scripts/self-review.sh`, `scripts/secret-scan.sh`.

> Ugh. Fall: A scanner allowlist had no effect and gave no warning. A global allowlist is ignored when default rules are extended. Fix: rule-scoped block, and two smoke tests per scanner config (one case must fire, one must pass). The sources do not record results for this gate's nine planned checks, so test it on a throwaway branch first.

> Rock rule: Never print a secret. Cite the rule id and the line. Nothing more.

Opening the PR

What One branch, one title, one short body, one linked ticket, the right reviewers.

Why Every repo uses the same headings, so a reviewer reads any PR the same way. Body must stand alone: the reader has not seen your ticket, branch, or chat.

How

1. Gate one passed. Branch is `<type>/<short-dash-summary>`, made from `main`.
2. Title is a Conventional Commit with ticket suffix: `type(scope): description (BUG-12345)`.
   - `type`: feat, fix, docs, refactor, perf, test, build, ci, chore, revert. `!` before the colon only for a breaking change. `scope`: the feature area, not the repo name.
   - `description`: lowercase, imperative, says the new behavior. Whole title under 100 characters. It becomes the squash commit message, so it must stand alone.
3. Body is template A (bug fix) or B (feature or chore). Exact headings, exact order, no extra heading.
   - A: `## Bug`, `## Root cause`, `## Fix`, `## Tests`, `## Evidence`.
   - B: `## What changed`, `## Why`, `## Tests`, `## Evidence`.
4. Body is 200 words or fewer, 4 bullets or fewer per section. `## Evidence` holds one before image and one after image for a visible change, one line on what each shows. Otherwise write "No visual change."
5. Create with the PR tool. It needs the repo id (a GUID), not the name. Fetch ids once with the repository list tool.
6. Link the work item. Re-fetch it and look for an `ArtifactLink` named "Pull Request" whose URL ends in your PR id. Do not trust the link call's own "ok". Branch policy can block an unlinked PR.
7. Add the standing optional reviewers (tech lead and main-workstream teammates). Ask the branch policy who is already attached (the CLI, a sanctioned exception, because no MCP tool shows it). Add only the missing people. Never add yourself.
8. Paste the PR link back in the same turn. Opening needs no prompt: the verification gate in the bug or story workflow is the permission.

Forbidden wording. Never write: "Fixes BUG 12345" or a hand-written "Related work items" (the tracker adds that line); "Live-verified against..." (a live check is part of a fix); a paragraph on what you left out of scope (file a Bug, cite it in one sentence); "Full suite: N/N passing"; "as discussed" or "see the diff"; an em dash; filler words (robust, comprehensive, properly, simply, just, significantly, cleanly). One fix across two repos? End each body with one sentence giving the other PR link.

Where in pack `skills/pr-creation/SKILL.md`, `skills/bug-workflow/SKILL.md` (Step 5 opens the PR), `skills/story-workflow/SKILL.md`, `rules/project/azure-cli-fallback.md`.

> Rock rule: Body for a stranger. If only you can read it, it failed.

The auto-complete trap

What Auto-complete merges a PR by itself once all policies pass.

Why A PR once auto-merged unreviewed in about 15 seconds. `main` needed a manual revert. The reviewer tool can add only optional reviewers, and auto-complete was set right after. No policy required an approval yet.

How Only this order is safe: (1) mark the reviewer required with the CLI, since no MCP tool has that flag; (2) re-fetch the PR and check the reviewer reads as required; (3) only then set `autoComplete: true`. Error `TF401181` on step 1 means the PR already completed. You are late. Story Session B never arms auto-complete. The merge flow arms it last, on fresh gate reads.

Where in pack `skills/pr-creation/SKILL.md` (Auto-complete safety), `skills/pr-merge/SKILL.md` (Step 5).

> Rock rule: No auto-complete until a fresh read shows a required reviewer.

The team note: one Slack line, automatic

What After a PR opens in the bug or story workflow, one line goes to `#<team-channel>`. No draft. No approval.

Why Every earlier approval got "yes". A prompt that always gets "yes" is noise. This line has fixed shape and no judgment, so it is safe.

How Exactly one line:

```
PR <PRID> for review (<repo>): <summary> (<Bug|Story> <N>)
```

- `<PRID>` is a link to the PR. `<N>` is a link to the work item.
- `<summary>` is the PR title with the `type(scope):` prefix removed. Do not reword.
- A returned bug's new PR adds `(re-fix)` at the end.
- Example: `PR 4567 for review (orchestrator): stop generating images for deck pictures (Bug 12345)`

Rules:

1. One post per PR (a ticket with PRs in two repos gets two posts). No @mentions: branch policy adds reviewers. A push to an open PR gets no post. A draft PR gets none.

2. Send with Slack `blocks`: one `rich_text` block with `text` and `link` elements, `text` set to the plain line as fallback. `<url|label>`, bare URLs, and markdown content type did not render links and collapsed lines. A post cannot be edited. Keep `%20` in tracker links. Test a new layout first as a DM to yourself.
3. After posting, read channel history and confirm it landed.

Any other Slack post: draft in a fenced block, wait for approval. "Talking to Tribe" covers it.

Where in pack `skills/pr-and-team-comms/SKILL.md`, `harness/project/RULES.md` (the one exception to the Slack gate).

> Rock rule: One fixed line posts alone. Everything else gets a draft.

Handling review comments

What Reviewer wrote comments on your PR. You answer each with evidence. Command: `/comments <pr>`.

Why An agent that "fixes" a comment it never reproduced fixes a ghost. Reviewer wants proof.

How Agent picks the mode from the PR author. Your PR: author mode. Someone else's PR that holds your comments: reviewer mode.

Author mode:

1. Find or create the task folder for the branch. Read every thread word for word. In scope: first comment is real text, not deleted, status Active or Pending, last comment not yours. Skip system threads. Write `comments/THREADS.md`.
2. Re-read the code at each cited `file:line`, because the branch may have moved.
3. Sort each thread, show a table, ask once. You may change any row.

Class	Meaning	Reply
fix	Real, belongs here	Reproduce, fix, verify, reply with commit
answer	Question, or code refutes claim	Reply with `file:line`
disagree	Real finding, wrong proposal	Reply with reasoning
defer	Real, out of scope	File a Bug, reply with its number

4. Each `fix` is a bug. Reproduce first at the true failure boundary. A finding that does not reproduce becomes an `answer`. Fix the source. Never special-case the reviewer's example input. Run the PR-build checks. A failing check may already fail on the target: prove it with a scratch worktree.
5. Commit once, message in a file, `git commit -F`. PR has 1 commit: amend, push with `--force-with-lease`. PR has more: add one commit, plain push. Never amend a merge commit. Never write "address review comments".
6. Run the security gate with base `origin/<src>`. Ask before pushing.

7. Draft each reply in chat, in its own fenced block with thread id. It posts under your name, so use your voice. Cite `file:line` on the pushed commit. A fix names the commit. A defer names the Bug. Never "addressed" or "done" without evidence. Post in the same thread. Re-list and confirm.

The thread resolution rule. Agent pushes the fix and replies. Agent leaves the thread **Active**. Only the person who started a thread may mark it Resolved, Closed, or Won't Fix. A rule file watches the status-update tool. Two narrow exceptions live in the merge flow, on your own PR only: a `superseded` thread (every other commenter approved after their last comment) and a `chit-chat` thread you answer "Close". Nothing else.

Reviewer mode (your comments on someone else's PR): never edit, commit, or push on their branch. Agent writes `review/VERIFY-HANDOFF.md` for a fresh session to verify. Sources say it was written but not run against a real PR: treat it as untested.

Where in pack `skills/pr-comments/SKILL.md`, `commands/comments.md`, `rules/project/pr-thread-resolution.md`, `skills/pr-security-gate/SKILL.md`.

> Rock rule: You fix. You reply. They close.

Merge conflicts

What Your PR no longer merges cleanly into `main`. Command: `/conflict <pr>`. It resolves the conflict and nothing else.

Why Others may have pulled your branch. Merge keeps history they know. Rebase rewrites it and breaks their copy.

How

1. Find the PR: one lookup per in-scope repo. Stop before any git command when the PR is not Active, is not yours (never touch another person's branch), has no conflict, or shows a policy rejection. Status "still computing": re-check after 15 seconds, up to 3 times.
2. Find the task folder from the repo symlink target. None? Create one with `setup-task-branch.sh --pr`. Sync the worktree with explicit fetch refspecs (a rule that tells git which branches to fetch). Bare clones have none by default, so a plain fetch does not move `origin/main`.
3. Save your PR's own diff first: `git diff origin/<target>...HEAD` (three dots), to an absolute path under `<task>/conflict/`. A redirect in a symlinked worktree lands inside the bare repo otherwise.
4. Merge the target in: `git merge --no-ff origin/<target>`. Expect a failure exit. Here `--ours` is your PR branch and `--theirs` is the target (rebase flips this). Read both sides of each hunk. Keep both intents. Leave no conflict marker. Never hand-merge a lockfile: fix the manifest, take the target's lockfile, regenerate it.
5. Prove nothing was lost. Save `git diff origin/<target>..HEAD` (two dots). Compare with the first patch. Explain every differing line. A vanished hunk of your change means a bad resolution: go back to step 4.
6. Run the PR-build checks. Write `conflict/CONFLICT-NOTES.md`. Ask. Then push with plain `git push origin HEAD:refs/heads/<src>`. Never `--force` or `--force-with-lease` here (unlike `pr-comments` on an amended commit). The merge only adds a commit.

7. Re-check merge status every 15 seconds, up to 4 times, until it says no conflict.

Never stash during a merge: `stash pop` once dropped the merge state. Never rebase. Never touch tickets or comments.

Where in pack `skills/merge-conflict/SKILL.md`, `commands/conflict.md`, `scripts/setup-task-branch.sh`.

> Ugh. Fall: Merging `main` into a branch silently retired part of an already shipped fix, because two designs touched the same files. Fix: state the trade-off in the PR, ask a human to confirm, prove the net change with the before and after patch diff.

> Rock rule: Merge, never rebase. Compare patches before you push.

Merging your own PR

What Command `/pr-merge <pr>`. Agent checks six gates, clears failures, asks once, then arms squash auto-complete.

Why A merge is hard to undo. One combined ask beats many small prompts. Auto-complete armed on stale facts once merged a PR unreviewed.

How

1. Find the PR. Abandoned: stop (a human decision). Already merged: jump to the ticket step. Active: go on. Not yours: stop. This skill merges your own PRs only.
2. Build the gate table from five parallel reads: votes, threads, pipeline, linked items, policies.

Gate	Pass rule
Approvals	At least one approve from someone who is not the author. No "waiting for author" or reject from anyone.
Comments	No open `feedback` thread.
Conflicts	Merges cleanly.
Pipeline	Newest build on the PR merge ref passed, against the current `main` tip.
Paired order	Title ends `(k/n)` with k above 1: every earlier partner already merged.
Policies	Every blocking policy approved or not applicable. Read through the CLI. Login declined: row says "not read" and does not block.

3. Thread classes. `superseded`: every other commenter approved after their last comment, so the run closes it with no reply. `chit-chat`: praise or thanks only; you pick Close or Keep open. `feedback`: anything else. It blocks. Show words, not codes: approver name plus "approved", a star for "approved with comments".

4. Clear failing gates in this order, because a push resets votes and re-queues policies: conflict (subagent resolves it, or stop; the push stays behind your ask); comments (resolve with `pr-comments`, or DM the reviewer); approvals (offer a DM; a vote is a human act); pipeline (failed: stop; stale: next point); policy (name it and stop; never `bypassPolicy`).
5. Stale build: `main` moved, so the green build expired. A plain new pipeline run does NOT attach to the policy. Use the policy re-queue command (the PR page's Re-queue button), poll every 30 seconds, then re-run every gate on fresh reads. A table from before a push is stale.
6. One combined ask: repo, PR, source to target, squash, approvers, build, threads to close, ticket move, vault action. One question per chit-chat thread, then "Go" or "Stop".
7. On "Go": close the `superseded` and chosen chit-chat threads, re-read the PR, and abort if merge status, a vote, or the threads changed. Then arm auto-complete (the trap above): squash, delete source branch, `transitionWorkItems: false`, `bypassPolicy: false`, no merge commit message.
8. Re-read every 15 seconds, up to 4 times, until merged. Record the squash commit. Confirm the source branch is gone. Still active with auto-complete set: a policy waits. Name it and stop.

Name every approver, because the "1 Owner approval" rule has no machine-readable list. Never vote to satisfy a gate. With no PR number, agent only lists your active PRs, checks each read-only, and asks which to merge. It never merges a PR you did not pick.

Where in pack `skills/pr-merge/SKILL.md`, `commands/pr-merge.md`, `skills/my-pr-status/SKILL.md`, `skills/pr-comments/SKILL.md`, `skills/merge-conflict/SKILL.md`.

> Ugh. Fall: Green build, but the PR would not auto-complete. `main` moved and the build expired. A fresh run did not satisfy the policy. Fix: classify the build against the `main` tip, re-queue through the policy action, re-read policies.

> Rock rule: Fresh reads, one ask, never bypass policy.

After the merge: state moves and vault entry

What Merge is not the end. The ticket moves one step. A tester note gets written. The vault learns what happened.

Why The gap between "merged" and "tester retests" is where fixes get lost.

How State ladder. One neighbouring state per write, then re-fetch:

Fact	Bug	Story	Who writes
Every linked PR merged	Ready for Fix to Resolved	In Development to Ready for Test	Agent, no prompt
QA proof (next section)	Resolved to Ready for Retest	Ready for Test to In Test	Agent, no prompt
Tester verified	Closed	Pending PO Review	Tester only
Accepted	none	Accepted	Product owner only

Rules for the write:

1. Fetch the item with relations. "All merged" means every linked PR Completed. Only some? No move: report "N of M merged".
2. Write a `test` on `/rev`, then `replace` the state. Re-fetch and check. A merge never moves the item, even with `transitionWorkItems: true`.
3. A returned bug (state went back from Resolved or later) is never overwritten. Agent never writes Closed, Pending PO Review, or Accepted.

Then `post-merge`, once per merged PR:

1. Dev deploy green: the deploy build on `main` is at or after the squash commit and succeeded. Running: look every 60 seconds, up to 20 times. Failed: stop and report.
2. Dev evidence. Web change: dev sits behind a private endpoint, so record "dev: not reachable from this host" and use the local flow run. API or answer change: run the API e2e regression pipeline against DEV (about 20 minutes) and confirm the new catalogue case passed.
3. Write `<task>/docs/HANDOFF.md` for the tester: what changed (user words), where to test, steps with expected results, evidence, a three-item tick list.
4. Draft a ticket comment in a fenced block. Post only after approval:

   ```
   @<tester> <what now happens, one sentence>. On <env>: <click path in one line>. Steps
   and evidence: <key steps>.
   ```

   Tester is the bug's creator, or whoever last moved it to "Bug Test Failed". Never reassign.
5. Vault: find the task page by ticket id. Append under `## Post-merge`: `- <date> · deploy <build id> <result> · dev: <evidence> · handoff docs/HANDOFF.md · comment <id or "not posted">`

The merge skill also closes the vault task page (status `review` to `done`) and appends one log line, `## [date] task | Completed PR <repo> #<pr>`, after reading the last lines of `log.md`. Never edit a past entry.

Decision records: per repo with a real code change, one page in `decisions/` (Decision, Context, Reasoning, Consequences). Reusable facts go in `notes/`. The skills and `vault/schema.md` agree on this layout.

Where in pack `skills/pr-merge/SKILL.md` (Steps 6, 7), `skills/post-merge/SKILL.md`, `harness/project/RULES.md`, `harness/project/AGENTS.md`, `vault/schema.md`, `vault/templates/decision.md`, `vault/examples/tasks/fix-meter-reading-rounding.md`, `vault/examples/decisions/2026-03-09-store-readings-as-decimals.md`.

> Ugh. Fall: The bug state rule changed three times in five weeks. A state write also returned HTTP 400 on a jump. Fix: one ladder note plus the `get_type` tool, one neighbouring state per write, skills hold only the triggers.

> Rock rule: Facts move state. Merge moves one step. QA proof moves the next. Tester closes.

The deployed-to-QA check

What Proof that your fix really runs on QA. Only then does the ticket say "ready to retest".

Why Tester must never retest a build without your fix. In the launch period, testers gave negative feedback on pre-fix code, because fixes sat in `main` but not yet on QA. Dates lie: a fix merged before a deploy can sit on a branch that deploy never ships.

How Two skills. `deployed-version-check` (`/qa-deployed [service]`) says what runs on QA. `tasks-deployed-to-qa` says which of your tasks are really live.

1. Find the real deploy pipeline per repo. Some repos have two definitions with a shared name. One builds on every `main` push but skips the push and restart. It looks like a deploy in history and never touches the live app. The other fires on a git tag. Use that one. Read its latest successful build's `sourceVersion`. That is the deployed commit.
2. Cross-check the live app config where the CLI allows. Denied? Report the gap, use pipeline evidence, mark low confidence. Say `high` only with fresh evidence.
3. For each item, read the linked PR and "Fixed in Commit" relations. One item can carry commits in several repos. Record each pair.
4. Prove with ancestry, never dates. Ancestry asks: is the fix commit in the deployed commit's history? Run `git merge-base --is-ancestor <fix-sha> <deployed-sha>` after fetching with explicit refspecs (heads and tags, prune). Exit 0 is confirmed. Any other exit is not proven. Never round up to "probably fine".
5. Report one row per item, one sub-row per repo: CONFIRMED or NOT CONFIRMED. A production-provisioning story deploys nothing to QA: say "not applicable".
6. After a confirmed check: Bug Resolved to Ready for Retest. Story Ready for Test to In Test. No prompt. Re-fetch.

Corrected reading of pack text. `skills/tasks-deployed-to-qa/SKILL.md` Step 2 says to exclude Resolved and Ready for Test. The rules and other skills use the check as the gate that leaves those two states. Run it on items in Resolved or Ready for Test, and on success write the next state. Its Step 5 runs git in a bare clone path, which a guard forbids. Use the task-folder symlinks or the dedicated `main` folder.

Where in pack `skills/tasks-deployed-to-qa/SKILL.md`, `skills/deployed-version-check/SKILL.md`, `commands/qa-deployed.md`, `rules/project/no-parent-folder-repo-access.md`.

> Ugh. Fall: The web app merges to `main` but QA deploys a separate `qa` branch. A date check would have been wrong for exactly that repo.

> Rock rule: Ancestry or it did not happen.

Task cleanup

What Remove finished task folders and their worktrees. Command: `/cleanup`, from the tasks root.

Why Every ticket leaves a folder and worktrees (second copies of the repo, own branch). Disk fills. Deleting work is destructive, so the tool reports first.

How

1. Stand at the tasks root (MCP servers load only from there). Run `./cleanup-task-folders.sh report`. Read-only. It prints one record per task folder, repo link, stray root file, and orphan worktree.
2. For each repo with unique commits, look up PRs by source branch, all statuses (Active beats Completed beats Abandoned). Get ticket state. PR-review folders use the PR status. Research folders have no ticket.
3. Classify:
   - **CLEAN**: every repo with unique commits has a Completed or Abandoned PR; nothing dirty, ahead, or diverged; age at least 7 days; ticket state Resolved, Ready for Retest, Closed, Cancelled, Duplicate, or Deffered (or no ticket).
   - **CLEAN (unused)**: no unique commits, any ticket state. **KEEP**: anything else, with the single strongest reason.
4. Show one table: folder, age, size, verdict, reason. Orphans and strays below. Ask once. Human may drop rows. Do nothing before approval.
5. Apply in order: `clean --apply`, `clean-worktree --apply`, `strays --apply`, `prune --apply`. Every mutating command is a dry run without `--apply`. The script archives markdown and text files, then removes worktree, local branch, and folder.
6. Batches of 3 or 4 folders, with an explicit `timeout`. A big batch once died at the 300 second tool cap after 7 of 14 folders. `async` does not lift the cap.
7. Report freed size, archive paths, and one line per kept folder.

Never delete a remote branch, push, complete or abandon a PR, or change a ticket state. `--force` needs your instruction naming the folder. Until cleanup runs, never remove a task-folder symlink, branch, or worktree the script made.

Where in pack `skills/task-cleanup/SKILL.md`, `commands/cleanup.md`, `scripts/cleanup-task-folders.sh`, `harness/project/RULES.md`.

> Ugh. Fall: An agent tidied out-of-scope branches, worktrees, and symlinks. The author needed them for testing. Now the wiring script has an exclusion list, and a rule says leave links alone.

> Rock rule: Report first. Delete only what the human names.

Judging Others: Code Review

Now you are the reviewer. Someone else's PR waits. Their branch is their field. You read. You write comments. You do not plant. You do not pull. Agent gives you three helper reviewers. Helpers are fast. Helpers also over-claim. You stay the judge.

What review is for

What A verdict on a PR you did not write: suspicious or not, does it do what it says, is it good, does it fit the whole system.

Why One tired reviewer has one blind spot. Three lanes cover three kinds. AI also raises PR volume faster than humans can read. The team asked to flag the important issues, not many comments.

How Command `/ado-review <pr-number>`. It loads `skill://pr-review` and runs to a verdict. An agent's review is a lane, not a vote. The human reviewer stays the merge authority.

Where in pack `commands/ado-review.md`, `skills/pr-review/SKILL.md`.

> Rock rule: Helper reports. You decide.

Read the ticket first

What Find the PR. Pull the linked work items. Read them before the code.

Why The ticket is the spec the PR claims to implement. Without it you cannot judge "does it do what it says". A changed file the description never mentions is a lead.

How

1. Find the repo: list PRs with status All. The row names it. Fetch the PR with work item refs and changed files. Capture title, description word for word, branches, head commit, author, reviewers and votes, linked items, changed files.
2. Fetch each linked item with expand `All` (capital A; lowercase is rejected). Capture type, title, state, description or repro word for word, parent. None linked? Say so in one line.
3. Wire a read-only worktree from the tasks root:

   ```
   ./setup-task-branch.sh --pr <repo> --target-branch <target> <src-branch> PR-REVIEW-12345-<summary>
   ```

   Leave out `--target-branch` when the target is `main`. The script creates the folder, refuses a duplicate `PR-REVIEW-<number>-*`, fetches with explicit refspecs, and links one repo with no `.env`.
4. Check the worktree head equals the PR's last merge source commit. If not, the branch moved: rerun. Then `mkdir -p review`, run `./setup-task-lsp.sh`, and move the session into the folder.
5. Write `review/PR-CONTEXT.md` (metadata, description, each linked item, changed files) and `review/diff.patch` (`git diff origin/<target>...HEAD`, to an absolute path).

Where in pack `skills/pr-review/SKILL.md` (Steps 1 to 4), `scripts/setup-task-branch.sh`, `scripts/setup-task-lsp.sh`.

> Rock rule: Ticket first, then code. Check the head commit.

Three lanes in parallel

What Three read-only helper agents, started in one batch. Each reads `PR-CONTEXT.md`, the worktree, and `diff.patch`.

Why One lens misses things. Run all three together.

How First **you read the diff yourself**. You write the final summary. You must be able to challenge each helper.

Lane	Agent	Output	Question
Security	`security-reviewer`	`SECURITY-REVIEW.md`	Anything suspicious or malicious?
Quality	`reviewer`	`QUALITY-REVIEW.md`	Convention, spec match, test strength?
Architecture	`architect-reviewer`	`ARCHITECTURE-REVIEW.md`	Boundaries, contracts, risk?

Security lane: read line by line for backdoors, inverted logic, weakened guards, exfiltration. Trace every consumer of each changed value: fails-closed, fails-open, or crashes. Check new tests still assert what they claim. Rule on any changed file the description omits.

Quality lane: repo convention; spec match both ways (ticket scope missing from the PR, PR scope missing from the description, description claims the code does not make); test strength. For test strength: a regression test sits where `regression-test-placement` says; the PR or its plan records a mutation check (test fails with the fix removed, passes with it back); before and after images for a visible change; no new code comments; no new dependency without reason; smallest change. A missing mutation record is a finding. The lane must not run tests to find out. Each finding names "one real way it could break". Example: a fix matched "please confirm" anywhere in a reply and cut short every drafted email holding those words.

Architecture lane: boundary breaks, contract stability for other repos, new response fields (the e2e contracts file forbids unlisted fields, so one fails the e2e run), design that concentrates risk.

Shared contract: read-only, no formatters, linters, or test suites. Every finding cites `file:line`. `## Verdict` first, then `## Findings` rated Critical, High, Medium, Low, Info. A dormant advisor, `PRSecurity`, wakes when `pr-review` loads and speaks up on a fails-open finding reported clean.

Where in pack `skills/pr-review/SKILL.md` (Step 5), `agents/architect-reviewer.md`, `harness/project/WATCHDOG.yml`, `skills/regression-test-placement/SKILL.md`.

> Rock rule: Read the diff before you send helpers.

Compile and check every claim

What One file, `review/REVIEW.md`, and one chat report.

Why The human reads one verified report, not three raw ones.

How

1. Answer five questions in order: suspicious or malicious content (yes or no, with evidence); does the change do what description and ticket claim (gaps both ways); quality findings by severity; architecture trade-offs by severity; recommended action (approve, approve with comments, request changes).
2. **Check every helper claim against code before you repeat it.** Cite `file:line` yourself, not "the agent said". Lanes may disagree: one marks a claim inference, another proved the opposite.
3. Chat report, two parts split by a rule line. First `**Ticket summary**` (3 to 5 lines: id, type, title, the ask, the claimed fix; or one line saying none is linked). Then `**Review summary**` with the five answers. The same two parts head `REVIEW.md`. Translate helper jargon to plain words.

Where in pack `skills/pr-review/SKILL.md` (Step 6), `harness/global/APPEND_SYSTEM.md`.

> Rock rule: A claim you did not check is not yours to repeat.

Writing comments

What Draft in chat. Post only when the author confirms each comment.

Why Comments post under the author's name. A long justification chain hides the point. A wrong patch costs a second review.

How Style notes from the pack:

1. Short, plain language. State the problem. Cite the concrete contract or file it breaks. Ask one direct question. Full reasoning lives in `REVIEW.md`.
2. **Small fix only when no design choice.** A few lines, one function: add a proposed-fix code block with no code comments inside it. A fix that needs a design decision stays a question. Say so. Do not draft a diff that needs its own review.
3. Iterate in chat: score each comment X out of 10 for importance, simplify the words, then add fixes only where a small one fits.
4. Strip incidental data (build ids, other pipeline files, other repos' branches) unless it is the finding.
5. Test a fix against the real failure mode. A sanitizer that returned the raw matched text still held link syntax. One that returned only the title still carried attacker text. What closed the gap: drop the tag when the URL is unverified.

The anchor rule. Every thread anchor (file path plus line range) sits on a file in the PR's changed-file list, inside its diff hunks. Comment text may name an untouched file when the diff's own change reaches into it. Findings the diff does not reach go in `REVIEW.md` and chat only. A thread anchored outside the diff never rendered and had to be deleted.

Posting after approval of each comment: one thread per finding, anchored with file path, start line, and end line. The API rejects an anchor without start and end offsets: use offset 1 and the line length. **Deleting** a posted comment: the MCP tool has no delete action. Use the CLI invoke call (a sanctioned exception), then set the empty thread shell to Closed.

Where in pack `skills/pr-review/COMMENT-STYLE-NOTES.md`, `skills/pr-review/SKILL.md` (Boundaries), `rules/project/azure-cli-fallback.md`.

> Rock rule: Name the problem. Fix only if tiny.

Never touch their branch

What Someone else's branch is read-only for you.

Why Their history is theirs. A push can break their local copy.

How Never edit, push, merge, or abandon another person's branch. A read-only fetch into the review worktree is fine. Never set a thread to Fixed, Closed, or Won't Fix when another person started it. Never vote, comment, or complete the PR without explicit approval in the session.

Where in pack `harness/project/AGENTS.md` (Someone else's PR), `skills/pr-review/SKILL.md`, `rules/project/pr-thread-resolution.md`.

> Rock rule: Reviewer reads. Author writes.

A vote is not merge authorization

What Casting "Approved" can merge the PR on the spot.

Why If the owner armed auto-complete and your vote satisfies the last blocking policy, the PR squash-merges at once. This happened.

How Before you vote approved, or approved with suggestions, on someone's PR:

1. Re-read the PR. Is auto-complete set, and by whom?
2. Read live policy state. No MCP tool shows it. Use the CLI.
3. Would your vote satisfy the last blocking policy? Then **ask the human first**.
4. Name votes in words: approved, approved with suggestions, waiting for author, rejected. Never a number.

Where in pack `harness/project/RULES.md`, `skills/pr-review/SKILL.md`.

> Ugh. Fall: An Approved vote on a colleague's PR merged it at once. A vote and a merge are two different acts.

> Rock rule: If your vote would merge, ask first.

Talking to Tribe

Code is half the work. Words are the other half. Words go to the ticket, the PR, Slack, your manager. Each place has its own manners. Wrong manners cost days.

tribe = everyone who reads your words: tester, tech lead, teammates, product owner, manager.

One finding shapes all the rules below. In a study of 28 confusion and friction cases, confusion came from identifiers and undefined labels, not from length. Short is good. Clear is better.

Ticket voice

What How to write in a work item: description, acceptance criteria, comments.

Why A ticket is a handoff note. A cold report ("the investigation found...") feels like a wall. A plain first-person note feels like a teammate.

How

1. Plain, first-person words, as if the requesting engineer wrote them. No third-person report tone. Source citations stay.
2. Write tag names without angle brackets. The tool strips them and leaves empty backticks.
3. Comments carry reasons and questions to people. Tag who must know. Keep short. No plan dumps.
4. Ask before you change a title, description, or state, and before you comment. No ask for: area path, iteration path, assigning to yourself, the state moves in the ladder of "After the merge", and any move or post a skill step orders. A step that says "show the draft" keeps its gate.
5. After any write, re-fetch and check the field. Always pass the project name, or you get "Client does not support form elicitation".
6. A story that fails its contract gate gets a comment naming the gaps. The tester comment after a merge has a fixed shape: see "After the merge".

Where in pack `rules/project/ado-ticket-voice.md`, `rules/project/ado-area-path.md`, `rules/project/tool-gotchas.md`, `harness/project/RULES.md`.

> Ugh. Fall: The voice rule first matched only the write call, after drafting was done. Its scope then widened to text drafted in chat. A rule must fire where words are made, not where they are sent.

> Rock rule: Write the ticket as the person who asked for it.

PR text

What Title and body of a PR. "Opening the PR" has the recipe. Here is the manner.

Why PR text stands alone in history. The title becomes the commit.

How

1. Plain full sentences. Expand an acronym on first use. No pasted logs or diffs. Avoid every word in the forbidden list.
2. Commit message: what changed and why, in the PR voice. Written to a file, committed with `-F`. Never "address review comments".
3. Add no code comments. `self-review.sh` flags new comments in Python, TypeScript, and TSX files. None inside proposed fixes either.

Where in pack `skills/pr-creation/SKILL.md`, `skills/pr-comments/SKILL.md`, `scripts/self-review.sh`.

> Rock rule: PR text says what changed and why. Nothing else.

Slack: draft or auto

What Two kinds of Slack post. One posts by itself. All others wait for a draft and approval.

Why A Slack post has an audience. It cannot be edited, and the tool cannot delete it. So most posts earn a gate. The one fixed line does not.

How

Post	Gate
PR-open line (fixed shape, one per PR)	Posts by itself.
Review ask, disruption notice, any other post	Fenced draft. Post after approval.
Merge-flow DM to a reviewer, open-PR list	Draft in chat. Post on "send it".

1. Default channel is `#<team-channel>`, unless the author names another. An engineering channel is for review asks and disruption notices, on request. To find a reviewer, check who authored recent PRs in that repo, then ask in the team channel and tag by name.
2. Post with `blocks` (rich text, link elements). Test a new format as a DM to yourself. Never post secrets, tokens, or `.env` contents. After a post, read channel history and confirm.

Where in pack `skills/pr-and-team-comms/SKILL.md`, `skills/wiki-comms/SKILL.md`, `skills/my-pr-status/SKILL.md`, `skills/pr-merge/SKILL.md`.

> Rock rule: Auto only for the one fixed line. Everything else, draft first.

Approval language

What The words that mean "yes, do it".

Why Prose gates get missed under autonomy. Clear words make a gate easy to pass and hard to fake. At least 35 accepted answers were one token: "A", "go", "send it", "approved".

How

1. Strong approval: "send it", "approved", "write it", "go". These open the gate for posts, ticket comments, commits, and PR actions with an audience. Not sure? Ask one direct question. Do not guess.
2. Show the next iteration only, then stop. The human's next message is an iteration request, not approval. Send approved text unedited.
3. Drafts say exactly what was dictated. "Link only" means link only. "Never shorten a draft" is not permission to add background, fixes, emoji, or sign-offs. Show drafts in a fenced block.
4. Turn always-yes prompts into fact checks. The author said "do not prompt me when everything is in order" about state moves, opening the PR, and the PR-open note. Keep approval for posts with an audience.

Where in pack `harness/project/RULES.md`, `harness/global/APPEND_SYSTEM.md` (Drafts), `skills/wiki-comms/SKILL.md`.

> Rock rule: "send it" opens the gate. Silence does not.

The writing standard (ASD-STE100) in plain words

What A simple-English standard for technical writing: ASD-STE100 (Simplified Technical English). The pack uses a short working subset for every sentence the agent writes.

Why Short, plain sentences cut misreads. One word, one meaning removes the clever word that can be read two ways.

How Check each reply against these before you send. Rewrite any sentence that fails.

Rule	In plain words
One word, one meaning	Pick one word per idea. Do not swap synonyms.
Active voice	"The script writes the file."
Simple tenses	Simple present or simple past. Avoid "has been configured".
Short sentences	Steps about 20 words or fewer. Descriptions about 25. One idea each: need a comma to add a new clause? Split.
Limits	One relative clause. No more than 3 nouns in a row.
Steps are commands	"Open the file." Reuse names: do not coin a new name for a named thing.
No em dash	Use a period, comma, or hyphen.

Exempt: quoted errors and logs, code and config, the human's text echoed back. The standard covers chat, drafts, PR text, ticket text, vault pages, and advisor notes.

Where in pack `harness/global/APPEND_SYSTEM.md`, `rules/global/no-emoji.md`, `vault/schema.md`.

> Rock rule: One idea, one sentence. Short word beats long word.

Translating codes for humans

What Replace every tracker code with its plain meaning before it reaches a person.

Why Reports once showed vote integers, merge-status codes, thread ids, and hashes. The author complained on three separate days. The fix came after the third.

How

Say these words. Never show the number behind them.

- PR status: Active, Abandoned, Merged.
- Merge status: conflicts, no conflicts (also: not set, queued, rejected by policy, merge failed).
- Thread status: Active, Resolved, Won't Fix, Closed, By Design, Pending. The API calls Resolved "fixed". Say "Resolved", the PR page word. Skip system threads.
- Vote: approved, approved with suggestions, waiting for author, rejected. Never "voted 10" or "-5".

1. Keep at most one raw anchor per item (one `file:line`, hash, or thread id), after its meaning. Put the rest in a doc in the task folder.
2. Define a coined label on first use: `<label> (<what it is>)`.
3. Translate every subagent or reviewer result before you show it. Relay is a source of jargon.

Where in pack `harness/project/AGENTS.md` (Vocabulary), `harness/global/APPEND_SYSTEM.md` (Words), `skills/my-pr-status/SKILL.md` (Step 4).

> Rock rule: If a human needs a decoder ring, translate.

The daily update

What A short note to your manager, usually in Teams, on what you did yesterday.

Why Manager reads many notes. Two sentences are enough. Outcomes matter. Process steps do not.

How Say "daily" or "manager update". The agent runs the daily flow in `wiki-comms`:

1. Pull the day's finished work from `vault://wiki/log.md`, plus the PR and ticket numbers in those entries.
2. Draft in the author's style: two sentences at most; open with "As for me yesterday:" (swap the day name); state outcomes, not steps ("linked to the ticket and moved to Resolved" becomes "tickets are up to date"); drop verification detail once work sits in an open PR; keep bug and PR numbers; drop state names, tool names, skill names.
3. Show the draft in a fenced block. The author sends it. No Teams send route exists.

Example (numbers illustrative):

```
As for me yesterday: opened PR 4567 for the upload-name bug (Bug 12345) and answered review
comments on PR 4568. Tickets are up to date.
```

Only when asked, a recap ("what did I do yesterday?") is one line each, no codes: PRs with owner, tasks touched, open decisions.

Where in pack `skills/wiki-comms/SKILL.md`, `vault/log.md`, `skills/morning-brief/SKILL.md`, `skills/daily-work-poll/SKILL.md`.

> Rock rule: Two sentences. Outcomes. Numbers kept, jargon dropped.

Scan findings: Checkmarx and security issues

What Two kinds of security finding: a static-scan report from a pipeline (Checkmarx One), and a gate finding on your own diff. Both end as Bugs or fixes, not as guesses.

Why The same scan template makes the same finding in every repo. The artifact download tool reports success and writes an empty file. A skipped scan stage does not mean a clean repo.

How Checkmarx report:

1. Get a task folder first. Never download into the tasks root. Use the ticket's folder, or the ticketless `RESEARCH-00000-checkmarx-<repo>`. Downloads go in `<task>/checkmarx/`.
2. Find the build that ran the scan. The scan is a pipeline stage set to continue on error, so it never fails the build. No report artifact means the stage was skipped, not that the repo is clean.
3. Read counts from the build log first: scan id, overall risk, the per-engine table, portal link. Container findings are base-image findings and triage differently.
4. Download the report. The MCP download is broken for this type. Retry once at most, then use the CLI. A real report is hundreds of KB. A 0-byte file means failure. Overwrite in place.
5. Triage each finding against the worktree with `file:line` evidence: Real, False positive (name the line that proves it), or Duplicate (name the ticket). Never classify from the title alone.
6. One Bug per real finding, not per scan. Base-image findings usually get one Bug for the image upgrade. Ask before you create any work item.
7. Record the scan as a vault note in `notes/`, named `<date>-<repo>-checkmarx-scan-report-build-<id>.md`.

Team rules around this: the release gate is zero critical and zero high findings. A false-positive class (for example typed queries flagged as injection) is documented as a waiver in a PR. Findings on your own diff follow the gate in "Shipping": fix introduced ones, file Bugs for pre-existing ones, rotate exposed secrets. Never put a secret value in a draft, comment, or file.

Filing a Bug from a finding. Model it on a recent real Bug from the same area. Fetch one with all fields and mirror its layout. Do not invent fields. Draft and show the human before you create. Title: what the user gets, in plain English, not the cause. Repro steps: numbered, what happens and what should happen. How QA checks it: the click path on dev or QA (a finding with no screen uses `file:line` and the check that proves the fix). Three things the tester can tick off. Screenshots attached. Area path per the area-path rule. Iteration, priority, severity from the model Bug. Assigned to a named person. After creating, re-fetch and confirm state, iteration path, and board column. Link it Related to the source item and re-fetch relations. From a PR thread? Post the new Bug number back in the thread.

Where in pack `skills/checkmarx-handling/SKILL.md`, `skills/pr-and-team-comms/SKILL.md`, `skills/pr-security-gate/SKILL.md`, `agents/pr-security.md`, `rules/project/task-files-in-ticket-folder.md`, `rules/project/azure-cli-fallback.md`, `rules/project/ado-area-path.md`.

> Ugh. Fall: A download "succeeded" with a 0-byte zip. The CLI login did not satisfy the extension's artifact tool. Fix: CLI fallback into the task folder. Check the file size.

> Rock rule: Triage by line, not by title. One Bug per real finding.

The Long Memory: Your Vault

Chat forgets. Files remember. The vault is a pile of small notes. The agent reads them before it speaks. The agent writes them after it learns. Like the story wall in the cave: paint a fact once, and every later hunt can see it.

Why a personal wiki

What. The vault is a folder of typed notes: Markdown files, each with a header block of fields. You open the folder in Obsidian (a note app that treats a folder of text files as a book). The agent reads and writes the same files.

Why. Chat is a campfire: warm, then ash. A chat dies at `/clear` or a restart, and its facts die too. The vault keeps them. It is also the truth for task state: the tracker mirrors the vault, never the reverse. It records rejected options, so nobody argues them twice.

How. Three layers, one law each.

Layer	Folder	Law
Raw sources	`raw/`	Originals. The agent reads them. Nobody edits them.
The wiki	typed folders	The agent owns it. It keeps links, index, and log fresh.
The schema	`schema.md`, `wiki-*` skills	The contract: types, fields, names, log format.

```
clip, transcript, chat --> raw/ (frozen)
                             | ingest
                             v
you, scribe, cron jobs --> typed pages tied by [[wikilinks]]
                             |-- schema.md says how
                             |-- index.md lists them
                             `-- log.md records each write (append only)
```

Where in pack. `vault/schema.md`, `vault/index.md`, `vault/log.md`, `vault/examples/` (fictional practice pages), `skills/wiki-spec/SPEC.md`.

The eight page types

What. Eight types, one folder each.

Type	Folder	Holds	Key fields
org	`orgs/`	a client, the employer, "Personal"	`ado_org`, `slack_channels`
project	`projects/`	one repo or product	`repo_path`, `ado_states`
task	`tasks/`	one unit of work	`status`, `priority`, `ado_id`, `pr`
meeting	`meetings/`	one meeting	`date`, `kind`, `attendees`

Type	Folder	Holds	Key fields
person	`people/`	one person	`org`, `role`, `slack_id`
decision	`decisions/`	one choice, with reasons	`date`, `status`, `supersedes`
note	`notes/`	reusable facts, filed answers, lint reports	`projects`, `topics`
source	`sources/`	one ingested thing	`origin`, `url`, `raw`

Why. Each folder is a table, each file a row, each header field a column. Obsidian Bases (`.base` files: saved tables over those fields) then give you a task board and a decision log for free.

How.

1. Names are kebab-case. Meetings, decisions, sources, and dated notes start `YYYY-MM-DD-`. People are `first-last.md`.
2. Links are wikilinks: `[[people/jane-doe]]`. In a header field, wrap the link in quotes.
3. A task `status` is inbox, todo, doing, review, done, or dropped.
4. A decision page has Decision, Context, Reasoning, Consequences. The deep form adds Mechanism, Files changed, Evidence, Known gaps, Alternatives rejected. Turn each known gap into a task or an owner. Gaps warn you of drift.
5. A replaced decision stays. Mark it `superseded`.

Where in pack. `vault/templates/`, `vault/views/` (six `.base` files), `vault/examples/decisions/` (a supersede chain).

The schema is the contract

What. One file, `vault/schema.md`, says what every page must look like.

Why. Many sessions write pages. Without a contract, pages wander. With one, lint can check them by machine.

How. It holds the tree, naming rules, a field table per type, the log format, and five hard laws:

1. `raw/` is immutable.
2. `log.md` is append-only.
3. Ask before a PR, before a tracker move to review or done, and before any Slack post. In the tasks root, project rules refine this: a skill step that says "open the PR" is the permission.
4. The vault is task truth. Tracker state never silently overwrites it.
5. All vault prose follows ASD-STE100 (plain English: short active sentences, one word per meaning).

Change in this order: schema, templates, views, skills.

Where in pack. `vault/schema.md`, `skills/wiki-spec/SKILL.md` (read it first), `skills/wiki-spec/SPEC.md` (nine locked decisions, gates, setup).

> Rock rule: change the schema first.

How agents reach the vault

What. Agents use `vault://` addresses, never file paths.

Why. The route gives the edit tool a snapshot tag. A raw path has no tag. A rule stops the shortcut and sends the agent to read the schema first.

How.

1. Find: `read vault://wiki/?op=search&q=<keywords>`.
2. Read, write, edit: `vault://wiki/<folder>/<page>.md`.
3. An edit header reads `[vault://wiki/<path>#TAG]`. Copy the tag from a read of the same page. A shortened path fails.
4. Ask structure with `?op=outline`, `?op=backlinks`, `?op=links`, `?op=tags`, `?op=tasks`. Ask vault-wide with `?op=orphans`, `?op=unresolved`.
5. Never `grep` the wiki folder. Read the schema before any write.
6. Write in three cases only: you name a wiki skill, you ask for a write, or you name a page. A statement of fact is not a write request.
7. The main agent writes directly. It hands off only a big, multi-page write.

Where in pack. `rules/global/vault-writes-via-vault-uri.md`, `harness/global/RULES.md`, `harness/global/APPEND_SYSTEM.md` (section `# Wiki`).

> Rock rule: address by `vault://`, never by path.

Ingest: put a source on the wall

What. `wiki-ingest` files one new source: a clipping, document, link, or pasted text.

Why. Raw stays pure. A summary and links make it findable.

How.

1. Pasted text goes to `raw/YYYY-MM-DD-<slug>.md`. Never edit a raw file.
2. Read it all. Make `sources/YYYY-MM-DD-<slug>.md` from the template: origin, raw link, projects, summary.
3. Add one line and a wikilink on each related project, org, or person page.
4. Add the index line. Log op `ingest`.

Where in pack. `skills/wiki-ingest/SKILL.md`, `vault/templates/source.md`, `vault/examples/sources/`.

Meeting notes

What. `wiki-meeting` turns a transcript into a meeting page, people pages, and tasks.

Why. Meetings hide decisions and chores. A page makes them searchable.

How.

1. Clean first: a caption file (`.vtt`, `.srt`) loses cue numbers, timestamps, and repeats. No speaker labels? Ask for attendees.
2. Save it in `raw/`. Pull out date, kind, project, org, attendees. Unsure? Ask. Never guess a project or org link.
3. Match attendees to `people/first-last.md`. Transcripts misspell names, so a close match needs your yes. A bracket tag in a Teams export means guest or host, not employer.
4. Name who raised each point. Each action item: same issue open in `tasks/`? Add a Notes bullet there. Else make a task with `status: inbox`, `priority: p3`.
5. Link the meeting from the project page. Log op `meeting`.

Where in pack. `skills/wiki-meeting/SKILL.md`, `vault/examples/meetings/2026-03-04-platform-standup.md`.

Sync with the tracker

What. `wiki-sync` keeps vault tasks and tracker items in step. The vault wins.

Why. Two truths rot. One boss keeps order. Drift gets flagged, never silently fixed.

How.

1. Load the project's `ado_states` map (vault status to tracker state). Missing? Read the real state names and write them down.
2. Push the vault to the tracker. A push to review or done waits for your yes. Never change vault `status` from the comparison. Copy `sprint` and `due` from the tracker.
3. Close `review` tasks whose PR merged, after you approve.
4. Pull new assigned items as `inbox` pages. Never promote without your answer.
5. Flag drift in a `## Drift` section. Leave header fields alone. Log op `sync`.

Where in pack. `skills/wiki-sync/SKILL.md`, `vault/examples/projects/meter-api.md` (see `ado_states`).

The task loop

What. `wiki-task-loop` works one vault task from `todo` to `review`.

Why. Some work has no ticket. A task page is the ticket. The loop gives it the same shape.

How.

1. Pick the named task, or the top `todo` (p1 first, oldest first). Set `status: doing`.
2. Make sure a tracker item exists. Find the repo path on the project page (ask once, save it).
3. Branch `task/<slug>`. Build and test with the repo's own commands. File a decision page for each design or scope choice.
4. Gate: show the diff. In the plain loop one approval covers the PR, the tracker review move, and `status: review`. Log op `task`.
5. Stuck? Add a `## Blocker` section, stay `doing`, report.

Correction for the ticket pipeline: inside the tasks root, project rules win. The PR opens with no prompt once the verification gate passes. Tracker moves follow the exception list in `harness/project/RULES.md`.

Where in pack. `skills/wiki-task-loop/SKILL.md`, `harness/project/RULES.md`.

Lint, log, query, comms

What. Four habits keep the wall clean.

Why. An unchecked wiki rots in silence.

How.

1. Lint weekly with `wiki-lint`: orphan pages, unresolved links, schema drift, stale `doing` tasks (older than five working days), contradictions (two decisions, one scope, neither superseded), pages missing from the index. It writes `notes/YYYY-MM-DD-lint-report.md`.
2. Log every write: `## [YYYY-MM-DD] <op> | <title>`, then one to three bullets. Ops: ingest, meeting, task, sync, lint, query, comms, setup. Read the last lines first, then append. Never insert at an old line number. Never edit a past entry.
3. `wiki-query`: read `index.md` first, search, read candidates in full. Answer only from pages, with a wikilink per claim. File a durable answer as a note.
4. `wiki-comms`: read Slack, resolve people to pages, draft, wait for approval, send the approved text unedited. A manager update is two sentences and has no send step.

Where in pack. `skills/wiki-lint/SKILL.md`, `skills/wiki-query/SKILL.md`, `skills/wiki-comms/SKILL.md`, `vault/log.md`, `vault/examples/notes/2026-03-13-lint-report.md`.

Who writes back: you, the scribe, two cron jobs

What. Three writers help you keep the wall painted.

Why. Facts used to vanish between sessions. Narrow writers fix that. A tight allowlist keeps them off task state, the schema, and every outside system.

How.

1. The scribe is the `WikiScribe` advisor. Once per finished run it files only facts the transcript already shows. It never goes hunting. It writes `notes/`, `decisions/`, an append to `log.md`, and the Links and Notes bullets of an existing project page. Never `raw/`, task state, `schema.md`, `index.md`, the tracker, PRs, or Slack. It extends a page that exists. It never makes a twin.
2. A config change counts as a decision, even two lines (a roster, a rule, a gate, a state order, a model pin). A finished research task counts too.
3. The reminder to write back: after a write, the scribe sends one note at nit severity, one line per page. Nothing to file means silence. The `advisor-drain` extension holds the turn open until the scribe finishes. After fall V2 the author raised the "did it file the fact?" check to blocker severity, because a concern-level card was silent and a missing write-back went unseen for days. `harness/global/WATCHDOG.yml` does not show that setting. Add it if you want the check that loud.

4. State a fact one turn before you close the session. The exit gate runs before the scribe reviews. The `end_session` tool drains advisors first. A typed `/exit` does not.
5. Two cron jobs fill the vault before you wake. `board-ingest` writes the board state and the morning brief. `slack-ingest` files one day of team chat. Both run headless, with an overlay that denies tracker and Slack writes, and a guard line in `log.md` that makes a rerun safe.

Where in pack. `harness/global/WATCHDOG.yml`, `harness/global/WATCHDOG.md`, `harness/extensions/global/advisor-drain.ts`, `automation/README.md`, `skills/morning-brief/SKILL.md`.

Set it up in six steps

What. Build your own vault in one afternoon.

Why. An empty vault with a schema beats a full vault with none.

How.

1. Copy `vault/schema.md`, `vault/index.md`, `vault/log.md`, `vault/templates/`, and `vault/views/` into your own folder (the pack uses `<VAULT_ROOT>`). Keep the fictional `vault/examples/` outside it. The example `vault/examples/notes/service-registry.md` shows a port column. Your real registry holds no ports (fall D18).
2. Open the folder in Obsidian as a vault, so `vault://wiki/` resolves. Keep `vault: enabled: true` in `harness/global/config.yml`.
3. Copy the `skills/wiki-*` folders into your global skills folder. Add the `# Wiki` section of `harness/global/APPEND_SYSTEM.md` to your prompt additions. Copy the vault rule, the vault laws in `harness/global/RULES.md`, the `WikiScribe` entry of `harness/global/WATCHDOG.yml`, and the two advisor extensions.
4. Check it. Read `vault://wiki/schema.md` (the route works). Read `skill://wiki-ingest` (skills load). Read one item from each MCP server.
5. Make org pages (you, your employer, your client, Personal) and a project page per repo, on demand. Run `wiki-sync` once to seed tasks from the board.
6. Run `wiki-lint` weekly. Restart after any skill or extension change.

Where in pack. `skills/wiki-spec/SPEC.md` (section Setup), `vault/`, `harness/global/`.

Glory: What Worked

Thirty things worked. Each is a stone that held. Copy them. The tags W1 to W30 are stable, so you can point at them.

Start clean

What. Four habits that start every ticket the same way. **Why.** A messy start taxes every later step.

W1. One command per entry point. Eleven slash commands (short prompts starting with `/`), each under ten lines. Each names a skill and passes the argument. *Why it works:* no retyped openers (15% of human turns were saved commands). It cannot drift, because it never repeats steps. *Where:* `commands/`, `skills/workflow-router/SKILL.md`.

W2. One folder per ticket, one script. Folder `{TYPE}-{NUM}-{summary}` holds repo symlinks. Only `scripts/setup-task-branch.sh` makes it. It is safe to rerun, refuses a second folder for one ticket, skips standing exclusions, and has a `--pr` mode. *Why it works:* parallel tickets, no branch juggling, no hand-made worktrees. *Where:* `scripts/setup-task-branch.sh`, `skills/ticket-bootstrap/SKILL.md`.

W3. Start at the root, move on purpose. Sessions start at the tasks root. A `move_session` tool moves into a task folder, or back. *Why it works:* MCP servers, rules, and skills load from the launch folder only. A fixed rule (root or direct child, never from a subagent) ends per-move prompts. The return to root after `/clear` is designed, not yet seen live. *Where:* `harness/extensions/global/move-session.ts`.

W4. One file of standing facts. `harness/project/AGENTS.md` holds org and project names, repo scope, state ladders, a vocabulary table for tracker codes, environments, folder habits. *Why it works:* about half of sessions match no skill, so a fact that lives only in a skill never loads. One copy ended repeated corrections and let 11 to 13 skills shrink.

> Rock rule: same start, every ticket.

Prove the fix

What. Six habits that turn "I think it works" into evidence. **Why.** Agent says "done" fast. Proof says "done" slow. Slow is cheaper than a returned bug.

W5. Reproduction is the contract. Bootstrap, reproduce, root cause with `file:line`, fix plan per repo, failing test first, smallest fix, mutation check, live verify, one commit per repo, scripted self-review, then open the PR without asking. *Why it works:* each step leaves evidence a reviewer can re-run. Gates earn autonomy. The mutation check (remove the fix: the test must fail; restore it: it must pass) proves the test guards the fix. *Where:* `skills/bug-workflow/SKILL.md`, `scripts/self-review.sh`, `rules/global/BUG_FIXING.md`.

W6. A table: where does the test go? A lookup from "what broke" to "where the test lives": function, API field, assistant behaviour, produced file, screen, journey. *Why it works:* "add a regression test" stops being a judgement call. *Where:* `skills/regression-test-placement/SKILL.md`.

W8. Design per repo, cross-check, walk the deploy order. A short design doc per repo with `file:line` proof, a second pass that checks both sides fit, then every ship order walked with what a user sees. *Why it works:* a design can be right once both sides ship and broken in between. Stale citations hide in good-faith documents. *Where:* `rules/global/BUG_FIXING.md` (multi-repo fixes; trust but verify).

W9. Live verification against the real dependency. A pass through real services, storage, and model gateway is part of "done". *Why it works:* a mock holds your guess. For varying answers, run a baseline on `main` against the fix branch: many runs, memory reset per run, control prompts, and a stop if the baseline shows no bug. *Where:* `skills/e2e-verification/SKILL.md`.

W10. Flow file plus flow-runner. One `flow.yaml` per UI bug, run three times (broken on the pre-fix build, fixed on the fix branch, fixed on QA) by a cheap vision model. *Why it works:* a flow that never says "broken" on the broken build proves nothing, so the dispatcher proves the served build first. *Where:* `skills/flow-schema/SKILL.md`, `agents/flow-runner.md`.

W30. Scripts tested in a sandbox, in the real shell. Prove scripts on synthetic bare repos under macOS `/bin/bash` 3.2, with fail-closed and rerun cases. *Why it works:* some bugs show only under cron's shell or on a second run. *Where:* `scripts/`.

> Rock rule: no proof, no done.

Tickets and plans as contracts

What. Four habits that treat the ticket as the spec. **Why.** A thin ticket makes a fast wrong answer.

W7. Two sessions for a story. Session A (refinement, or `/triage`) grills the human and writes an Agent Contract. Session B starts blank, trusts only the ticket, checks the contract, builds, opens the PR. *Why it works:* it tests the ticket as a handoff. A thin contract fails the gate: a comment names the gaps, the state returns to refinement, the original text stays. *Where:* `skills/story-refinement/SKILL.md`, `skills/story-workflow/SKILL.md`, `skills/ticket-triage/SKILL.md`.

W24. Plans as files with numbered decisions. Seven headings: Context, Decisions, Verified facts, Approach, Critical anchors, Verification, Assumptions. *Why it works:* a fresh session can build and verify without the chat. The implementer makes no design choices. *Where:* no template ships. Copy the headings into your plan notes.

W29. Say what "actionable" means. Four rules decide when a board bug has work left. "Picked up" means a folder, branch, PR, or live session, never mere assignment. *Why it works:* the brief stops guessing. *Where:* `skills/morning-brief/SKILL.md`.

W13. Facts decide state moves. The agent writes `Resolved` or `Ready for Test` once every linked PR merges, and `Ready for Retest` or `In Test` once a QA deploy is proven. It never writes `Closed`, `Pending PO Review`, or `Accepted`. *Why it works:* each move follows a checkable fact, has one legal next value, and can be undone. One rung per write, then re-fetch. *Where:* `harness/project/RULES.md`, `skills/pr-merge/SKILL.md`, `skills/tasks-deployed-to-qa/SKILL.md`.

> Rock rule: the ticket is the contract. Facts move the state.

Fences that hold

What. Six fences, mostly in code, not prose. **Why.** Prose gets missed under autonomy. Code does not blink.

W11. Approval gates in config. Prompt and deny policies on write tools, plus one global gate that fails closed, with project overrides. An optional auditor model lets only a clear SAFE verdict skip the dialog. *Why it works:* a fail-closed list survives missing config. *Where:* `harness/extensions/global/tool-approval-gate.ts`, `harness/global/tool-call-auditor.md`, `harness/project/config.yml`.

W12. Guard extensions in code. Hooks block: stopping a process this session did not start, touching anything outside the tasks folder, starting a service from a bare repo, leaking secrets. *Why it works:* a `tool_call` hook checks the final arguments once, before the tool runs. *Where:* `harness/extensions/global/guardrails.ts`, `harness/extensions/project/no-parent-folder-guard.ts`, `harness/extensions/project/process-ownership-guard.ts`.

W15. Security gate before every PR. A fail-closed secret scan before any push, plus a `pr-security` subagent review of your own diff. Introduced finding: fix now. Pre-existing: file a Bug, go on. At most two fix rounds. Too big: draft PR. *Why it works:* explicit rules end the loop. *Where:* `skills/pr-security-gate/SKILL.md`, `agents/pr-security.md`, `scripts/secret-scan.sh`.

W27. Secrets by key name. The Playwright MCP types real credentials when the agent passes a key name, and redacts the value. TOTP codes come from a tiny CLI. The secrets file is hard-blocked for reads. *Why it works:* secret values never enter the model's context. *Where:* `skills/e2e-verification/SKILL.md`, `automation/playwright/totp.mjs`.

W28. One owner per port. Whatever starts a service owns its port, checks conflicts, and reports the real port. Docs point at the owner. *Why it works:* a copied port is wrong once a default changes. *Where:* `skills/service-startup-reference/SKILL.md`, `harness/extensions/project/services/index.ts`.

W25. Safe cleanup tooling. A script that reports first, defaults to dry run, refuses dirty repos, removes the worktree's Docker leftovers, and acts only on items the human names. *Why it works:* cleanup destroys, so a report plus a named list stops bulk deletes. *Where:* `scripts/cleanup-task-folders.sh`, `skills/task-cleanup/SKILL.md`.

> Rock rule: if it must hold, put it in code.

Review and merge

What. Three habits for the last mile. **Why.** A merge is hard to undo.

W14. PR hygiene as a checklist. Conventional-commit title, two description templates under 200 words, an evidence section, a verified work-item link, a required reviewer before auto-complete, a table of forbidden wording. *Why it works:* the description carries only what a reviewer cannot infer: no "Fixes N", no suite-pass count. *Where:* `skills/pr-creation/SKILL.md`, `skills/pr-and-team-comms/SKILL.md`.

W16. Review lanes, with claim-checking. Read-only security, quality, and architecture subagents. The main agent checks each claim against code before repeating it. *Why it works:* lanes over-claim. The human reads one verified two-part report and posts drafts one at a time. *Where:* `skills/pr-review/SKILL.md`, `agents/architect-reviewer.md`, `agents/error-detective.md`.

W17. Merge gates. `pr-merge` checks pipeline freshness, paired-PR order, and branch policies, sorts open threads, asks once, and never merges an unpicked PR. *Why it works:* one combined ask keeps friction low. Only a thread's author resolves it. A vote on a colleague's PR is not merge authority. *Where:* `skills/pr-merge/SKILL.md`, `skills/pr-comments/SKILL.md`, `rules/project/pr-thread-resolution.md`.

> Rock rule: ask once, with facts on the table.

Honest, short, plain

What. Four habits about how the agent talks and works. **Why.** The author reads many replies a day. Clear beats clever.

W20. Honesty rules in the system prompt. "Say I do not know." Check a tool before stating a fact. Mark only unverified claims as `[INFERENCE]`. Ask one direct question on real ambiguity. *Why it works:* the author never objected to "I did not test X". *Where:* `harness/global/APPEND_SYSTEM.md`.

W21. Chat rules derived from evidence. Answer first. Ten lines or fewer. Tracker codes become words. Coined labels get defined. Drafts match the dictation. *Why it works:* the rules came from 28 classified confusion cases. Confusion came from identifiers and undefined labels, not length. *Where:* `harness/global/APPEND_SYSTEM.md`, `harness/project/AGENTS.md`.

W22. The read-tool-first rule. An always-on rule (advisors get it too) sends known-path reads to `read`, with a tested table of tool traps for symlinked repos. *Why it works:* agents fall back on `ls` and `cat` at session start. An A/B run showed 2 shell `ls` calls before and 0 after. *Where:* `rules/global/read-tool-first.md`, `rules/project/tool-gotchas.md`.

W23. Delegation discipline. Self-contained worker briefs. One worker per repo, absolute paths. The orchestrator holds every gated write and re-reads every real diff. *Why it works:* a worker's "done" is a claim. Port cross-wiring makes parallel live runs unsafe, so live checks run one at a time. *Where:* `rules/global/BUG_FIXING.md`, `skills/story-workflow/SKILL.md`.

> Rock rule: say what you know. Say what you did not check.

Memory and automation

What. Three habits that work while you sleep. **Why.** Memory outside the chat survives the chat.

W18. Unattended crons, two barriers. `board-ingest` and `slack-ingest` run `omp -p` with an overlay that denies tracker and chat write tools, plus a prompt allowlist. *Why it works:* the unattended path cannot write even if the prompt is wrong. Guard lines make reruns safe, and missing days backfill oldest first. *Where:* `automation/board-ingest/`, `automation/slack-ingest/`.

W19. The vault as durable memory. Schema, index, append-only log, immutable `raw/`, and decision pages with "Known gaps". *Why it works:* knowledge survives `/clear` and restarts, and rejected options are not re-argued. *Where:* `vault/`, `skills/wiki-spec/SPEC.md`.

W26. Papercuts log plus rule timeline. One line per time sink (date, symptom, fix, project), plus an analysis that sorts each failure: rule missing, rule broken, or rule followed but rejected. *Why it works:* it shows which layer to change. Adding a rule did not remove the friction. Changing the rule did. *Where:* `harness/project/AGENTS.md` (the order to keep it).

> Rock rule: write it where the next session can read it.

The Fall: What Hurt

Sixty falls. The author fell so you can walk. Each story gives what happened, why, the fix, and the law it made. The tags (A1, H2, and so on) are stable, so you can point at them. "Pack" names the file that holds the fix. "None" means the lesson lives in habit, not in a file.

Agent behaviour (A1 to A7)

Agent clever. Agent also wanders. Seven stories.

A1. Ask, not look. Agent said "want me to check?" about a thing a tool could answer. No rule said fetch first. Fix: look in code, config, docs, tracker; ask only when tools cannot. *Rock rule: fetch, then speak.* Pack: `harness/global/APPEND_SYSTEM.md`.

A2. Mock, not real. Checks ran against a stubbed API. The author demanded the real app. A live-check rule existed, but nothing gated it. Fix: a live pass is part of done; an advisor flags reports with no evidence. *Rock rule: no live pass, not done.* Pack: `rules/global/BUG_FIXING.md`.

A3. "Done, tests pass." A worker said done; the orchestrator forwarded it. A report is a claim, and workers share no context. Fix: read the real diff and test output first. *Rock rule: a report is a claim.* Pack: `rules/global/BUG_FIXING.md`.

A4. Guess and hide. "Strengthen the rule" was settled by guessing, then announcing the guess. Another agent rebuilt an artefact the author had rejected. The agent picked a reading and moved on. Fix: two readings, one direct question; drop a thread once decided. *Rock rule: two readings, one question.* Pack: `harness/global/APPEND_SYSTEM.md` (first fix only).

A5. Notes the human never saw. A reply began by correcting an advisor note the author never saw, and reviewer jargon arrived raw. No rule covered unseen sidecar output. Fix: never mention what the human did not see; translate subagent results into plain words. *Rock rule: show only what the human saw.* Pack: `harness/global/APPEND_SYSTEM.md`.

A6. Re-checking a clean verdict. A clean advisor verdict set off another round of re-testing. Approval felt like a nudge. Fix: a clean verdict is final; re-verify only for a new concern or changed code. *Rock rule: clean means stop.* Pack: `rules/global/no-redundant-reverify.md`.

A7. Plan mode forgot the laws. In plan mode the agent proposed touching a secret despite a ban. House rules reach plan mode only by hook. Fix: a plan-only hook injects two rules: no secret in a plan; every state-changing step names its verify and undo. *Rock rule: plans obey the laws.* Pack: not shipped. Rebuild it from these two rules.

Harness and tooling (H1 to H11)

The harness is a strange beast. Eleven bites.

> **H1. Five tripwire gates.** Five stream-rule gates (write, PR, Slack, comment, implement-first) stalled autonomous runs. Stream rules abort output on a match. Fix: remove all five; use prose, config prompts, and an advisor; put hard laws in hooks. *Rock rule: a tripwire is not a judge.* Pack: `harness/project/WATCHDOG.yml`.

> **H2. The path guard that cried wolf.** A path rule interrupted sessions 37 times in four days. It re-tested half-streamed text, so a chunk ending "tas" matched. Fix: a `tool_call` hook on final arguments. *Rock rule: judge the final call.* Pack: `harness/extensions/project/no-parent-folder-guard.ts`.

> **H3. Rules that never fired, or fired wrong.** A length rule almost never fired; ticket rules fired on a `grep`. `^` needs `(?m)`, `.` needs `(?s)`, and a bare scope hits every tool. Fix: scope `tool:<exact name>`, add flags, test on logged calls. *Rock rule: test rules on real logs.* Pack: `rules/project/ado-area-path.md`.

> **H4. Changes with no effect.** An extension, setting, or skill made mid-session stayed invisible; one extension died on "Cannot find package". Everything loads at start. Fix: restart, test in a fresh `omp -p` run, import only types. *Rock rule: restart, then test fresh.* Pack: `harness/extensions/global/lib/advisor-config-compat.ts`.

> **H5. Denied, with no reason.** A browser code tool was blocked with no prompt, and a headless worker could not approve it. After another denial with no recorded reason, an agent tried a browser sign-in and a PAT `curl` instead. Two layers denied it, and no deny gave a reason. Fix: one prompt owner, a reason on every deny, and when denied, ask or stop. *Rock rule: denied means ask, never route around.* Pack: `harness/extensions/global/tool-approval-gate.ts`.

> **H6. Answers said three times.** Final answers repeated up to three times after a hidden "attempt N/3" nudge: the stop detector took a finished answer for a stall. Fix: turn it off. *Rock rule: a nagging helper needs an off switch.* Pack: `harness/global/config.yml`.

> **H7. Prose said ask, config said allow.** "Ask before modifying a work item" prompted nothing: the gate fell back to `allow`. Fix: write down who enforces what; ask in chat before a state, title, or description write. *Rock rule: prose and config are one document.* Pack: `harness/project/RULES.md`.

> **H8. The 300-second wall.** A batch cleanup died at the shell cap of 300 s after 7 of 14 folders; four relaunches hit it again. `async` does not lift the cap. Fix: batches of three or four, an explicit `timeout`. *Rock rule: know the cap, size the batch.* Pack: `skills/task-cleanup/SKILL.md`.

H9. Fine in my shell, dead in cron. Scripts crashed only in cron: "unbound variable". macOS `/bin/bash` 3.2 treats an empty array as unbound under `set -u`. Fix: guard empty arrays, test under `/bin/bash`. *Rock rule: test in the shell that will run it.* Pack: `scripts/secret-scan.sh`.

H10. One browser profile, four processes. Playwright failed with "Browser is already in use", then `@latest` moved the Chromium revision. Four MCP processes shared one profile lock, and the version was unpinned. Fix: `--isolated` per session, pin the version. *Rock rule: pin it, isolate it.* Pack: `harness/mcp.template.json` (still `@latest`: pin yours), `agents/flow-runner.md`.

H11. Search blind to symlinks. `grep` said "No matches found" for text that exists. `read` follows task-folder symlinks, `glob` works from the link down, `grep` fails on the link itself. Fix: a per-tool table, `git -C <repo> grep`, an always-on `read` rule. *Rock rule: test each tool on a symlink.* Pack: `rules/project/tool-gotchas.md`, `rules/global/read-tool-first.md`.

Git and worktrees (G1 to G5)

Git is honest. Git is also picky. Five stories.

G1. Phantom remotes. `--force-with-lease` failed with "stale info", and `git push -u` exited 128 though it worked. Bare-clone worktrees have no fetch refspec. Fix: explicit refspecs; confirm a push with `git ls-remote`. *Rock rule: ask the remote, not the exit code.* Pack: `scripts/self-review.sh`.

G2. Stash during a merge. After `git stash` mid-conflict, `stash pop` dropped `MERGE_HEAD`. Stash and an open merge fight. Fix: never stash or rebase during a merge; merge the target into the branch; never force-push a shared branch. *Rock rule: merge, never stash, never rebase.* Pack: `skills/merge-conflict/SKILL.md`.

G3. The tidy-up that deleted rigs. An agent deleted the branches, worktrees, and symlinks of out-of-scope repos, and the author needed them for testing. The script had no exclusion idea. Fix: an `excluded_repos` array; after setup remove nothing. *Rock rule: after setup, remove nothing.* Pack: `scripts/setup-task-branch.sh`, `harness/project/RULES.md`.

G4. Reviewing yesterday's code. A PR review in a reused worktree showed the old tip. Reuse checked out the cached branch, not the fresh fetch. Fix: in PR mode force the local ref to the fetched head and compare with the PR head. *Rock rule: prove the head before you review.* Pack: `scripts/setup-task-branch.sh` (`--pr`).

G5. The `..` that escaped. A subagent wrote `../plan/...` from a repo symlink and made a folder outside the tasks folder. The shell resolved the link before the `..`. Fix: absolute task-folder paths for subagents. *Rock rule: absolute paths for helpers.* Pack: `harness/extensions/project/no-parent-folder-guard.ts`.

Local services (S1 to S7)

Services are noisy neighbours. Seven stories.

> **S1. Green and wrong.** Health was green, but the stack served another branch's code, so the first flow run was void. Hard-coded localhost ports cross-wire stacks. Fix: prove the served build (`lsof` the PID's cwd, `curl` one source module); run live checks one ticket at a time. *Rock rule: health is not identity.* Pack: `skills/service-startup-reference/SKILL.md` .

> **S2. Ready, but not ready.** `hub start` timed out on a dev server that was serving pages: it binds the IPv6 loopback, and the probe checked IPv4. Fix: readiness by log regex; `curl localhost` as proof. *Rock rule: ready means the log says so.* Pack: `harness/extensions/project/services/index.ts` .

> **S3. Debugger could not attach.** The readiness probe's connect-and-close used up debugpy's single client slot, and `--reload` forks a child the debugger cannot see. Fix: no ready gate in debug mode, a passive `lsof` check, no `--reload` . *Rock rule: never poke the debug port.* Pack: `harness/project/dap.json` .

> **S4. The kill that hit a neighbour.** A service agent killed a process another task owned. "Never stop a foreign process" was prose only. Fix: an ownership ledger; block `pkill` , `killall` , `xargs kill` , and any `kill` without a literal PID. *Rock rule: whoever starts it owns it.* Pack: `harness/extensions/project/process-ownership-guard.ts` .

> **S5. Services that did not talk.** An unprefixed sibling-URL override is accepted and ignored; the database container was down. Fix: exact prefixed names on the command line; read the startup log's printed URL. *Rock rule: the printed URL is the proof.* Pack: `harness/extensions/project/services/index.ts` .

> **S6. Dead key, silent probes.** Every image call returned 401; a PAT check returned 302 for real and bogus tokens. A probe with no control cannot tell good from bad. Fix: every probe carries a control; escalate dead keys to the owner. *Rock rule: a probe needs a control.* Pack: none (papercuts log).

> **S7. The orphans.** A Playwright browser and dev servers kept running after the PR. Nothing tore them down. Fix: close tabs, stop only what this session started, run an orphan check. *Rock rule: clean up what you started.* Pack: `rules/project/no-orphan-processes-after-use.md` .

Tracker (T1 to T6)

The tracker is a polite liar. Six stories.

T1. "Success" that lied. A write "succeeded" but angle-bracket tags became empty backticks, and an API-added reviewer was optional. A response reports the call, not the state. Fix: re-fetch and check the field after every write. *Rock rule: the response is not the state.* Pack: `harness/project/RULES.md`.

T2. Unattended reads that asked a person. Cron reads failed with "Client does not support form elicitation": with no `project`, the tool asks, and nobody is there. Fix: always pass `project`. *Rock rule: name the project, every call.* Pack: `rules/project/tool-gotchas.md`.

T3. State jumps that bounce. A state write got HTTP 400 though the matrix allows all-to-all. Jumps fail in practice. Fix: one neighbouring state per write, the ladder from `get_type`, then re-fetch. *Rock rule: one rung at a time.* Pack: `skills/ticket-bootstrap/SKILL.md`.

T4. The returned bug treated as new. A bug the tester moved back was handled as new; bootstrap would have overwritten `Bug Test Failed`. No branch existed for returned tickets. Fix: a Step 0 for returned bugs: read history and comments, reproduce on current code, say why the first fix failed, skip the state write. *Rock rule: a returned bug keeps its state.* Pack: `skills/bug-workflow/SKILL.md`.

T5. Who touched this? "Changed by me" returns only the last editor, and the board is an area path, not an iteration path. Fix: attribute through revision fields (changed, resolved, closed by); query the area path. *Rock rule: ask the field that holds the truth.* Pack: `skills/morning-brief/SKILL.md`.

T6. The zero-byte report. An artefact download "succeeded" with a 0-byte zip. The MCP download is broken for this report type. Fix: CLI fallback into the task folder, never the tasks root. *Rock rule: check the file size, not the success message.* Pack: `skills/checkmarx-handling/SKILL.md`.

PR and review (P1 to P8)

The PR is where trust meets code. Eight stories.

P1. Merged in fifteen seconds. A PR auto-merged unreviewed in about 15 seconds; `main` was reverted by hand. The API adds only optional reviewers, and auto-complete was set right after. Fix: make the reviewer required (CLI), re-fetch, then arm auto-complete. *Rock rule: policy is the gate; prose only points at it.* Pack: `skills/pr-creation/SKILL.md`.

P2. A vote that merged. An `Approved` vote on a colleague's PR merged it: their auto-complete was armed and the vote met the last policy. Fix: check `autoCompleteSetBy` and the live policy first; if the vote would complete the PR, ask. *Rock rule: a vote can be a merge.* Pack: `harness/project/RULES.md`.

P3. Closing a stranger's thread. The agent marked another reviewer's thread `Fixed`, assuming it could resolve what it fixed. Fix: push, reply with the commit, leave it `Active`. *Rock rule: only the starter closes.* Pack: `rules/project/pr-thread-resolution.md`.

P4. Comments that missed. A thread anchored outside the diff never rendered, and drafts carried unwanted patches. Anchors must sit on diff lines. Fix: name the problem; anchor on a changed file; offer a small fix only where no design choice is needed. *Rock rule: name the problem, anchor on the diff.* Pack: `skills/pr-review/COMMENT-STYLE-NOTES.md`.

P5. Lanes that over-claimed. A lane called a test assertion vacuous; another said a change reached templates it does not. Lanes read partial context. Fix: the main agent reads the diff first and checks each claim against code. *Rock rule: check a claim before you repeat it.* Pack: `skills/pr-review/SKILL.md`.

P6. Green build, stuck PR. `main` moved, so the policy build expired, and a plain pipeline run does not attach to the policy. Fix: class the build against `main`'s tip; re-queue through the policy action. *Rock rule: re-queue the policy, not the pipeline.* Pack: `skills/pr-merge/SKILL.md`.

P7. Two fixes, one symptom. Merging `main` into a branch silently retired part of an already-shipped fix. Two designs touched the same files, and the merge kept one. Fix: state the trade-off in the PR and get a human yes; prove the net change with patch diffs. *Rock rule: prove nothing was lost.* Pack: `skills/merge-conflict/SKILL.md`.

P8. The allowlist that did nothing. A global allowlist is ignored when default rules are extended, with no warning. Fix: a rule-scoped allowlist and a smoke test (one case must fire, one must pass). *Rock rule: test a scanner with a known hit.* Pack: `scripts/secret-scan.sh`.

QA and deploy (Q1 to Q5)

Tests are not the world. Five stories.

Q1. Green tests, dead fix. The mock assumed a library rejects on abort; the real one resolves, so code checking only in `catch` never ran. Fix: verify once against the real dependency, at the real failure boundary. *Rock rule: tests prove your model, not the world.* Pack: `rules/global/BUG_FIXING.md`.

Q2. A flash too fast to see. A 1.5 s highlight could not be captured. Screenshot timing missed the short tint. Fix: observe the resting layout; test transient states with a throwaway spec on the mocked API. *Rock rule: flows watch resting states.* Pack: `skills/flow-schema/SKILL.md`.

Q3. One lucky run. A model-behaviour fix looked proven after one run, but behaviour varies. Fix: the baseline on `main` must show the bug (1 of 15 runs, else stop); repeat runs; add control prompts. *Rock rule: many runs, with controls.* Pack: `skills/regression-test-placement/SKILL.md`.

Q4. A contract that lied. A contract said no nightly test pack exists, but one did. Nobody had checked the contract's facts in code. Fix: verify contract claims with `git grep` on `origin/main` of every repo; record a weaker check as a waiver. *Rock rule: check a contract's facts in code.* Pack: `skills/story-refinement/SKILL.md`.

Q5. Locked doors on QA and dev. A test login failed (wrong tenant), MFA prompts varied, and dev was unreachable from the laptop. Roles, tenants, and a private endpoint gate each door. Fix: ask for the role first; use a same-tenant test account with TOTP; record "dev: not reachable from this host". *Rock rule: name what you could not reach.* Pack: `skills/e2e-verification/SKILL.md`.

Comms (C1 to C5)

Words are tools. Five stories.

C1. Numbers in reports. Reports showed vote integers, merge-status codes, thread ids, and hashes; the author complained on three days. No rule translated tracker codes. Fix: a vocabulary table in the context file, UI words, one raw anchor at most. *Rock rule: words, not codes.* Pack: `harness/project/AGENTS.md`.

C2. Dense, with undefined labels. Status dumps used labels nobody defined ("step 3e"). Length was not the cause. Fix: verdict first, labelled sections, one decision at the end. *Rock rule: define a label where you coin it.* Pack: `skills/morning-brief/SKILL.md`.

C3. Drafts with extras. Drafts carried background, emoji, and sign-offs; PR text repeated "Fixes N" and "suite passing". "Never shorten a draft" was read as "add". Fix: drafts say exactly what was dictated; a forbidden-wording table. *Rock rule: write what was dictated, add nothing.* Pack: `skills/pr-creation/SKILL.md`, `rules/project/ado-ticket-voice.md`.

C4. The Slack post that collapsed. A post collapsed into one line and links did not render. Markdown content is not rendered, and links need rich-text elements. Fix: post a `blocks` rich-text payload; test the format in a DM first. *Rock rule: test the format in a DM.* Pack: `skills/pr-and-team-comms/SKILL.md`.

C5. Prompts that always said yes. The author kept saying "do not prompt me when everything is in order". The gates were followed as written, and always got "yes". Fix: turn always-yes prompts into fact checks; skill-directed moves need no prompt; keep approval for posts with an audience. *Rock rule: if the answer is always yes, check facts.* Pack: `harness/project/RULES.md`.

Vault (V1 to V6)

The wall can crack too. Six stories.

> **V1. Three owners in three days.** The vault write path changed three times in three days (a subagent, an advisor, the main agent). Nobody named the contract's owner. Fix: one owner, written exceptions. *Rock rule: one owner per contract.* Pack: `harness/global/WATCHDOG.yml`.

> **V2. The missed write-back.** A missing write-back was noticed days later. The check ran at concern severity, a silent card. Fix: blocker severity; drive the exit from the `turn_end` hook; state the fact one turn before closing. *Rock rule: a check nobody sees is no check.* Pack: `harness/extensions/global/end-session.ts` (the severity setting is yours to add to `harness/global/WATCHDOG.yml`).

> **V3. The $265 advisor bill.** 6,991 per-turn reviews cost about $265 in 14 days; 83% made no tool call. Review ran per tool round. Fix: `reviewMode: agent-end` (about nine times fewer reviews). *Rock rule: review at the end of a run.* Pack: `harness/global/WATCHDOG.yml`.

> **V4. The header that vanished.** An edit at a stale line number can delete a neighbouring log header. Edits used line numbers from an old read. Fix: read the tail, append after it, copy the exact `[path#TAG]` header. *Rock rule: read the tail, then append.* Pack: `vault/schema.md`.

> **V5. Retiring a policy by regex.** A marker policy was stamped across 223 files. A blind regex strip corrupted instruction prose, because mechanical edits ran over instruction text. Fix: hand-edit instruction files; diff live files against what the human approved. *Rock rule: never regex-edit instructions.* Pack: none.

> **V6. Two lost days.** A test run dated one day ahead left a guard line, so the first real run skipped two days. Fix: the script reads guard lines and backfills oldest first (cap 21 days); test in a sandbox copy. *Rock rule: test in a copy, never on the real vault.* Pack: `automation/slack-ingest/run.sh`.

Skill drift: when recipes fight

What. Skill drift is two skills, or a skill and a rule, that say different things. **Why.** Agent obeys the nearest recipe. If recipes fight, it picks one and you never know which.

Contradictions and copies

ID	Two places disagreed	The fix, and the law
D1	The story command routed "Ready for Development or later" to Session B. The skill took only "Ready for Development".	One place routes. The command passes the state.

ID	Two places disagreed	The fix, and the law
D2	The bug skill started services from a detached shell. A rule forbade it.	The rule wins. Use the `services` tool.
D3	Six skills ran git in `-bare` paths. A guard forbids it.	Use task-folder links and the `main` folder.
D4	One skill stashed a committed fix. Another said that saves nothing.	`git diff --quiet HEAD -- <files>` picks: stash if uncommitted, checkout from `origin/main` if committed.
D5	One rules file said skill-directed moves need no prompt, then said ask.	One statement. Exceptions in one list.
D6	The bug skill let the agent write `Closed`. Others forbade it.	The tester closes.
D7	Is `Resolved` a "review or done" move? Rules asked. Skills did not.	Name the four states the agent writes and the three it never writes.
D8	The script symlinks `.env`. Two skills said "copy".	The script is the truth.
D9	One skill kept the old board after the rule changed.	Skills point at the rule. They never hold the value.
D10	The bug state order changed three times in five weeks.	One ladder note plus `get_type`. Skills hold triggers only.
D11	Four PR-build command lists. A story passed its gate and failed CI.	One table, checked against each PR pipeline YAML.
D12	The router omitted 7 skills.	A row for every skill.
D13	`tasks-deployed-to-qa` takes an environment input but checks QA only.	State the limit, or implement it.
D14	A rollout changed the default channel in one skill. Three more kept the old one.	Constants in one file. Grep every copy. Second reviewer.
D15	A folder move left stale paths in six skills and 56 dangling symlinks.	Grep, grep again, remove generated links.
D16	Skills told the human to type `/move` after the tool existed.	When a tool replaces a step, search every skill for it.
D17	An older skill copy in a memory cache lacked a step.	One copy. Check the roster for twins.
D18	Port numbers copied into three skills and a registry page.	The starter of the service owns ports.
D19	A skill bound an advisor an audit said no roster defined. [INFERENCE] The audit looked in the wrong place.	Verify a binding in a headless run.
D20	Skills cite each other by step number. A renumbering broke three.	Cite by heading.
D21	The state-move table sat in 4 skills, the re-fetch rule in 6.	One shared rule or context file.
D22	A note claimed a Monday catch-up. A skill claimed all four agent files used the debugger override.	Read the target file before you claim.

What a teammate's guide showed

A teammate's guide listed 61 practices. Against 24 skills: 8 covered, 27 partial, 26 missing. Six groups were folded in (13 files touched, two new skills). QA and prod release steps stayed out on purpose: the author does not own releases.

Claims not proven yet

Do not present these as proven. Each is a test you owe.

- `tool-call-auditor`: a live smoke test ran with no dialog. The cause stayed open. No automated test.
- Local auto-login with `flow-runner`: not run end to end. Mode presets on Ctrl+0 to 4: key delivery not confirmed.
- `pr-comments` reviewer mode and colleague-branch pickup: written, never run on a real PR.
- Returned-bug Step 0, the mutation check, the self-review gate: not exercised on a live bug when added.
- The pre-PR security gate: nine verification steps planned, results not recorded.
- The nine chat rules, and `/clear` returning to the launch root: no live confirmation.
- A poller framework: design only. [INFERENCE] A detached mode survives the broker.
- Vibe mode for the ticket pipeline: the director loses shell, MCP, and `services`. Proposal only.
- The reference `main` folder is shared: uncommitted changes there affect other sessions.

Meta-lessons

Nine laws from the fights above.

1. One home per fact. Facts in the context file, procedures in skills, gates in rules plus config, mechanics in scripts.
2. Reference, do not copy. Cite by heading, never by step number.
3. Every decision triggers a repo-wide search for the old wording, plain-word synonyms too. Then a second reviewer.
4. Audit on a schedule. One read-only pass after three months found 23 contradictions.
5. A rules file that contradicts itself is a bug. Keep exceptions in one list.
6. Skills made mid-session do not load. Test fresh.
7. "Known gaps" in a decision is the early warning for drift. Make each gap a task or an owner.
8. Order edits that share a file. One change can renumber every later line.
9. A human who changes a rule twice was guessing the first time. Ask why before the fourth edit.

Team and process lessons

The team learned too. The short list.

From the retrospective (nine people, twelve action items)

- Keep: quality over speed, with engineers owning estimates. Open talk about surprises. Quick stakeholder updates.

- Stop: many chat tools (pick one channel). Merging to `main` untested. PRs waiting days for review. AI-driven PR volume outrunning reviewers (split big PRs early). Under-specified tickets (name who owns readiness). A UAT environment that reuses QA. Last-minute deploy asks (name one person who may decline). Many voices to the client (use one contact).
- Start: share the retro board a day early. Refine the backlog before every planning. Write a Definition of Ready and a Definition of Done. T-shirt size requests before design.

From the launch month

- Turn the brain-dump plan into an ordered task list with owners and a week-by-week timeline. Name who confirms Go or No-Go.
- Split "technically live" from "launched". Freeze scope. New asks go through product and delivery leads.
- Route feedback through named testers. Make "what is deployed where" a cheap query: testers once judged pre-fix code.
- Escalate shared blockers early. Name a backup for every credential. Write a handover plan before a lead leaves.
- Release gate: zero critical and zero high static-scan findings. Document false-positive classes as waivers.
- Commit to a regression test per bug fix, the regression suite on dev before answer-changing merges, and local tests against a client-like environment.
- Code freeze before go-live. Name an incident owner. Keep one source for dates.

About agents in a team

- Ticket quality is the multiplier. A thin ticket bounces. That matches the retro's Definition of Ready.
- Bugs and stories run alone to an opened PR. Humans own posts with an audience (except the fixed PR-open note), merge, release, and the closing states.
- The tester closes bugs. The product owner accepts stories.
- An agent's review is a lane, not a vote.
- Expected evidence: reproduction, failing test, mutation check, before and after image, a click-path comment for the tester.

The numbers

Numbers are quotable. Each carries its context.

What	Number	Context
The window	about 160 sessions, 680 to 700 human turns, two weeks	Main sessions only. Agent-side logs outnumber top-level ones 2.7 to 1.
Prompts	median 61 characters (11 words); 15% saved commands	Terse and dictated. Typos are normal.
Accepted replies	up to 3,400 characters; at least 35 one-word answers	Confusion came from codes and labels, not length.
Confusion and friction	28 cases: 11 rule missing, 9 broken by the main agent, 3 followed but rejected	6 of 8 confusion cases had no rule that could help.

What	Number	Context
Session outcomes	300 sessions in five weeks: 31% harness work, 23% aborted	Ticket and PR sessions aborted 29%.
Habits	110 `/clear` in five weeks; 31 shell calls in the first six calls of 60 sessions	A resume file per ticket was proposed. After the `read` rule: 2 shell `ls` calls before, 0 after.
Advisor economics	6,991 reviews, about $265, 14 days	83% made no tool call. `agent-end` gives about 9 times fewer.
Size of the machine	27 project skills, 7 wiki skills, 11 commands, 12 rules, 4 agents, 3 extensions	From 15 skills in early September.
Model-behaviour proof	baseline held in 14 of 15 runs; the fix passed 5 of 6 criteria	A five-run recheck did not reproduce the miss.
Tool limits	shell cap 300 s; Playwright 30 s; a gateway probe hung past 60 s	Plan for them in scripts.
Vault and papercuts	279 decision pages; 30 papercuts in 30 days	Decision pages carry "Known gaps".

Make It Yours

You are new. The org is new. Do not copy blind. Copy the laws. Rewrite the names. Walk the plan below, in order.

The plan at a glance

When	Goal	Done when
Day 1	Light the fire	The harness loads your files, a guard blocks a bad call, placeholders are filled
Week 1	One bug, end to end	One real bug goes from ticket to opened PR, with evidence
Weeks 2 to 4	Walk the road	PR, review, merge, QA check, vault, one story, one morning brief
After that	Grow	Your own skills, from your own papercuts

Day 1: light the fire

What. Install the harness. Copy the pack. Fill every placeholder. Prove a guard bites.

Why. All the craft hangs on the harness loading your files. A bad first hour costs a day.

How. ("The Workshop" has the long version.)

1. Install your harness (the author used OMP). Run it once.
2. Make the tasks root: `~/work/<PROJECT>/tasks`. Start every session there.
3. Bare-clone each repo as `<repo>-bare` beside `tasks/`, with `skills/repo-onboarding/SKILL.md`. Put every file of `scripts/` in the tasks root, the two `gitleaks` files too. Edit the `excluded_repos` array in `scripts/setup-task-branch.sh`.
4. Make `<tasks-root>/.omp/`. Copy in `harness/project/*`, `skills/`, `rules/project/`, `commands/`, `agents/`, and `harness/extensions/project/`. Copy `harness/global/*`, `rules/global/`, and `harness/extensions/global/` into `~/.omp/agent/`.
5. Make `mcp.json` from `harness/mcp.template.json`. Export the token variables in your shell before you launch. Pin the Playwright MCP version. Keep the secrets file outside git.
6. Fill every placeholder (table below). Search for `<`.
7. Restart. Open a fresh session in the tasks root. Ask which skills it sees. Run `/pr-status`: it only reads, so it is a safe first call.
8. Prove a guard. Ask the agent to read a file above `tasks/`. See the block. Ask it to kill a process it did not start. See the block.
9. Start your papercuts log: one line per time sink (date, symptom, fix, project).

Where in pack. `harness/`, `scripts/`, `skills/repo-onboarding/SKILL.md`, `harness/mcp.template.json`. Give your agent `HANDOFF-FOR-AGENTS.md` too: its Section 9 is the adoption procedure.

> Rock rule: restart after every change. Test in a fresh session.

Week 1: one bug, end to end

What. Take one small, real bug. Run `/bug <number>`.

Why. One full lap teaches more than ten pages. Every gate shows itself in the order it bites.

How.

1. Pick a bug with a visible symptom and one repo. Ask a teammate to confirm it is small.
2. Run `/bug <number>`. Watch bootstrap make the task folder. Check it yourself with `read`.
3. Watch the agent reproduce first. No repro, no fix.
4. Read the root cause. It must name `file:line`. Check one citation by hand.
5. Watch the failing test come first. Then the smallest fix.
6. Run the mutation check yourself once. Remove the fix: the test must fail. Restore it: the test must pass.
7. Demand one live check against the real dependency. A mock is not proof.
8. Read the self-review and the security gate report.
9. The PR opens. Read it as a stranger: title, template, evidence, work-item link. Re-fetch the work item and check the link.
10. Hold the state at `Ready for Fix` while the PR is open. `Closed` belongs to the tester.
11. Tear down. Nothing should still run. Write each lost hour into the papercuts log.

Where in pack. `skills/bug-workflow/SKILL.md`, `commands/bug.md`, `skills/regression-test-placement/SKILL.md`, `skills/pr-creation/SKILL.md`.

> Rock rule: reproduce, test, fix, prove. In that order.

Weeks 2 to 4: the PR road, review, vault

What. Walk the rest of the road. Then plant the vault.

Why. After the fix the road is long: review, conflicts, merge, deploy, retest. Each stop has its own skill.

How.

1. Week 2: live a PR's life. `/comments <pr>` for review threads. `/conflict <pr>` for a conflict. `/pr-merge <pr>` when green. Then `post-merge`. Ask `/qa-deployed` before the tester retests.
2. Week 3: review someone else's PR with `/ado-review <pr>`. Read the three lane reports and the verified summary before you post. Check auto-complete before you vote.
3. Week 3: build the vault (six steps in "The Long Memory"). File one meeting, one decision, one lint report.
4. Week 4: run one story as two sessions. Refine with `/triage` or `/story`. Run Session B in a fresh session. Let a contract gate fail once, on purpose. Watch the bounce.
5. Week 4: add the morning brief. Test `board-ingest` and `slack-ingest` by hand on a sandbox copy before you install cron (`automation/README.md` says how).
6. Fix the first gaps in the list below.

Where in pack. `skills/pr-comments/SKILL.md`, `skills/pr-merge/SKILL.md`, `skills/post-merge/SKILL.md`, `skills/pr-review/SKILL.md`, `skills/story-workflow/SKILL.md`, `skills/morning-brief/SKILL.md`.

What to keep, what to adapt

What. Keep the laws. Adapt the facts. **Why.** The laws came from falls. The facts came from one client.

Keep: the laws	Adapt: the facts
Re-fetch after every tracker write	Your tracker, state ladders, vocabulary table
A ticket is a contract. A thin one bounces.	Ticket template, Definition of Ready
Reproduce, test first, mutation check, live verify	Test commands, the PR-build table
One boring script wires every folder	Repo roster, standing exclusions
Guards in code for hard laws. Fail closed.	Which paths, tools, and processes you guard
The human owns the end of the ladder	Who closes: your tester, your product owner
Gates by facts, not always-yes prompts	Which posts have an audience on your team
A vault with a schema and an append-only log	Vault folder, cron times, time zone
A papercuts log	Models per role, login method, environments, scan tool

The placeholder list

What. Every `<NAME>` token you must replace. **Why.** An unfilled placeholder is a silent bug.

Placeholder	Meaning	Where you meet it
`<PROJECT>`	the platform name, and the workspace folder name	everywhere; start at `harness/project/AGENTS.md`
`<CLIENT>`, `<EMPLOYER>`, `<PRODUCT>`	the client org, the consultancy, the assistant product	`harness/project/AGENTS.md`
`<ADO_ORG>`, `<ADO_PROJECT>`, `<ADO_PROJECT_URL_ENCODED>`	tracker org, project, and the project name with `%20` for URLs	`harness/mcp.template.json`, `skills/my-pr-status/SKILL.md`
`<AREA_PATH>`, `<BOARD>`, `<AREA_PATH_1>`, `<AREA_PATH_2>`	tracker area path and board	`rules/project/ado-area-path.md`, `automation/board-ingest/prompt.md`

Placeholder	Meaning	Where you meet it
`<QA_URL>`, `<DEV_URL>`, `<DEV_API_URL>`	web-app and API address per environment	`harness/project/AGENTS.md`, `skills/post-merge/SKILL.md`
`<YOUR_EMAIL>`, `<YOUR_EMAIL_ESCAPED>`, `<YOUR_DISPLAY_NAME>`, `<TEST_USER_EMAIL>`	your tracker identity and the test user	`harness/project/config.yml`, `harness/project/AGENTS.md`
`<TECH_LEAD>`, `<QA_TESTER>`, `<PRODUCT_OWNER>`, `<TEAMMATE>`, `<MANAGER>`	people, by role. Real names live in vault `people/` pages	`harness/project/AGENTS.md`, `skills/pr-creation/SKILL.md`
`#<team-channel>`, `#<eng-channel>`, `#<all-teams-channel>`	Slack channels	`harness/project/AGENTS.md`, `skills/pr-and-team-comms/SKILL.md`
`<PIPELINE_ID>`, `<E2E_PIPELINE_ID>`	pipeline definition ids (deploy, API e2e regression)	`skills/deployed-version-check/SKILL.md`, `skills/post-merge/SKILL.md`
`<TENANT_ID>`, `<SUBSCRIPTION_ID>`	cloud ids. Keep them on a vault project page for infra	`skills/deployed-version-check/SKILL.md`
`<STANDARDS_REPO>`	the tracker repo that holds Git and PR standards	`harness/project/AGENTS.md`, `skills/pr-creation/SKILL.md`
`<TEAMMATE_1>`, `<IDENTITY_ID_1>`, `<REPO_ID_*>`, `<RESOURCE_GROUP_*>`, `<APP_SERVICE_*>`, `<QA_SUBSCRIPTION_NAME>`, `<REGION>`	standing reviewers, repo ids, and cloud names for PR creation and the live-config check	`skills/pr-creation/SKILL.md`, `skills/deployed-version-check/SKILL.md`
`<VAULT_ROOT>`, `<TIMEZONE>`, `<ENV_VAR_NAME_HOLDING_BASE64_PAT>`, `<PATH_TO_E2E_SECRETS_FILE>`, `<ENV_VAR_NAME_HOLDING_SLACK_XOXP_TOKEN>`	your vault folder, your cron day boundary, and the names (never the values) of secret variables and files	`harness/global/APPEND_SYSTEM.md`, `automation/README.md`, `harness/mcp.template.json`

Open gaps worth fixing first

What. Real drift, checked in the pack's files. **Why.** The clear-cut text drift is fixed in the shipped files. What is left needs a human or design call. Fix the top of this list in weeks 1 to 4. `HANDOFF-FOR-AGENTS.md` (Section 8) keeps the longer list for your agent.

How. Work down the list.

1. Scanner config. Copy `scripts/gitleaks.toml` and `scripts/gitleaks-report.tmpl` into the tasks root, next to `scripts/secret-scan.sh`. The config only extends the default rules. Add a rule-scoped allowlist block and smoke-test it: one case must fire, one must pass (fall P8). Not shipped at all: a plan template, the plan-mode hook (fall A7), and the papercuts file. Make them.
2. Git against bare repos. `skills/daily-work-poll/SKILL.md`, `skills/tasks-deployed-to-qa/SKILL.md`, `skills/ticket-triage-profile/SKILL.md`, and `skills/story-refinement/SKILL.md` still read `-bare` paths. The guard blocks them. Point them at the `main` folder or task-folder links (fall D3).
3. Resume routing. `commands/story.md` routes "Ready for Development or later" to Session B. The contract gate in `skills/story-workflow/SKILL.md` passes only on exactly `Ready for Development`. A resumed story in `In Development` would be sent back to refinement. Decide how a resume passes the gate (fall D1).
4. Unpinned browser. `harness/mcp.template.json` says `@latest` and sets no isolation flag. Pin the version and run isolated (fall H10).
5. Unproven claims. Run each item of "Claims not proven yet" once, live, before you trust it.

Grow your own skills

What. A skill is a recipe you wrote because you did the thing twice.

Why. Every skill here began as a papercut. The set grew from 15 skills in early September to 27 project skills and 7 wiki skills. That came from this habit alone.

How.

1. Do the thing by hand once. Note each step and each trap.
2. Do it a second time. Now write the recipe. Not before: you guess wrong. Not after the third time: it is late.
3. Shape it like `skills/wiki-sync/SKILL.md`: a header with `name` and a `description` that starts "Use when" (the agent sees the description, so it is the trigger), then When, Inputs, Steps, Gates, Writes, Log.
4. Keep one home per fact. Facts in `harness/project/AGENTS.md`, procedures in skills, gates in rules plus config, mechanics in scripts. Reference. Never copy. Cite by heading.
5. Add a row in `skills/workflow-router/SKILL.md`. Add a slash command only for a thing you type daily: under ten lines, naming the skill, passing the argument.
6. Test in a fresh session. A skill made mid-session does not load.
7. After any decision, search the whole pack for the old wording, plain-word synonyms too. Then ask a second reviewer.
8. Keep the papercuts log. When something hurts, sort it. Rule missing: write one. Rule broken: add a guard or a gate. Rule followed but wrong: change the rule.
9. Audit every three months. Changed a rule twice? Stop. Ask why. The first version was a guess.

Where in pack. `skills/wiki-sync/SKILL.md`, `skills/workflow-router/SKILL.md`, `skills/third-party/` (upstream skills; credits in `skills/third-party/CREDITS.md`; the workflow calls `grilling` and `research` by name), `harness/project/AGENTS.md`.

> Rock rule: second time, write the recipe. Third time, trust the recipe.

Dictionary

Words are hard. Here is every hard word in the book and the pack. Sorted A to Z. Each has a plain meaning and an example. A word confuses you? Come here first.

The examples use made-up names and numbers. Paths point into the pack. A path in angle brackets is a placeholder you fill.

Term	Plain meaning	Example
Acceptance criteria (AC)	Plain tests a story must pass to count as done. Each one can pass or fail.	AC-1: the list shows three items after a refresh.
Accepted	The last Story state. The product owner decides it. The agent never writes it.	A story moves to Accepted after the tester and the owner agree.
Actionable bug	A board bug with work left for you. "Picked up" means a folder, branch, PR, or live session, not just assignment.	A bug with a task folder and no PR yet.
ADO (Azure DevOps)	The tracker, repo host, and pipeline service the author used.	Bugs, Stories, PRs, and builds all live in ADO.
ado_states	A project page field: the map from vault status to tracker state.	`doing` maps to Active.
Advisor (watchdog)	A second model that reads the agent's work and whispers a warning. Silence means fine.	`WikiScribe` files facts. `PRSecurity` watches a review.
advisor-drain	An extension that holds the turn open until every advisor finishes.	It keeps an in-flight vault write from being lost.
Agent	The AI that does the work: reads files, calls tools, writes code.	The main agent runs `/bug 12345`.
Agent Contract	Six sections written into a Story ticket: acceptance criteria, resolved questions, repos, verification plan, deploy-order notes, stop conditions. Session B trusts only this.	A story with no stop conditions fails the gate.
AGENTS.md	The file of standing facts for a project: names, repos, state ladders, vocabulary.	`harness/project/AGENTS.md`
Allowlist	A list of things allowed to pass. A scanner allowlist hides known false alarms.	A rule-scoped block in `scripts/gitleaks.toml`.

Term	Plain meaning	Example
Always-on rule	A rule loaded into the prompt every turn. Its header says `alwaysApply: true`.	`rules/global/read-tool-first.md`
Amend	Change the last commit instead of adding a new one.	`git commit --amend` keeps one commit per repo.
Ancestry check	A git test: is commit A inside the history of commit B? It proves a fix shipped. Dates cannot.	`git merge-base --is-ancestor <fix> <deployed>`
Anchor (review comment)	The file and lines a review comment points at. It must sit on lines the PR changed.	A thread anchored on an untouched file never shows.
API	A set of web addresses that programs call to ask for things.	The `orchestrator` is a backend API.
Append-only	You may add lines at the end. You may never edit or delete old lines.	`vault/log.md`
APPEND_SYSTEM.md	A file that adds your manners to the system prompt: knowledge limits, wiki, writing standard, chat style.	`harness/global/APPEND_SYSTEM.md`
Approval gate	A check that stops a tool call until a person says yes, or blocks it. Three layers: prose rule, skill step, harness config.	`tools.approval` in `harness/project/config.yml`.
Approval phrase	A clear yes: "send it", "approved", "go". Anything vaguer means ask again.	The agent posts a Slack draft only after "send it".
Area path	A tracker field that places an item on a board or team. The board is an area path, not an iteration path.	Query the area path to see one board.
ArtifactLink	A work item relation that points to a PR or a commit.	Read it to find the fix commit.
ASD-STE100	Simplified Technical English: a plain writing standard. Short active sentences, one word per meaning.	"Open the file." not "The file should be opened."
Assistant (the product)	The client's chat product that the platform builds. Not the coding agent that helps you.	A change to what the assistant says gets a catalogue case.
Attach (debugger)	Connect the debugger to a service that already runs.	Attach to the Python service on its debug port.
Auditor (`tool-call-auditor`)	A small model that reads one ticket comment or browser snippet and answers SAFE or not. Only SAFE skips the dialog.	It reviews comment text for secrets.

Term	Plain meaning	Example
AUTH HANDOFF	A `flow-runner` stop: the page needs a human login. Log in, then say `continue`.	QA runs always stop here.
Author (the author)	The engineer who built this setup. The only human the agent works for.	The author approves drafts and merges.
Auto-complete	A PR setting that merges the PR by itself once every policy passes. Arm it last.	Armed too early, a PR merged unreviewed in about 15 seconds.
autoCompleteSetBy	A PR field that shows who armed auto-complete.	Check it before you vote on a colleague's PR.
Backfill	Fill in the days a job missed, oldest first.	`slack-ingest` covers Friday to Sunday on Monday.
Backlog	The list of work not yet planned into a sprint.	A story in Backlog still needs refinement.
Bare repo	A git clone with history and no checked-out files. It stores the worktrees. Agents never touch it directly.	`web-app-bare/`
.base file	An Obsidian saved table over the header fields of vault pages.	`vault/views/tasks.base` shows the task board.
Base64	A way to write bytes as plain text.	The PAT variable holds `user:pat` in base64.
Baseline	A run on `main` before the fix, to prove the problem exists.	Gate: at least 1 of 15 baseline runs must show the bug.
Blocked	A state that says: stopped, waiting on a person.	A story moves to Blocked when the code contradicts the contract.
Blocker (severity)	The top advisor level: fix this now.	A fails-open guard about to be reported clean.
blocks (Slack)	A structured Slack message format. Links render only through its rich-text elements.	The PR-open line uses a `blocks` payload.
Board	The tracker view of work items in columns. The area path picks the board.	The team board.
Bootstrap	Make a ticket's folder, branches, and worktrees with the script, then set the first state.	`ticket-bootstrap`
Branch	A named line of commits. One ticket, one branch.	`fix/prompt-search-mismatch`
Branch policy	Tracker rules that guard `main`: required reviewers, builds, a linked work item. The real gate.	An unlinked PR can be blocked.

Term	Plain meaning	Example
Breakpoint	A line where the program pauses so you can look inside.	Set one on the suspect path, then re-fire the request.
Brief (morning brief)	A vault page the morning job writes: what to close, what needs a QA check, what needs you.	`/morning` works it.
Broker (harness)	A harness process that supervises background processes. The sources use the word without more detail.	"A detached mode survives the broker." [INFERENCE]
Bug	A work item for broken behaviour.	Bug 12345.
Bug states	The ladder: New, Ready for Triage, Ready for Fix, Resolved, Ready for Retest, Closed. Side states: Bug Test Failed, Blocked, Reopen. Ends: Cancelled, Duplicate, Deffered (the tracker's spelling).	The agent writes up to Ready for Retest. The tester writes Closed.
BUG_FIXING (rule)	The global rule for bug work: reproduce at the real boundary, verify live, re-check every helper claim.	`rules/global/BUG_FIXING.md`
Bun	A fast JavaScript runtime. The pack uses it to build the PDF.	`bun build.ts`, run inside `build/`.
bypassPolicy	A PR option that skips branch policy. Never set it.	`bypassPolicy: false` in the merge call.
Caption file (.vtt, .srt)	A subtitle text file of a recorded call: cue numbers, timestamps, spoken lines.	`wiki-meeting` cleans it before filing.
Catalogue (prompt-regression catalogue)	A JSON list of prompts with the behaviour the assistant must show. Each case has an id, a prompt, guards, and `expect` rules. It runs nightly on dev.	Add a case when an answer broke. `expect` keys: `tools_none`, `max_seconds`.
Checkmarx (CxOne)	A static scan product. It finds risky code patterns without running the code.	A scan report gives counts per engine.
Chromium	The open-source browser that Playwright drives.	Pin the Playwright MCP, or Chromium moves under you.
CI	Continuous integration: a server builds and tests every push.	The PR pipeline runs lint, typecheck, and tests.
`/clear` (clear context)	Wipe the chat memory of one session. Files and vault stay.	The agent calls `clear_context` when you ask in plain words.
CLI	Command-line interface: a program you run by typing.	`az` is the cloud CLI.

Term	Plain meaning	Example
Click path	The steps a tester clicks to see the fix.	"On QA: Settings, then Export, then Download."
Client	The organisation that owns the platform. The author works for it through a consultancy.	`<CLIENT>` in the pack.
Closed	The last Bug state: fix verified by a human. Only the tester writes it.	The tester retests, then writes Closed.
Code freeze	A period when only critical fixes merge.	Before go-live.
Commit	A saved snapshot of changes, with a message. One per repo per ticket.	`fix(scope): stop the crash (BUG-12345)`
Compaction	Shrink a long chat into a summary to free room.	The agent calls `compact_context` when you ask.
Conflict (merge conflict)	Two changes touched the same lines and git cannot choose.	`/conflict 4567`
Consultancy (employer)	The firm that employs the author and sends them to the client.	`<EMPLOYER>` in the pack.
Context window	How much text the model can hold at once.	A long chat can overflow it.
Contract gate	Session B's first check: is the contract whole? If not, stop and send the story back.	Missing stop conditions fails it.
Control prompt	A prompt that must still behave right after a fix. It guards against over-fixing.	A prompt that must still call a tool.
Conventional Commits	A title format: `type(scope): description`.	`fix(prompt-ideas): filter by direct match (BUG-12345)`
Cron	A clock-based job runner.	`board-ingest` runs on weekday mornings.
Cross-wiring	A service talks to the wrong sibling, often from another branch. Health stays green.	`lsof` shows the PID's cwd is another worktree.
CSS selector	A pattern that picks one element on a web page.	The Playwright MCP takes a snapshot ref or a CSS selector.
curl	A command that makes a web request.	`curl localhost:5173` proves the dev server answers.
cwd	Current working directory: the folder a command runs in.	The harness reads `<cwd>/.omp`.
Daily work poll	A report of what you did on a day, from the tracker, local review folders, and git history.	"What did I do on Tuesday?"

Term	Plain meaning	Example
DAP	Debug Adapter Protocol. How the agent talks to a debugger.	`harness/project/dap.json`
Debugger	A tool that pauses a running program so you can look at real values. Better than print statements.	Set a breakpoint, re-fire the request, read the values.
debugpy	The Python debugger server.	Start it with `--wait-for-client`.
Decision page (ADR)	A vault page for one choice: Decision, Context, Reasoning, Consequences.	`vault/examples/decisions/`
Definition of Ready / Done	Team checklists: when a story may enter a sprint, and when it counts as done.	No unrefined story enters planning.
Deploy order	Which repo ships first. The contract says what a user sees in each order.	A fix in two repos has three orders: A first, B first, together.
Dev, QA, prod	Three environments. Dev is for developers, QA for testers, prod for real users.	Dev sits behind a private endpoint.
Diff	The list of changes between two versions of the code.	`git diff origin/main...HEAD`
Director (vibe mode)	A mode where the main agent only reads and hands work to workers.	`vibeExtraTools` adds tools back to the director.
Discriminating step	A flow step whose result differs between the broken and the fixed build. The verdict looks only at these.	`discriminating: true`
DM	Direct message in Slack.	Merge asks go by DM, never to a channel.
Docker (compose)	Docker runs apps in containers. Compose starts several from one file.	The orchestrator needs a database container.
Draft	Text shown in a fenced block before it is posted. Nothing is sent yet.	A Slack draft waits for "send it".
Draft PR	A PR marked not ready: no auto-complete, no Slack line.	Opened when a security finding is too big for this PR.
Drift	Two things that should match no longer match.	The vault says review. The tracker says Active.
Dry run	Show what would happen. Change nothing.	`scripts/cleanup-task-folders.sh` without `--apply`.

Term	Plain meaning	Example
e2e (end to end)	A test that goes through the whole path, from the screen to the storage.	The API e2e regression pipeline.
Elicitation	A tool asking a human for a missing choice. It fails when nobody is there.	A tracker call with no `project` in a cron run.
.env file	A text file of settings and secrets for one repo. Agents never read it, print it, or copy it.	One `.env` sits at the bare-repo root. Each worktree links to it.
.env.local	A second env file the agent may manage. Its first line is a fixed marker. No secrets inside.	Non-secret local overrides only.
Escalation	Stop, comment with proof, tag a person, and set the state to Blocked.	The code contradicts the contract.
Evidence	Proof you can re-run: command output, a diff, a screenshot.	Failing test output under `## Failing test`.
Exfiltration	Sneaking data out of a system.	A change that posts secrets to a stranger's server.
Exit code	The number a program returns. 0 means fine.	`scripts/secret-scan.sh` exits 1 on findings and 2 on a usage error.
expect (flow)	What a flow run must see: `broken` on the pre-fix build, `fixed` on the fix branch.	Run 1 uses `expect: broken`.
Extension	A TypeScript file the harness loads at start. It adds tools or guards.	`harness/extensions/global/guardrails.ts`
Fail closed / fail open	Closed: on error, block. Open: on error, allow. Guards should fail closed.	The approval gate blocks governed tools if its config is unreadable.
Fail-first	Another name for test first: the new test must fail before the fix.	Record the failing output.
Failure boundary	The exact place where the bad thing really happens. Test there, not behind a proxy.	Call the backend directly, not through the dev proxy.
Fall	A story of something that went wrong, with its fix and the law it made.	"Merged in fifteen seconds."
False positive	A scan hit that is not a real problem.	A test key in a fixture.
FastAPI, FastMCP	Python frameworks for web APIs and for MCP servers.	`orchestrator` uses FastAPI.

Term	Plain meaning	Example
Feedback thread, chit-chat thread	Merge-flow classes of an open PR thread. Feedback asks for a change and blocks. Chit-chat asks for nothing, and the human decides.	A thank-you comment is chit-chat.
Fenced block	A code block between triple backticks. All drafts go in one.	A Slack draft in a fenced block.
Fetch refspec	A rule that maps remote branches to local refs for `git fetch`. Bare clones have none.	`+refs/heads/<b>:refs/remotes/origin/<b>`
FETCH_HEAD	Git's note of the last fetch. A plain fetch in a bare clone updates only this.	Use an explicit refspec instead.
Flow	A step list in `flow.yaml` that a cheap vision model follows in a browser, then returns a verdict.	Three runs: broken, fixed, QA.
flow-runner	The subagent that runs one flow, in one environment, with one expectation.	`agents/flow-runner.md`
Force push	Overwrite a remote branch with your local one. Dangerous.	Never `--force`. Use force-with-lease.
Force-with-lease	A safer force push. It fails if someone else pushed meanwhile.	Only on your own amended branch.
Frontmatter	The header block at the top of a Markdown file, between `---` lines.	`type: task`, `status: doing`
Gate	A point where the agent must stop and ask, or must prove facts before going on.	The verification gate before a PR.
get_type	The tracker call that returns a work item type's real states and rules.	Read the Bug ladder with it. Never assume.
gitleaks	A tool that scans commits for secrets.	`scripts/secret-scan.sh` runs it.
Go or No-Go	The launch decision. One named person confirms it.	The launch plan names who says Go.
Go-live	The day real users get the product.	A code freeze comes before it.
Grilling	One hard question per turn, each with a recommended answer, grounded in code.	"The ticket says X, but `file:line` does Y. Which wins?"
Guard	Code that checks a tool call before it runs. It can block, ask, or allow.	`harness/extensions/project/no-parent-folder-guard.ts`

Term	Plain meaning	Example
Guard line	A fixed log line a job checks, so it skips work already done.	`board scan <date>` in `log.md`
Guardrails	The global extension that hides secrets and blocks key files. "A guardrail, not a sandbox."	`harness/extensions/global/guardrails.ts` masks values as `*****`.
GUID	A long unique id.	Creating a PR needs the repository GUID.
Handoff doc	A page for a reader with no context.	The tester handoff page (see `skills/post-merge/SKILL.md`).
Harness	The program that runs the agent: prompt, tools, skills, rules, model calls.	OMP.
Headless	Run with no screen and no person.	`omp -p` in a cron job.
Health check	A request that asks a service "are you alive?". Green does not prove it runs the right code.	A green stack served another branch.
Hook	A place where an extension can step in: before a tool call, at turn end.	The `tool_call` hook.
HTTP status code	A number a web server returns: 200 fine, 400 bad request, 401 not signed in.	A state jump returned 400.
hub	A harness command that starts and stops named processes. A direct `hub` start needs the human's approval. The `services` tool is the default door.	`hub start` timed out waiting on a dev server.
Idempotent	Safe to run twice. Same result.	`scripts/setup-task-branch.sh` reuses existing worktrees.
Inbox (task status)	A task you have not yet decided to do.	Meeting action items land here.
Incident owner	The person named to run the response to an outage.	Named before go-live.
Index (`index.md`)	The vault's table of contents. Read it first.	`vault/index.md`
[INFERENCE]	An inline tag on a claim nobody verified.	"The cause is a cache. [INFERENCE]"
Ingest	File a new source into the vault: raw, source page, links, index, log.	`wiki-ingest`
Integration test	A test that uses real dependencies, not mocks.	Marked `integration`. Not in the PR build.

Term	Plain meaning	Example
Iteration (path)	The tracker's sprint slot for a work item. Not the board.	`System.IterationPath`
JWT	JSON Web Token: a signed login token.	`file-gateway` checks one on its routes.
Kebab-case	Lowercase words joined by hyphens.	`fix-meter-reading-rounding`
Key name (secret by name)	The agent passes a name like `TEST_USER_PASSWORD`. The tool types the real value and hides it.	The secret never enters the model's context.
kill, pkill, killall	Commands that stop processes by id or by name. The guard always blocks `pkill` and `killall`. It blocks `kill` unless the target is a literal PID this session started.	`kill $(lsof -ti :PORT)` is blocked.
Known gaps	A decision page section that lists what is not proven. The early warning for drift.	Turn each gap into a task or an owner.
Lane (review lane)	One reviewer's slice: security, quality, or architecture. Read-only.	Three lanes run in parallel.
Ledger	A list of processes this session started. Only these may be stopped.	`process-ownership-guard`
Light check	A fast secret scan before any push. A finding triggers the full security gate.	`scripts/secret-scan.sh` before an amend and force-push.
Lint	A tool that flags style and error patterns in code. The vault has its own lint.	`pnpm run lint`, `wiki-lint`
Live verification	Prove the fix against real services, not mocks.	A real model call and a real blob read.
Localhost	This computer, as a network name. The IPv6 loopback is the same machine over IPv6.	The web app runs at `localhost:5173`.
Lockfile	A file that pins exact package versions. Regenerate it, never hand-merge.	`pnpm-lock.yaml`
Log (`log.md`)	The vault's append-only event list.	One header per write: date, op, title.
lsof	A command that lists the files and ports a process uses.	`lsof -nP -iTCP:5173 -sTCP:LISTEN`
LSP	Language Server Protocol. It gives the agent go-to-definition, references, and errors.	`scripts/setup-task-lsp.sh` wires it.

Term	Plain meaning	Example
Main agent	The one session you talk to.	It delegates to workers and holds the gated writes.
`main` (trunk, folder)	The protected shared branch. Also a folder in the tasks root with repo links synced to `main`.	Read current main from the `main` folder.
Markdown, HTML	Two text formats. A tracker field uses one. Match it when you write.	An HTML field needs an HTML contract.
Mask	Replace a secret with `*****` in tool output. Never write the mask back to a file.	A `.env` read shows `*****`.
MCP	Model Context Protocol: a way to plug tool servers into the agent.	Azure DevOps, Slack, Playwright.
Merge status	The tracker's flag for whether a PR merges cleanly: conflicts or no conflicts.	Show the words, never the number.
MERGE_HEAD	A marker that a merge is in progress.	`stash pop` once dropped it.
MFA	Multi-factor authentication: a second proof at sign-in, such as a code.	Prompts varied between test runs.
Mock	A fake stand-in for a dependency in a test. It holds your guess.	A mock rejected on abort. The real library resolved.
Mode preset	One key that sets the model and a mode (plan, vibe, goal).	Ctrl+0 to 4. Key delivery is not confirmed.
Model role	A named slot in the config that picks a model for a job: planning, small tasks, advisors, auditor.	`modelRoles` in `harness/global/config.yml`.
`move_session`	The tool that moves the session into a task folder.	It moves only to the launch folder or a direct child.
Mutation check	Remove the fix: the new test must fail. Restore it: the test must pass.	It proves the test guards the fix.
Nit (severity)	The lowest advisor level: a hint.	The scribe's note after a write.
OBO (on-behalf-of)	A login flow where a service swaps your token for a new one, to call another service as you.	A backend reads your file through a second service, as you.
Obsidian	A note app that opens a folder of Markdown files as a vault.	It makes `vault://wiki/` resolve.
Obsidian Bases	Obsidian's saved tables over page header fields. See `.base` file.	`vault/views/`
OIDC	OpenID Connect: the standard "sign in with your account" flow.	The web app signs in through OIDC.

Term	Plain meaning	Example
OMP	oh-my-pi: the terminal coding-agent harness the author used.	`omp -p` runs one headless prompt.
.omp folder	The project folder where the harness reads settings, skills, rules, and MCP servers. It reads only `<cwd>/.omp`.	`<tasks-root>/.omp/`
Optional reviewer	A reviewer who may vote but never blocks the merge.	The API can add only optional reviewers.
orchestrator	Two meanings. The repo: the backend API that runs the assistant's LLM agent. Also the main agent that hands work to workers.	"The orchestrator re-reads every worker diff."
Orphan (page, process)	A vault page nothing links to, or a process left running.	`wiki-lint` lists orphan pages.
Overlay	A config layered over defaults for one run.	The cron jobs use one that denies write tools.
Pack	The folder of files this book describes: skills, rules, scripts, harness, vault kit.	Every section says where in the pack.
Paired PRs	One fix split across PRs, titled `(k/n)`. Merge order matters.	Merge `(1/2)` before `(2/2)`.
Papercut	A small thing that costs time. Log it in one line.	Date, symptom, fix, project.
PAT	Personal access token: a long password for a tool. Never print it.	Held base64 encoded in an env variable.
Pending PO Review	A Story state after the tester passes it. The product owner decides next. The agent never writes it.	The tester moves a story here.
PID	Process id: the number of a running process.	`lsof` shows the PID on a port.
PII	Personal data that identifies a person.	A home address or a phone number.
Pipeline	A scripted build, test, or deploy run on a server.	The PR build. The deploy pipeline.
Placeholder	A `<NAME>` token you replace with your own value.	`<ADO_ORG>`
Plan mode	A harness mode that plans before it edits.	A plan-only hook injects house rules.
Playwright	A browser automation tool, used through MCP.	`flow-runner` drives it.

Term	Plain meaning	Example
pnpm	A Node package manager. `npm` is blocked here.	`pnpm install --frozen-lockfile`
Policy	A tracker check a PR must pass before it merges.	Required reviewers, a build, a linked work item.
Poller	A job that checks a status again and again.	A design to watch PR status. Not built.
Port	A numbered door a service listens on. One owner per port.	`web-app` on 5173.
PR (pull request)	A request to merge a branch into `main`, with review.	PR 4567.
PR status	Active, Abandoned, or Merged. Say the word, never the code.	A merged PR shows Merged.
PR status card	The one fixed card `/pr-status` prints for your open PRs.	Votes, comments, checks, one state line.
PR template A and B	The two PR description shapes. A is for bug fixes. B is for features and chores.	A has Bug, Root cause, Fix, Tests, Evidence.
PR-build checks	The commands the PR pipeline runs: lint, typecheck, tests, build. Run them locally before every push.	`pnpm run lint`, `pnpm run test:ci`
PR-open line	The one fixed Slack line posted when a PR opens. No draft, no approval.	`PR 4567 for review (orchestrator): ...`
Private endpoint	A network address reachable only from inside a private network.	Dev answers only from a remote desktop.
Probe (with a control)	A test call that checks a service is alive. A control proves the probe can tell good from bad.	A bogus credential must fail the expected way.
Product owner (PO)	The person who accepts a story.	Writes Accepted.
Project agents (pr-security, error-detective, architect-reviewer)	Read-only helpers. `pr-security` rules on security findings. `error-detective` correlates logs across services. `architect-reviewer` checks boundaries and contracts.	`agents/pr-security.md`
Prompt	The text you give the model.	"Work Bug 12345."
Prompt injection	Text that tells the AI to ignore its rules. The auditor looks for it.	A comment that says "ignore previous instructions".
Proxy (dev proxy)	A local server that forwards requests. It can hide a client disconnect.	Test the backend port directly.

Term	Plain meaning	Example
PRSecurity	The dormant advisor that wakes when the `pr-review` skill loads.	It watches for fails-open findings during a review.
QA tester	The person who retests fixes and closes bugs.	Writes Closed on a Bug.
Raw (`raw/`)	The vault folder for untouched originals.	A pasted transcript.
Re-fetch	Read an item again after a write, to see the real state.	A write's success reply is not proof.
React, Vite	React builds web screens. Vite is its dev server and build tool.	`web-app` uses both.
Read-only	Allowed to look, not to change.	Review lanes.
Ready for Fix	A Bug state: the bug reproduced. It stays here while the PR is open.	The agent writes it after the repro.
Ready for Retest	A Bug state: the fix is on QA and the tester may retest.	Written after the ancestry check passes.
Ready for Test	A Story state: every PR merged. QA deploy not yet proven.	The agent writes it after the merge.
Rebase	Replay your commits on top of another branch. This pack never rebases to fix a PR conflict.	Merge the target in instead.
Refinement	Make a story clear enough to build: grill the author, write the contract.	Session A.
Regex	A pattern language for matching text.	`(?m)^foo` matches at each line start.
Regression test	A test that fails if an old bug comes back.	Every bug fix adds one.
Repo onboarding	Make a bare clone and a `main` worktree for a repo that has none.	`skills/repo-onboarding/SKILL.md`
Repo roles	Names for the platform's repos: `orchestrator`, `media-mcp`, `data-mcp`, `file-gateway`, `web-app`, `infra`, `legacy-agent`.	`web-app` is the React and Vite front end.
Repro (reproduction)	The exact steps that show the bug.	No repro, no fix.
Required reviewer	A reviewer whose approval blocks the merge until given.	Make the reviewer required before you arm auto-complete.
RESEARCH-00000	The task folder prefix for work with no ticket.	`RESEARCH-00000-blob-owner-check`

Term	Plain meaning	Example
Resolved	A Bug state: fixed, not yet verified. Written once every linked PR merged.	The agent writes it after the merge.
Retro (retrospective)	A team meeting about what to keep, stop, and start.	A retro gave twelve action items.
Returned bug	A bug the tester moved back after a fix.	It starts at Step 0.
`/rev` test	A tracker write guard: update only if the revision is still the one you read.	Optimistic concurrency.
Review mode (agent-end)	An advisor setting: review once per finished run, not once per tool round.	`reviewMode: agent-end` cut reviews about nine times.
Rock rule	A one-line law taken from a fall.	"Whoever starts it owns it."
Root cause	The real reason the bug happens. Name the file and line.	"The check runs only in `catch`."
Roster	The list of skills or advisors the harness knows.	The skill roster shows name and description.
Router skill	The skill that maps a ticket to the chain of other skills.	`workflow-router`
Rule	A law file. Always on, or on when text matches.	`rules/global/read-tool-first.md`
Rung	One step on a state ladder.	Write one rung per call.
Sandbox	A throwaway copy for tests.	Test scripts on synthetic bare repos.
SAS URL	A storage link with an access token built in. Never post one.	The auditor marks it UNSAFE.
SAST	Static application security testing: scan code without running it.	Checkmarx.
Schema (vault)	The contract file for vault pages.	`vault/schema.md`
Scoped rule	A rule with a `condition` regex and a `scope`. It fires only when text or a tool call matches.	`rules/project/ado-area-path.md`
Scout	A read-only helper that finds code for a ticket.	Triage uses scouts.
Scribe (WikiScribe)	The advisor that files durable facts into the vault.	It writes notes, decisions, and log lines.
SDD spec	The spec that `/triage` writes into a ticket: requirements, ACs, and tasks by tier and wave. The pack does not expand "SDD".	A tasks table with Tier, Executor, Wave.

Term	Plain meaning	Example
Secret scan	A check that finds keys and passwords in a diff before a push.	`scripts/secret-scan.sh`
Secrets file	The file that holds test credentials. Every read tool is blocked from it.	The Playwright MCP reads it, not the agent.
Security gate	The pre-PR check: a secret scan plus a `pr-security` ruling per finding, with fix or defer rules.	`skills/pr-security-gate/SKILL.md`
Self-review	A script that writes your diff and flags new code comments, env files, dependency changes, and secrets.	`scripts/self-review.sh`
Services tool	The custom tool that starts, stops, and checks the five local services and keeps a ledger.	`services` with `action: start`.
Services tool codes	Named results of the `services` tool, such as `port_in_use`, `health_failed`, `docker_unavailable`.	`health_failed` leaves the process up for a look.
Session	One run of the harness, from start to close.	One ticket, one session.
Session A / Session B	The two halves of a story. A: grill and write the contract. B: start blank and build from the ticket.	A ends at Ready for Development.
Session tools	`end_session`, `clear_context`, `compact_context`. The agent calls them when you ask in plain words.	"Close the session."
Severity	An advisor's level: blocker, concern, nit.	A blocker means fix it now.
SHA	A commit's long id.	A squash SHA names the merged commit.
Shell cap (300 s)	The shell tool kills a command after 300 seconds. `timeout` raises it. `async` does not.	Clean three or four folders per call.
Sibling (service)	Another service that the first one calls.	Sibling URLs are set with prefixed variable names.
Skill	A recipe file. Name and description sit in the prompt. The body loads on demand.	`skills/bug-workflow/SKILL.md`
Skill drift	Skills that contradict each other or the code.	The router skipped seven skills.

Term	Plain meaning	Example
Slash command	A short prompt template that starts with `/`.	`/bug 12345`
Slug	A short kebab-case name made from a title.	`add-retry-to-export-job`
Smoke test	A tiny test that shows the thing runs at all.	One case must fire. One must pass.
sourceVersion	The commit a build used.	The deployed commit is the build's `sourceVersion`.
Sprint	A fixed time box of planned work.	Stories enter at sprint planning.
Squash merge	Merge all PR commits into one commit on `main`. The PR title becomes the message.	Every repo here squashes.
SSO	Single sign-on: one account for many apps.	The web app signs in through SSO.
Stale (build)	A build that passed but is out of date, because `main` moved or a newer push has no run.	The policy build expired.
Standing exclusion	A repo the wiring script always skips.	`legacy-agent`, set in the `excluded_repos` array of `scripts/setup-task-branch.sh`.
Standing reviewer	A person added as an optional reviewer on every PR.	The tech lead.
Stash	Shelve uncommitted changes with `git stash`.	Never stash during a merge.
State ladder	The ordered list of states for a work item type.	Bug: New to Closed.
Stop condition	A named case that forces the implementer to stop, comment, and set Blocked.	The code contradicts a contract item.
Story (User Story)	A work item for new behaviour.	Story 12346.
Story states	The ladder: Backlog, In Refinement, Ready for Development, In Development, Ready for Test, In Test, Pending PO Review, Accepted. Side states: Blocked, Removed.	The agent never writes the last two rungs.
STORY-	The task folder prefix for a Story. Old notes say `US-`. They are stale.	`STORY-12346-memory-prompt-guard`
Stray	A loose file or folder in the tasks root that is not on the keep list.	A stray report folder.
Stream rule (TTSR)	A rule that fires while the model is still writing, by matching partial text. A tripwire.	The path rule that fired 37 times in four days.

Term	Plain meaning	Example
Subagent	A short-lived helper agent with a narrow brief and no shared memory.	One worker per repo.
Superseded	Replaced by a newer item. A decision page is `superseded`. A PR thread is superseded when its commenters approved later.	`status: superseded`
Symlink	A file that points to another path.	Repo entries in a task folder.
System prompt	The text the harness gives the model before your message.	`harness/global/APPEND_SYSTEM.md` adds to it.
System thread	A PR thread the system made, not a person. Skip it.	A vote update thread.
T-shirt size	A rough size: S, M, L. Used before design starts.	Size a new request before you design it.
Tag (git tag, work item tag)	A label. A git tag can trigger a deploy.	A `v*` tag.
Task folder	One folder per ticket with repo links, plan, docs, and review.	`BUG-12345-prompt-search-mismatch`
Tasks root	The one folder where you start every session.	`~/work/<PROJECT>/tasks`
TBD	"To be decided." Banned in an Agent Contract. The gate rejects it.	A contract with `TBD` fails Session B's gate.
TDD	Test-driven development: write the test first.	The pack's name for it is test first.
Teach-back	You explain the ticket in your own words. The agent compares.	The triage gate.
Teardown	Stop and clean up everything you started.	Close the browser. Stop the services.
Tech lead	The technical lead. A standing optional reviewer on every PR.	`<TECH_LEAD>` in the pack.
Tenant	An organisation's own space in a cloud login system.	The test user lives on the consultancy's tenant.
Terraform	A tool that builds cloud resources from text files.	`infra` is Terraform only.
Test first	Write the test, watch it fail, then fix.	Failing output goes in the plan.
Test user	A login made for tests, on a tenant you control.	The agent logs in as it by itself.
TF401181	The tracker's error for a PR that already completed.	You armed auto-complete too late.

Term	Plain meaning	Example
Thread (PR thread)	A chain of comments on a PR. Only the starter closes it.	Leave it Active after your fix.
Thread status	Active, Resolved, Won't Fix, Closed, By Design, or Pending.	Say Resolved, not `fixed`.
Tier (Big, Mid, Small)	Model size classes. Big for design, Mid for scoped work, Small for mechanical work.	A Small model runs flows.
Tool call	One use of a tool by the agent, such as `read` or `bash`.	A guard checks it before it runs.
toolApprovalConditions	A config block that sets allow, prompt, or deny per tool, with ordered rules. Global defaults merge with the project block.	Work-item writes fall back to allow.
TOTP	Time-based one-time password: six digits that change every 30 seconds.	`automation/playwright/totp.mjs`
Tracker	The system that holds tickets.	ADO.
Triage	Study a ticket, check it against code, and decide what to do.	`/triage 12345`
Trunk-based	Everyone merges small changes into `main`. Branches live short.	No long-lived feature branches.
TTSR	OMP's name for a stream rule. See Stream rule.	A rule that fires mid-stream.
Turn	One message from you and the agent's answer.	About 7.8 assistant messages per human turn.
Typecheck	A tool that checks types in code before it runs.	`pnpm run typecheck`
UAT	User acceptance testing: users try the product before release.	A UAT environment that reuses QA hides faults.
Unattended	Runs with no human watching.	The cron jobs.
Unit test	A test of one small piece of code.	A test file next to a component.
UTC	World clock time. The tracker stores times in UTC.	Pin a date range twice: UTC for the tracker, local for files.
uv	A fast Python package and project manager.	`uv sync`, `uv run pytest`
uvicorn, venv	A venv is a folder with one project's Python packages. Uvicorn runs a FastAPI app.	Launch through the venv's uvicorn, with no `--reload`.

Term	Plain meaning	Example
validated (flow)	A block the runner writes into a flow after run 1 returns broken. It proves the flow can see the bug.	No `validated` block, no `expect: fixed` run.
Vault	The personal wiki.	`vault://wiki/`
Vault URI	The `vault://wiki/<path>` address agents use for vault pages.	`vault://wiki/schema.md`
Verdict	A clear result line: clean or findings, pass or fail.	`VERDICT: clean`
Verification gate	The list of proofs that must pass before a PR opens.	Build, tests, live run, mutation check, self-review, security gate.
VM (virtual machine)	A computer that runs inside another computer.	The Docker VM must be up for the database container.
Vote	A reviewer's PR answer: approved, approved with suggestions, waiting for author, rejected.	Not merge authority.
Waiver	A written decision to accept a known gap, with a reason.	A false-positive class documented in a PR.
Wave	A group of tasks by dependency order. Same-wave tasks may run in parallel.	Wave 1 has no dependencies.
Wiki	A set of linked pages you keep up to date.	The vault.
Wikilink	`[[folder/page]]` : a link between vault pages.	`[[people/jane-doe]]`
WIQL	Work Item Query Language: SQL-like queries for the tracker.	`SELECT [System.Id] FROM WorkItems`
Work item	One tracker record: a Bug, a Story, a Task.	Bug 12345.
Worker	A subagent that implements. One per repo.	A worker gets a self-contained brief.
Worktree	A second checkout of a repo on its own branch, sharing history.	`<repo>-bare/fix/<slug>`
YAML	A text format for settings, using indentation.	`harness/global/config.yml` , `flow.yaml`